TUGAS BESAR 1 IF3070 DASAR INTELEGENSI ARTIFISIAL

Implementasi dan Analisis Algoritma Local Search untuk Optimasi Penataan Kendaraan pada Kapal Kargo

Presented by: 1stDAIBareng / K02



Riko Satriya G
18224108



Anisa Aulia A.
18224080



Nadine Arindy O.
18224012

BAB I
Deskripsi Persoalan

1.1 Latar Belakang Persoalan

Anda adalah seorang Forward Deployed Engineer (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

1.2 Spesifikasi

Tugas ini bertujuan untuk membuat konfigurasi penempatan kendaraan paling optimal pada sebuah geladak kapal kargo. Setiap kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi dapat memilih kendaraan mana yang diangkut dan menentukan posisi serta orientasinya di dalam geladak, dengan tetap memenuhi seluruh batasan.

1.2.1 Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Vehicle	id	“Owen”	*Identifier*
	Dimensions	{w:3, l:4}	Ukuran sebuah kendaraan, terdiri atas *width* dan *length* (satuan grid)
	Orientation	Horizontal	*Horizontal* atau *Vertical*

	ShippingFee	60	Tarif/keuntungan pengiriman kendaraan (satuan juta rupiah)
	Weight	5	Berat kendaraan (satuan unit)
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan (dalam sisa hari)
Ship	Dimensions	{w:10, l:20}	Ukuran geladak pada kapal, terdiri atas *width* dan *length* (satuan grid)
	MaxCapacity	500	Kapasitas angkut maksimum suatu kapal (satuan unit)

1.2.1 Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Kendaraan dan geladak kapal direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang.
- Kendaraan dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada kapal sehingga suatu kendaraan dapat berada di luar (seluruh bagian kendaraan di luar kapal)
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan key-value seperti JSON.
- Inisialisasi (initial state) random.
- Langkah neighbor (move) yang diperbolehkan tiap iterasi:
    1. Menukar dua kendaraan, baik di dalam maupun di luar kapal
    2. Memindahkan sebuah kendaraan ke suatu koordinat yang berbeda
    3. Mengubah orientasi kendaraan
- Representasi solusi (final state) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada kapal. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua kendaraan yang di dalam maupun yang di luar

2. Bagaimana keadaan geladak pada kapal
3. Value pada state tersebut

1.2.2 Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *ShippingFee* pada kapal.
- **Kendaraan tidak boleh saling bertumpuk (no overlapping)**
  Tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.
- **Kendaraan harus berada di dalam batas geladak.**
  Seluruh sel grid yang ditempati suatu kendaraan harus berada di dalam dimensi geladak kapal.
- **Kapal tidak boleh mengangkut melebihi kapasitasnya.**

1.3 Cakupan

Anda akan diminta untuk mengimplementasikan rencana yang telah Anda buat pada Tugas Besar 1. Berikut merupakan hal-hal yang perlu dilakukan oleh setiap kelompok.
Implementasikan 3 algoritma *local search* dengan rincian sebagai berikut:

1. Salah satu algoritma Hill-climbing
2. Simulated Annealing
3. Genetic Algorithm

Jalankan setiap algoritma (selain Genetic Algorithm) sebanyak 3 kali, kemudian catat beberapa hal berikut:

1. Berlaku untuk semua algoritma
    1. *State* awal dan akhir
    2. Nilai *objective function* akhir yang dicapai
    3. Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati
    4. Durasi proses pencarian
2. Berlaku hanya untuk Steepest Ascent Hill-Climbing dan Stochastic Hill-climbing
    1. Banyak iterasi hingga proses pencarian berhenti
3. Berlaku hanya untuk Hill-Climbing with Sideways Move
    1. Banyak iterasi hingga proses pencarian berhenti
    2. Tambahkan parameter *maximum sideways move*, dimana ketika banyak *sideways move* yang dilakukan sudah mencapai maksimum, pencarian dihentikan
4. Berlaku hanya untuk Random Restart Hill-climbing

1. Banyak *restart*
2. Banyak iterasi per *restart*
3. Tambahkan parameter *maximum restart*, dimana ketika banyak *restart* sudah mencapai maksimum, pencarian dihentikan.

5. Berlaku hanya untuk Simulated Annealing
    a. Plot terhadap banyak iterasi yang telah dilewati
    b. Frekuensi '*stuck*' di *local optimal*

Khusus untuk Genetic Algorithm, lakukan beberapa hal berikut:

1. Terdapat 2 parameter yang dapat diubah, yaitu jumlah populasi dan banyak iterasi.
2. Jadikan jumlah populasi sebagai kontrol, kemudian pilih 3 variasi banyak iterasi yang berbeda. Jalankan program sebanyak masing-masing 3 kali untuk setiap konfigurasi parameter.
3. Jadikan banyak iterasi sebagai kontrol, kemudian pilih 3 variasi jumlah populasi yang berbeda. Jalankan program sebanyak masing-masing 3 kali untuk setiap konfigurasi parameter.
4. Untuk setiap eksperimen, catat beberapa hal berikut:
    1. *State* awal dan akhir
    2. Nilai *objective function* akhir yang dicapai
    3. Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati (Cukup plot nilai *objective function* maksimum dan rata-rata dari populasi terhadap banyak iterasi yang telah dilewati.
    4. Jumlah populasi
    5. Banyak iterasi
    6. Durasi proses pencarian

Lakukan analisis terhadap hasil eksperimen, berikut merupakan beberapa pertanyaan yang dapat menjadi acuan untuk analisis yang Anda lakukan:

1. Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?
2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?
3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?
4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?
5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Anda boleh mengganti dan/atau menambahkan beberapa pertanyaan jika dirasa perlu untuk memudahkan penjelasan.

Program yang dibuat harus bisa memvisualisasikan *initial state*, *final state* dan juga hasil eksperimennya (sesuaikan informasi hasil eksperimen yang ditampilkan dengan ketentuan yang telah dijelaskan di poin sebelum ini).

Gunakan bahasa Python dalam implementasi *local search*. Semua bahasa dan *library* boleh digunakan untuk visualisasi (misalnya ingin *web based*). Cara visualisasi tidak memengaruhi penilaian dan dibebaskan kepada masing-masing kelompok sedemikian rupa untuk memudahkan penjelasan saat demo nanti.

Diperbolehkan untuk menggunakan heuristik yang Anda buat sendiri atau dari referensi lain untuk optimasi pencarian solusi, asalkan masih dalam lingkup *local search*. Jelaskan *heuristik* yang Anda pakai di laporan.

BAB II
Implementasi Algoritma Local Search

2.1 Kode Dasar

2.1.1 model.py

2.1.1.1 Class Vehicle

```
def __init__(self, id, w, l, ShippingFee, Weight, ETA):
    self.id = id
    self.w = w
    self.l = l
    self.ShippingFee = ShippingFee
    self.Weight = Weight
    self.ETA = ETA
```

Class Vehicle merepresentasikan satu kendaraan yang akan dimuat ke dalam kapal. Saat diinisialisasi, class ini menyimpan identitas kendaraan.

```
def dimensi(self):
    return self.w, self.l
```

Method dimensi mengembalikan ukuran kendaraan dalam bentuk pasangan (lebar, panjang) tanpa memperhitungkan rotasi.

2.1.1.2 Class Ship

```
def __init__(self, w, l, MaxCapacity):
    self.w = w
    self.l = l
    self.MaxCapacity = MaxCapacity
```

Class Ship merepresentasikan kapal sebagai area penempatan kendaraan. Saat diinisialisasi, class ini menyimpan lebar (w), panjang (l), dan kapasitas berat maksimum (MaxCapacity) kapal.

```
def dimensi(self):
    return self.w, self.l
```

Method dimensi mengembalikan ukuran kapal dalam bentuk pasangan (lebar, panjang), yang digunakan sebagai batas area penempatan kendaraan.

2.1.2 state.py

```
import random
```

2.1.2.1 Class Placement

```
def __init__(
    self,
    vehicle,
    Orientation="H",
    x=None,
    y=None,
    on_ship=False
):
    self.vehicle = vehicle
    self.Orientation = Orientation
    self.x = x
    self.y = y
    self.on_ship = on_ship
```

Menyimpan **penempatan satu kendaraan**, yaitu kendaraan, orientasi, posisi, dan status apakah sudah dimuat di kapal.

1. self.vehicle = vehicle: menyimpan objek Vehicle yang ditempatkan.
2. self.Orientation = Orientation: menyimpan orientasi, "H" (horizontal, default) atau "V" (vertikal, diputar 90°).
3. self.x = x dan self.y = y: menyimpan posisi pojok kendaraan di grid kapal. Default None karena belum ditempatkan.
4. self.on_ship = on_ship: menandai kendaraan dimuat atau tidak. Default False.

```
def effective_dimensions(self):
    if self.Orientation == "V":
        return self.vehicle.l, self.vehicle.w

    return self.vehicle.w, self.vehicle.l
```

Mengembalikan **dimensi kendaraan sesuai orientasinya** dalam bentuk (lebar, panjang).

1. Kalau orientasi "V", lebar dan panjang ditukar: (l, w).
2. Kalau tidak ("H"), dimensi dikembalikan apa adanya: (w, l).

```
def occupied_cells(self):
    if not self.on_ship:
        return []

    width, length = self.effective_dimensions()

    cells = []

    for dx in range(width):
        for dy in range(length):
            cells.append((self.x + dx, self.y + dy))

    return cells
```

Menghasilkan **daftar sel grid yang ditempati kendaraan**, dipakai untuk

pengecekan overlap.

1. Kalau kendaraan tidak di kapal (on_ship = False), langsung kembalikan list kosong. Ini yang mencegah false overlap di check_overlap.
2. Ambil dimensi efektif dengan effective_dimensions().
3. Siapkan list cells kosong.
4. Loop dx dari 0 sampai width - 1 dan dy dari 0 sampai length - 1.
5. Untuk tiap kombinasi, tambahkan sel (x + dx, y + dy) ke list.
6. Kembalikan cells, yaitu seluruh sel persegi panjang yang ditutupi kendaraan.

```
def copy(self):
    return Placement(
        self.vehicle,
        self.Orientation,
        self.x,
        self.y,
        self.on_ship
    )
```

Membuat **salinan placement** supaya perubahan posisi atau orientasi pada salinan tidak memengaruhi aslinya.

1. Objek Placement baru dibuat dengan nilai atribut yang sama.
2. vehicle tidak disalin (dipakai bersama) karena datanya tidak berubah selama pencarian.
3. Salinan dikembalikan dan dipakai oleh State.copy().

2.1.2.2 Class State

```
def __init__(self, ship, placements):
    self.ship = ship
    self.placements = placements
```

Menyimpan **kondisi penempatan**, yaitu kapal beserta daftar posisi semua kendaraan.

1. self.ship = ship: menyimpan objek kapal sebagai tempat penempatan.
2. self.placements = placements: menyimpan daftar Placement, yang masing-masing berisi kendaraan, orientasi, posisi (x, y), dan status on_ship.

```
def copy(self):
    return State(
        self.ship,
        [p.copy() for p in self.placements]
    )
```

Membuat **salinan state** supaya perubahan pada salinan tidak memengaruhi state asli.

1. Kapal dipakai ulang tanpa disalin (self.ship), karena tidak berubah selama pencarian.
2. Setiap placement disalin dengan p.copy() lewat list comprehension.
3. Kapal dan salinan placement dibungkus jadi objek State baru, lalu dikembalikan.

```
@classmethod
def random(cls, ship, vehicles, tries=20):
    from constraints import is_valid

    placements = [
        Placement(v, random.choice(["H", "V"])) for v in vehicles
    ]
    state = cls(ship, placements)

    order = list(range(len(vehicles)))
    random.shuffle(order)

    for i in order:
        p = state.placements[i]
        width, length = p.effective_dimensions()

        if width > ship.w or length > ship.l:
            continue

        for _ in range(tries):
            p.on_ship = True
            p.x = random.randint(0, ship.w - width)
            p.y = random.randint(0, ship.l - length)

            if is_valid(state):
                break

            p.on_ship, p.x, p.y = False, None, None

    return state
```

Membangun **state awal secara acak yang sudah valid**, dipakai sebagai titik mulai algoritma local search.

1. from constraints import is_valid: is_valid diimpor di dalam method untuk menghindari circular import.
2. Setiap kendaraan dibuat jadi Placement dengan orientasi acak, "H" atau "V", lalu dibungkus jadi state.
3. Urutan kendaraan diacak dengan random.shuffle supaya tidak ada kendaraan yang selalu dapat giliran pertama.
4. Untuk tiap kendaraan, ambil dimensi efektif sesuai orientasi dengan effective_dimensions().
5. Kalau lebar atau panjang melebihi kapal, kendaraan dilewati (continue) dan tidak dimuat.
6. Untuk tiap kendaraan, coba maksimal tries kali: set on_ship = True,

lalu pilih x dan y acak yang masih di dalam batas kapal.
7. Kalau is_valid(state) lolos (tidak overlap dan tidak melebihi kapasitas), posisi dipakai dan loop berhenti (break).
8. Kalau percobaan gagal, kendaraan dikembalikan ke on_ship = False dengan x, y = None.
9. Setelah semua kendaraan diproses, state dikembalikan. Hasilnya valid, tapi tidak semua kendaraan pasti terpasang.

2.1.3 constraints.py

```
def check_within_ship(placement, ship):
  if not placement.on_ship:
    return True

  width, length = placement.effective_dimensions()

  return (
    placement.x >= 0
    and placement.y >= 0
    and placement.x + width <= ship.w
    and placement.y + length <= ship.l
  )
```

Memeriksa apakah satu kendaraan **masih di dalam batas kapal**.

1. if not placement.on_ship: return True: kalau kendaraan nggak ditaruh di kapal (misal di-skip atau di luar), otomatis dianggap valid karena nggak ada yang perlu dicek.
2. width, length = placement.effective_dimensions(): ambil dimensi efektif kendaraan. Kemungkinan ini memperhitungkan rotasi (kalau diputar 90°, width dan length tertukar).
3. Return True kalau semua kondisi terpenuhi:
    - x >= 0 dan y >= 0: titik pojok kiri-atas tidak keluar dari sisi kiri/atas kapal.
    - x + width <= ship.w: sisi kanan kendaraan tidak melewati lebar kapal.
    - y + length <= ship.l: sisi bawah kendaraan tidak melewati panjang kapal.

```
def check_overlap(state):
  all_cells = set()

  for placement in state.placements:
    for cell in placement.occupied_cells():

      if cell in all_cells:
        return False

      all_cells.add(cell)
```

```
  return True
```

Memeriksa apakah ada **dua kendaraan yang menempati sel yang sama**.

1. all_cells = set(): wadah untuk semua sel grid yang sudah terpakai.
2. Loop tiap placement, lalu loop tiap sel dari placement.occupied_cells().
3. Kalau sel itu sudah ada di all_cells, berarti ada kendaraan lain yang sudah menempati sel tersebut, jadi langsung return False.
4. Kalau belum, sel dimasukkan ke set.
5. Kalau semua sel unik sampai akhir, return True.

```
def check_capacity(state):
  total_weight = 0

  for placement in state.placements:
    if placement.on_ship:
      total_weight += placement.vehicle.Weight

  return total_weight <= state.ship.MaxCapacity
```

Memeriksa **total berat kendaraan di kapal tidak melebihi kapasitas**.

1. total_weight = 0, lalu loop semua placement.
2. Hanya kendaraan yang on_ship yang beratnya dijumlahkan (placement.vehicle.Weight).
3. Return True kalau total_weight <= state.ship.MaxCapacity.

```
def is_valid(state):
  for placement in state.placements:
    if not check_within_ship(placement, state.ship):
      return False

  if not check_overlap(state):
    return False

  if not check_capacity(state):
    return False

  return True
```

Fungsi utama yang **menggabungkan semua constraint**. State dianggap valid hanya kalau:

1. Semua kendaraan berada dalam batas kapal (loop check_within_ship per placement, langsung False kalau ada yang gagal).
2. Tidak ada overlap (check_overlap).
3. Berat total tidak melebihi kapasitas (check_capacity).

2.1.4 objectives.py

```
def calculate_objective(state):
    total = 0

    for placement in state.placements:
        if placement.on_ship:
            total += placement.vehicle.ShippingFee

    return total
```

Menghitung **nilai objective dari sebuah state**, yaitu total biaya pengiriman semua kendaraan yang berhasil dimuat di kapal. Semakin besar nilainya, semakin baik solusinya.

1. total = 0: menyiapkan variabel penampung total pendapatan, mulai dari nol.
2. Loop semua placement di state.placements.
3. Hanya placement dengan on_ship = True yang diproses, karena kendaraan yang tidak dimuat tidak menghasilkan pendapatan.
4. Untuk tiap kendaraan yang dimuat, placement.vehicle.ShippingFee ditambahkan ke total.
5. Setelah semua placement diperiksa, total dikembalikan sebagai nilai objective.

2.1.5 visualisasi.py

```
import os
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from objectives import calculate_objective

BACKGROUND = "#fcfcfb"
PRIMARY = "#0b0b0b"
GRID = "#e1e0d9"
CATEGORY = [
    "#196cb1",
    "#d63230",
    "#1c77c3",
    "#39a9db",
    "#40bcd8",
    "#c79d60",
    "#f39237",
    "#e56234",
]
```

Menyiapkan **library dan konfigurasi warna** untuk seluruh visualisasi.

1. os dipakai untuk membuat folder penyimpanan gambar.
2. matplotlib.pyplot dan patches dipakai untuk membuat grafik dan menggambar kotak kendaraan.

3. calculate_objective diimpor untuk menampilkan nilai objective di panel teks.
4. BACKGROUND, PRIMARY, dan GRID adalah warna latar, teks/garis tepi, dan garis grid.
5. CATEGORY adalah palet 8 warna yang dipakai bergantian untuk membedakan kendaraan dan garis pada grafik.

```
def plot_state(
    state,
    title="",
    save_path=None,
    iterations=None,
    duration=None,
    algorithm=None,
    max_sideways=None,
    max_restarts=None,
    stuck_frequency=None,
    population_size=None,
    iteration=None,
    **kwargs,
):
    ship = state.ship

    fig, (ax_grid, ax_text) = plt.subplots(
        1, 2, figsize=(11, 6), gridspec_kw={"width_ratios": [2.2, 1]}
    )
    fig.patch.set_facecolor(BACKGROUND)

    ax_grid.set_facecolor(BACKGROUND)
    ax_grid.set_xlim(0, ship.w)
    ax_grid.set_ylim(0, ship.l)
    ax_grid.set_aspect("equal")
    ax_grid.invert_yaxis()
    ax_grid.set_xticks(range(ship.w + 1))
    ax_grid.set_yticks(range(ship.l + 1))
    ax_grid.grid(True, color=GRID, linewidth=0.8)

    inside = []
    outside = []

    for i, placement in enumerate(state.placements):
        vehicle = placement.vehicle
        if not placement.on_ship:
            outside.append(vehicle.id)
            continue

        inside.append(vehicle.id)
        w, l = placement.effective_dimensions()
        color = CATEGORY[i % len(CATEGORY)]

        box = patches.Rectangle(
            (placement.x, placement.y),
            w,
```

```
            l,
            facecolor=color,
            edgecolor=PRIMARY,
            linewidth=1,
            alpha=0.85,
        )
        ax_grid.add_patch(box)
        ax_grid.text(
            placement.x + w / 2,
            placement.y + l / 2,
            vehicle.id,
            ha="center",
            va="center",
            fontsize=8,
            color=PRIMARY,
            fontweight="bold",
        )

    # alternatif
    if iterations is None:
        iterations = iteration if iteration is not None else kwargs.get("iterasi", kwargs.get("iteration",
None))
    if duration is None:
        duration = kwargs.get("durasi", None)
    if max_sideways is None:
        max_sideways = kwargs.get("max_sideway", None)
    if max_restarts is None:
        max_restarts = kwargs.get("max_restart", kwargs.get("restarts", None))
    if stuck_frequency is None:
        stuck_frequency = kwargs.get("stuck_count", kwargs.get("frekuensi_stuck", None))
    if population_size is None:
        population_size = kwargs.get("populasi", kwargs.get("population", None))

    value = calculate_objective(state)
    total_weight = sum(p.vehicle.Weight for p in state.placements if p.on_ship)

    inside_str = ", ".join(inside) if inside else "None"
    outside_str = ", ".join(outside) if outside else "None"

    if iterations is not None:
        try:
            iteration_str = str(int(iterations))
        except (ValueError, TypeError):
            iteration_str = str(iterations)
    else:
        iteration_str = "-"

    if duration is not None:
        if isinstance(duration, (int, float)):
            duration_str = f"{duration:.3f}s"
        else:
            duration_str = str(duration)
    else:
        duration_str = "-"
```

```
    info_lines = [
        f"Objective: {value}",
        f"Weight: {total_weight} / {ship.MaxCapacity}",
        f"Vehicle Inside: {len(inside)} ({inside_str})",
        f"Vehicle Outside: {len(outside)} ({outside_str})",
        f"Iteration: {iteration_str}",
        f"Duration: {duration_str}",
    ]

    algo = str(algorithm).lower().strip() if algorithm else ""

    if "sideways" in algo or max_sideways is not None:
        val = max_sideways if max_sideways is not None else "-"
        info_lines.append(f"Maximum sideways move: {val}")

    if ("restart" in algo or "rr" in algo) or max_restarts is not None:
        val = max_restarts if max_restarts is not None else "-"
        info_lines.append(f"maximum restart: {val}")

    if ("anneal" in algo or algo == "sa") or stuck_frequency is not None:
        val = stuck_frequency if stuck_frequency is not None else "-"
        info_lines.append(f"stuck frequency: {val}")

    if ("gene" in algo or algo == "ga") or population_size is not None:
        val = population_size if population_size is not None else "-"
        info_lines.append(f"jumlah populasi: {val}")

    info = "\n\n".join(info_lines)

    ax_text.axis("off")
    ax_text.text(0, 1, info, va="top", ha="left", fontsize=9.5, color=PRIMARY, wrap=True)

    if title:
        fig.suptitle(title, color=PRIMARY)
    fig.tight_layout()

    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        fig.savefig(save_path, dpi=120, facecolor=BACKGROUND, bbox_inches="tight")

    plt.close(fig)
    return fig
```

Menggambar **kondisi penempatan kendaraan di kapal** beserta ringkasan nilainya.

1. Membuat figure dengan dua panel: grid kapal di kiri dan panel teks di kanan (rasio lebar 2.2 : 1).
2. Panel grid diatur seukuran kapal (xlim = ship.w, ylim = ship.l), skala sumbu sama (equal), dan sumbu y dibalik (invert_yaxis) supaya titik (0, 0) ada di pojok kiri atas, sesuai sistem koordinat Placement.
3. Tiap kendaraan dicek. Yang on_ship = False dicatat idnya ke list

outside dan tidak digambar.
4. Kendaraan yang di kapal digambar sebagai Rectangle di (x, y) dengan ukuran dari effective_dimensions(), warna diambil dari CATEGORY secara bergantian, lalu id kendaraan ditulis di tengah kotak.
5. Dihitung nilai objective, total berat kendaraan di kapal, dan status is_valid.
6. Ketiganya, bersama daftar kendaraan yang tidak dimuat, ditulis di panel kanan (sumbu dimatikan dengan axis("off")).
7. Kalau title diisi, dipasang sebagai judul. Kalau save_path diisi, folder dibuat otomatis lalu gambar disimpan (dpi 120).
8. Figure ditutup dengan plt.close(fig) supaya tidak menumpuk di memori, lalu dikembalikan.

```
def plot_value(value_history, title="Value vs Iteration", save_path=None):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    fig.patch.set_facecolor(BACKGROUND)
    ax.set_facecolor(BACKGROUND)
    ax.grid(True, color=GRID, linewidth=0.8)

    ax.plot(range(len(value_history)), value_history, color=CATEGORY[0],
linewidth=1.5)
    ax.set_xlabel("iterasi")
    ax.set_ylabel("nilai objektif")
    ax.set_title(title, color=PRIMARY)
    fig.tight_layout()

    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        fig.savefig(save_path, dpi=120, facecolor=BACKGROUND)

    plt.close(fig)
    return fig
```

Menggambar **grafik nilai objective terhadap iterasi** untuk satu proses pencarian.

1. Membuat satu figure dengan latar dan grid sesuai konfigurasi warna.
2. Memplot value_history terhadap indeks iterasi sebagai satu garis.
3. Sumbu x diberi label “iterasi”, sumbu y “nilai objektif”, lalu judul dipasang.
4. Kalau save_path diisi, gambar disimpan. Figure lalu ditutup dan dikembalikan.

```
def plot_value_prob(
    value_history,
    prob_history=None,
    title="Simulated Annealing: Nilai Objektif & e^(ΔE/T) vs Iterasi",
```

```
    save_path=None,
):
    if prob_history is None and value_history and isinstance(value_history[0],
(tuple, list)):
        prob_history = value_history
        value_history = []

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6.5), sharex=True)
    fig.patch.set_facecolor(BACKGROUND)

    # Nilai Objektif
    ax1.set_facecolor(BACKGROUND)
    ax1.grid(True, color=GRID, linewidth=0.8)
    if value_history:
        ax1.plot(
            range(len(value_history)),
            value_history,
            color=CATEGORY[0],
            linewidth=1.5,
            label="Nilai Objektif",
        )
        ax1.legend(loc="lower right")
    ax1.set_ylabel("nilai objektif")
    ax1.set_title(title, color=PRIMARY)

    # Subplot 2 buat si e^(ΔE/T)
    ax2.set_facecolor(BACKGROUND)
    ax2.grid(True, color=GRID, linewidth=0.8)
    if prob_history:
        if isinstance(prob_history[0], (tuple, list)):
            x = [item[0] for item in prob_history]
            y = [item[1] for item in prob_history]
        else:
            x = list(range(len(prob_history)))
            y = prob_history

        ax2.scatter(
            x,
            y,
            color=CATEGORY[1],
            s=6,
            alpha=0.4,
            edgecolors="none",
            label="e^(ΔE/T)",
        )
        ax2.legend(loc="upper right")
```

```
    ax2.set_xlabel("iterasi")
    ax2.set_ylabel("e^(ΔE/T)")
    ax2.set_ylim(-0.05, 1.05)

    fig.tight_layout()

    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        fig.savefig(save_path, dpi=120, facecolor=BACKGROUND)

    plt.close(fig)
    return fig
```

Menggambar **dua grafik bertumpuk untuk Simulated Annealing**: nilai objective dan probabilitas penerimaan e^(ΔE/T) terhadap iterasi.

1. Kalau prob_history kosong dan isi value_history ternyata berupa tuple/list, argumen dianggap salah posisi: value_history dipindah jadi prob_history, lalu value_history dikosongkan.
2. Membuat dua subplot atas-bawah dengan sumbu x yang dibagi (sharex=True).
3. Subplot atas memplot nilai objective sebagai garis lengkap dengan legend, kalau datanya ada.
4. Subplot bawah memplot e^(ΔE/T) sebagai scatter titik kecil semi-transparan. Data boleh berupa pasangan (iterasi, probabilitas) atau list probabilitas biasa, yang dalam kasus ini x-nya diambil dari indeks.
5. Sumbu y subplot bawah dibatasi -0.05 sampai 1.05 karena nilai probabilitas ada di rentang 0 sampai 1.
6. Gambar disimpan kalau save_path diisi, lalu figure ditutup dan dikembalikan.

```
def plot_value_ga(max_history, avg_history, title="GA: Objective vs Iteration", save_path=None):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    fig.patch.set_facecolor(BACKGROUND)
    ax.set_facecolor(BACKGROUND)
    ax.grid(True, color=GRID, linewidth=0.8)

    ax.plot(range(len(max_history)), max_history, color=CATEGORY[0], linewidth=1.5, label="Maksimum")
    ax.plot(range(len(avg_history)), avg_history, color=CATEGORY[1], linewidth=1.5, label="Rata-rata")
    ax.set_xlabel("iterasi")
    ax.set_ylabel("nilai objektif")
    ax.set_title(title, color=PRIMARY)
    ax.legend()
```

```
    fig.tight_layout()

    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        fig.savefig(save_path, dpi=120, facecolor=BACKGROUND)

    plt.close(fig)
    return fig
```

Menggambar **perkembangan nilai objective populasi pada Genetic Algorithm**.

1. Membuat satu figure dengan latar dan grid.
2. Memplot max_history (nilai terbaik per generasi) sebagai garis berwarna biru berlabel “Maksimum”.
3. Memplot avg_history (rata-rata populasi per generasi) sebagai garis berwarna merah berlabel “Rata-rata”.
4. Memasang label sumbu, judul, dan legend.
5. Menyimpan gambar kalau save_path diisi, lalu menutup dan mengembalikan figure.

```
def plot_value_rr(
    value_history,
    best_history=None,
    restart_indices=None,
    title="Random Restart: Objective vs Iterasi",
    save_path=None,
):
    fig, ax = plt.subplots(figsize=(7, 4.5))
    fig.patch.set_facecolor(BACKGROUND)
    ax.set_facecolor(BACKGROUND)
    ax.grid(True, color=GRID, linewidth=0.8)

    ax.plot(
        range(len(value_history)),
        value_history,
        color=CATEGORY[0],
        linewidth=1.2,
        alpha=0.75,
        label="Nilai Objektif",
    )
    if best_history:
        ax.plot(
            range(len(best_history)),
            best_history,
            color=CATEGORY[1],
            linewidth=1.8,
```

```
        label="Nilai Terbaik (Best)",
    )

  if restart_indices and len(restart_indices) > 1:
    first = True
    for idx in restart_indices[1:]:
      label = "Restart" if first else None
      ax.axvline(
        x=idx,
        color="#898781",
        linestyle="--",
        linewidth=0.9,
        alpha=0.6,
        label=label,
      )
      first = False

  ax.set_xlabel("iterasi")
  ax.set_ylabel("nilai objektif")
  ax.set_title(title, color=PRIMARY)
  ax.legend(loc="lower right")
  fig.tight_layout()

  if save_path:
    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    fig.savefig(save_path, dpi=120, facecolor=BACKGROUND)

  plt.close(fig)
  return fig
```

Menggambar **grafik Random Restart Hill Climbing** lengkap dengan penanda titik restart.

1. Memplot value_history sebagai garis tipis semi-transparan (nilai objective di tiap iterasi).
2. Kalau best_history ada, memplot nilai terbaik sejauh ini sebagai garis lebih tebal.
3. Kalau restart_indices punya lebih dari satu elemen, digambar garis vertikal putus-putus di tiap indeks restart, kecuali yang pertama karena itu awal proses, bukan restart.
4. Label “Restart” hanya dipasang di garis vertikal pertama supaya legend tidak berulang.
5. Memasang label sumbu, judul, dan legend, lalu menyimpan gambar kalau save_path diisi. Figure ditutup dan dikembalikan.

2.2 Algoritma *Local Search*

2.2.1 *Steepest Ascent Hill Climbing Algorithm*

2.2.1.1 steepest_ascent.py

```
import random
import time

from neighbors import generate_neighbors
from objectives import calculate_objective
```

Menyiapkan **library dan modul pendukung** untuk Steepest Ascent Hill Climbing.

1. random dipakai untuk memilih satu neighbor secara acak saat ada beberapa neighbor terbaik yang nilainya sama.
2. time dipakai untuk mengukur durasi eksekusi.
3. generate_neighbors dipakai untuk membangkitkan seluruh neighbor dari state saat ini.
4. calculate_objective dipakai untuk menghitung nilai tiap state.

```
def hill_climbing_steepest_ascent(state, max_iterations=1000):
    start = time.time()

    current = state
    current_value = calculate_objective(current)

    value_history = [current_value]
    iteration = 0
    stop_reason = "max_iterations"
```

Menyiapkan **kondisi awal pencarian**.

1. Waktu mulai dicatat dengan time.time().
2. State awal dijadikan current, lalu nilai objective-nya dihitung.
3. value_history diisi nilai awal dan akan bertambah tiap iterasi, dipakai oleh plot_value.
4. iteration dimulai dari 0.
5. stop_reason diberi nilai awal "max_iterations", yang akan diganti kalau pencarian berhenti karena alasan lain.

```
    while iteration < max_iterations:
        neighbors = generate_neighbors(current)

        if not neighbors:
            stop_reason = "no_neighbor"
            break
```

```
    values = [calculate_objective(n) for n in neighbors]
    best_value = max(values)

    if best_value <= current_value:
        stop_reason = "local_optimum"
        break

    best_neighbors = [n for n, v in zip(neighbors, values) if v == best_value]
    current = random.choice(best_neighbors)
    current_value = best_value

    value_history.append(current_value)
    iteration += 1
```

Melakukan **pencarian iteratif** dengan selalu berpindah ke neighbor terbaik, selama masih ada perbaikan.

1. Perulangan berjalan selama iteration belum mencapai max_iterations.
2. Seluruh neighbor dari current dibangkitkan dengan generate_neighbors.
3. Kalau tidak ada neighbor, pencarian berhenti dengan alasan "no_neighbor".
4. Nilai objective semua neighbor dihitung, lalu diambil nilai terbesarnya (best_value).
5. Kalau best_value tidak lebih besar dari current_value, berarti tidak ada perbaikan. Pencarian berhenti dengan alasan "local_optimum".
6. Kalau ada perbaikan, dikumpulkan semua neighbor yang bernilai sama dengan best_value (best_neighbors).
7. Satu dari neighbor terbaik dipilih secara acak sebagai current baru, lalu current_value diperbarui. Pemilihan acak ini memecah kasus seri.
8. Nilai baru ditambahkan ke value_history dan iteration bertambah satu.

```
  return {
      "initial_state": state,
      "final_state": current,
      "final_value": current_value,
      "value_history": value_history,
      "iterations": iteration,
      "stop_reason": stop_reason,
      "duration": time.time() - start,
  }
```

Mengembalikan **hasil pencarian dalam bentuk dictionary**.

1. initial_state dan final_state berisi state awal dan state hasil akhir.
2. final_value adalah nilai objective akhir.
3. value_history berisi riwayat nilai tiap iterasi.
4. iterations adalah jumlah iterasi yang benar-benar dijalankan.

5. stop_reason menjelaskan kenapa pencarian berhenti: max_iterations, no_neighbor, atau local_optimum.
6. duration adalah lama eksekusi dalam detik.

```
steepest_ascent = hill_climbing_steepest_ascent
```

Membuat **alias nama pendek** steepest_ascent untuk fungsi di atas, supaya bisa diimpor dengan nama yang lebih singkat (misalnya di exp_steepest_ascent.py).

2.2.1.2 exp_steepest_ascent.py

```
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from algorithms.steepest_ascent import hill_climbing_steepest_ascent
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data", "data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "output",
"steepest_ascent")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Steepest Ascent.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. hill_climbing_steepest_ascent, load_data, calculate_objective, serta plot_state dan plot_value diimpor sebagai komponen eksperimen. State tidak diimpor karena initial state diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/steepest_ascent sebagai tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
```

```
jumlah_percobaan = 3

print("\n--- Eksperimen Hill-Climbing Steepest Ascent ---")
```

Menyiapkan **data dan penampung hasil** sebelum eksperimen dijalankan.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. load_data membaca data.json menjadi objek kapal (ship) dan daftar kendaraan (vehicles).
3. Folder output dibuat kalau belum ada.
4. rekap disiapkan sebagai penampung hasil semua percobaan.
5. jumlah_percobaan = 3 berarti eksperimen diulang tiga kali dengan state awal acak berbeda.
6. Judul eksperimen dicetak ke layar.

```
for i in range(1, jumlah_percobaan + 1):
    initial_state = initial_states[i - 1].copy()
    initial_value = calculate_objective(initial_state)

    hasil = hill_climbing_steepest_ascent(initial_state.copy())

    final_state = hasil["final_state"]
    final_value = hasil["final_value"]
    durasi = hasil["duration"]
    iterasi = hasil["iterations"]
    stop_reason = hasil["stop_reason"]
    history = hasil["value_history"]
```

Menjalankan **Steepest Ascent pada tiap percobaan**.

1. Perulangan berjalan dari percobaan 1 sampai jumlah_percobaan.
2. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Nilai objective-nya dihitung sebagai nilai_awal.
3. hill_climbing_steepest_ascent dijalankan dengan salinan initial state (batas iterasi memakai default 1000).
4. Dari dictionary hasil diambil state akhir, nilai akhir, durasi, jumlah iterasi, alasan berhenti, dan riwayat nilai.

```
    plot_state(initial_state, title=f"Percobaan {i} - Initial State",
save_path=path_init)
    plot_state(
        final_state,
        title=f"Percobaan {i} - Final State",
        save_path=path_final,
        iterations=hasil["iterations"],
```

```
        duration=hasil["duration"],
    )
    plot_value(history, title=f"Percobaan {i} - Objective vs Iterasi",
save_path=path_plot)
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state menggambar penempatan kendaraan pada kondisi awal, lalu menyimpannya.
3. plot_state untuk final state menerima iterations dan duration, sehingga panel teks menampilkan jumlah iterasi dan durasi. Tidak ada baris parameter tambahan karena algoritma ini tidak punya parameter khusus.
4. plot_value menggambar grafik nilai objective terhadap iterasi dari value_history.

```
    rekap.append({
        "percobaan": i,
        "nilai_awal": initial_value,
        "nilai_akhir": final_value,
        "durasi_detik": f"{durasi:.4f}",
        "iterasi": iterasi,
        "stop_reason": stop_reason,
        "state_awal_img": f"percobaan_{i}_initial.png",
        "state_akhir_img": f"percobaan_{i}_final.png",
        "plot_obj_img": f"percobaan_{i}_objective.png",
    })

    print(
        f"Percobaan {i}: awal={initial_value}, akhir={final_value}, "
        f"iterasi={iterasi}, berhenti={stop_reason}, waktu={durasi:.3f}s"
    )
```

Mencatat **hasil tiap percobaan** ke rekap dan ke layar.

1. Satu baris dictionary ditambahkan ke rekap, berisi nomor percobaan, nilai awal dan akhir, durasi (4 desimal), jumlah iterasi, alasan berhenti, dan nama file ketiga gambar.
2. Ringkasan percobaan dicetak ke terminal supaya kemajuan eksperimen terlihat.

```
csv_path = os.path.join(OUTPUT_DIR, "rekap_steepest_ascent.csv")
fieldnames = [
    "percobaan",
    "nilai_awal",
    "nilai_akhir",
    "durasi_detik",
```

```
        "iterasi",
        "stop_reason",
        "state_awal_img",
        "state_akhir_img",
        "plot_obj_img",
    ]
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rekap)

    print(f"Rekap disimpan ke {csv_path}")
```

Menyimpan **seluruh rekap eksperimen ke file CSV**.

1. Path file ditentukan: rekap_steepest_ascent.csv di folder output.
2. fieldnames menentukan urutan dan nama kolom.
3. File dibuka dengan newline="" dan encoding UTF-8 supaya tidak muncul baris kosong ekstra di Windows.
4. Header ditulis, lalu semua baris rekap ditulis sekaligus.
5. Lokasi file dicetak ke terminal.

```
if __name__ == "__main__":
    main()
```

Menjalankan **main() hanya kalau file dieksekusi langsung** (python exp_steepest_ascent.py), bukan saat diimpor sebagai modul.

2.2.2 *Sideways Move Hill Climbing Algorithm*

2.2.2.1 sideways_move.py

```
import random
import time

from neighbors import generate_neighbors
from objectives import calculate_objective
```

Menyiapkan **library dan modul pendukung** untuk Hill Climbing dengan Sideways Move.

1. random dipakai untuk memilih satu neighbor secara acak saat ada beberapa neighbor terbaik yang nilainya sama.
2. time dipakai untuk mengukur durasi eksekusi.
3. generate_neighbors dipakai untuk membangkitkan seluruh neighbor dari state saat ini.
4. calculate_objective dipakai untuk menghitung nilai tiap state.

```
def hill_climbing_sideways(state, max_sideways=100,
max_iterations=1000):
    start = time.time()

    current = state
    current_value = calculate_objective(current)

    value_history = [current_value]
    iteration = 0
    sideways_count = 0
    stop_reason = "max_iterations"
```

Menyiapkan **kondisi awal pencarian**.

1. Waktu mulai dicatat dengan time.time().
2. State awal dijadikan current, lalu nilai objective-nya dihitung.
3. value_history diisi nilai awal dan akan bertambah tiap iterasi, dipakai oleh plot_value.
4. iteration dan sideways_count dimulai dari 0. sideways_count menghitung berapa kali algoritma berpindah ke state bernilai sama.
5. stop_reason diberi nilai awal "max_iterations", yang akan diganti kalau pencarian berhenti karena alasan lain.
6. Parameter max_sideways membatasi jumlah sideways move berturut-turut yang diizinkan (default 100), supaya tidak berputar selamanya di plateau.

```
  while iteration < max_iterations:
        neighbors = generate_neighbors(current)

        if not neighbors:
            stop_reason = "no_neighbor"
            break

        values = [calculate_objective(n) for n in neighbors]
        best_value = max(values)

        if best_value < current_value:
            stop_reason = "local_optimum"
            break

        if best_value == current_value:
            if sideways_count >= max_sideways:
                stop_reason = "max_sideways"
                break
            sideways_count += 1

        best_neighbors = [n for n, v in zip(neighbors, values) if v ==
best_value]
    current = random.choice(best_neighbors)
        current_value = best_value
```

```
        value_history.append(current_value)
        iteration += 1
```

Melakukan **pencarian iteratif** yang boleh berpindah ke neighbor bernilai sama (sideways) dengan batas tertentu.

1. Perulangan berjalan selama iteration belum mencapai max_iterations.
2. Seluruh neighbor dari current dibangkitkan dengan generate_neighbors.
3. Kalau tidak ada neighbor, pencarian berhenti dengan alasan "no_neighbor".
4. Nilai objective semua neighbor dihitung, lalu diambil nilai terbesarnya (best_value).
5. Kalau best_value lebih kecil dari current_value, tidak ada neighbor yang sama baik atau lebih baik. Pencarian berhenti dengan alasan "local_optimum".
6. Kalau best_value sama dengan current_value, berarti terjadi sideways move. Kalau sideways_count sudah mencapai max_sideways, pencarian berhenti dengan alasan "max_sideways". Kalau belum, sideways_count bertambah satu.
7. Dikumpulkan semua neighbor yang bernilai sama dengan best_value (best_neighbors), lalu satu dipilih secara acak sebagai current baru.
8. current_value diperbarui, nilai baru ditambahkan ke value_history, dan iteration bertambah satu.

```
    return {
         "initial_state": state,
         "final_state": current,
         "final_value": current_value,
         "value_history": value_history,
         "iterations": iteration,
         "sideways_moves": sideways_count,
         "stop_reason": stop_reason,
         "duration": time.time() - start,
     }
```

Mengembalikan **hasil pencarian dalam bentuk dictionary**.

1. initial_state dan final_state berisi state awal dan state hasil akhir.
2. final_value adalah nilai objective akhir.
3. value_history berisi riwayat nilai tiap iterasi.
4. iterations adalah jumlah iterasi yang benar-benar dijalankan.
5. sideways_moves adalah total sideways move yang dilakukan.
6. stop_reason menjelaskan kenapa pencarian berhenti: max_iterations, no_neighbor, local_optimum, atau max_sideways.
7. duration adalah lama eksekusi dalam detik.

2.2.2.2 exp_sideways.py

```
import csv
import os
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
"..")))

from algorithms.sideways_move import hill_climbing_sideways
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data",
"data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..",
"output", "sideways")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Sideways Move.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. hill_climbing_sideways, load_data, calculate_objective, serta plot_state dan plot_value diimpor sebagai komponen eksperimen. State tidak diimpor karena initial state diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/sideways sebagai tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    inp = input("Masukkan maximum sideways move (default 100):
").strip()
    try:
        max_sideways = int(inp) if inp else 100
    except ValueError:
        print("Input tidak valid, menggunakan default 100.")
        max_sideways = 100
```

Membaca **batas maksimum sideways move dari pengguna**.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. Program meminta nilai max_sideways lewat terminal.
3. Input kosong memakai default 100.
4. Kalau input tidak bisa diubah ke angka (ValueError), pesan ditampilkan

dan nilai dikembalikan ke 100.

```
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
    jumlah_percobaan = 3

    print(f"\n--- Eksperimen Hill-Climbing Sideways Move
(max_sideways={max_sideways}) ---")
```

Menyiapkan **data dan penampung hasil** sebelum eksperimen dijalankan.

1. load_data membaca data.json menjadi objek kapal (ship) dan daftar kendaraan (vehicles).
2. Folder output dibuat kalau belum ada.
3. rekap disiapkan sebagai penampung hasil semua percobaan.
4. jumlah_percobaan = 3 berarti eksperimen diulang tiga kali dengan state awal acak berbeda.
5. Judul eksperimen beserta nilai max_sideways dicetak ke layar.

```
  for i in range(1, jumlah_percobaan + 1):
        initial_state = initial_states[i - 1].copy()
        initial_value = calculate_objective(initial_state)

        hasil = hill_climbing_sideways(initial_state.copy(),
max_sideways=max_sideways, max_iterations=1000)

        final_state = hasil["final_state"]
        final_value = hasil["final_value"]
        durasi = hasil["duration"]
        iterasi = hasil["iterations"]
        sideways = hasil["sideways_moves"]
        stop_reason = hasil["stop_reason"]
        history = hasil["value_history"]
```

Menjalankan **Hill Climbing Sideways Move pada tiap percobaan**.

1. Perulangan berjalan dari percobaan 1 sampai jumlah_percobaan.
2. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Nilai objective-nya dihitung sebagai nilai_awal.
3. hill_climbing_sideways dijalankan dengan salinan initial state, max_sideways dari input, dan max_iterations=1000.
4. Dari dictionary hasil diambil state akhir, nilai akhir, durasi, jumlah iterasi, jumlah sideways move, alasan berhenti, dan riwayat nilai.

```
    plot_state(initial_state, title=f"Percobaan {i} - Initial State",
save_path=path_init)
    plot_state(
      final_state,
```

```
        title=f"Percobaan {i} - Final State",
        save_path=path_final,
        iterations=hasil["iterations"],
        duration=hasil["duration"],
        algorithm="sideways move",
        max_sideways=max_sideways,
    )
    plot_value(history, title=f"Percobaan {i} - Objective vs Iterasi",
save_path=path_plot)
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state menggambar penempatan kendaraan pada kondisi awal, lalu menyimpannya.
3. plot_state untuk final state menerima iterations, duration, algorithm="sideways move", dan max_sideways, sehingga panel teks menampilkan jumlah iterasi, durasi, dan baris Maximum sideways move.
4. plot_value menggambar grafik nilai objective terhadap iterasi dari value_history.

```
    rekap.append({
        "percobaan": i,
        "max_sideways": max_sideways,
        "nilai_awal": initial_value,
        "nilai_akhir": final_value,
        "durasi_detik": f"{durasi:.4f}",
        "iterasi": iterasi,
        "sideways_moves": sideways,
        "stop_reason": stop_reason,
        "state_awal_img": f"percobaan_{i}_initial.png",
        "state_akhir_img": f"percobaan_{i}_final.png",
        "plot_obj_img": f"percobaan_{i}_objective.png",
    })

    kondisi = "mencapai max_sideways" if stop_reason == "max_sideways" else
stop_reason
    print(f"Percobaan {i}: awal={initial_value}, akhir={final_value},
iterasi={iterasi}, sideways={sideways}/{max_sideways}, berhenti={kondisi},
waktu={durasi:.3f}s")
```

Mencatat **hasil tiap percobaan** ke rekap dan ke layar.

1. Satu baris dictionary ditambahkan ke rekap, berisi nomor percobaan, max_sideways, nilai awal dan akhir, durasi (4 desimal), jumlah iterasi, jumlah sideways move, alasan berhenti, dan nama file ketiga gambar.
2. Kalau berhenti karena max_sideways, teks alasan diganti menjadi

“mencapai max_sideways” agar lebih mudah dibaca.
3. Ringkasan dicetak ke terminal, termasuk rasio sideways/max_sideways.

```
csv_path = os.path.join(OUTPUT_DIR, "rekap_sideways.csv")
fieldnames = [
    "percobaan",
    "max_sideways",
    "nilai_awal",
    "nilai_akhir",
    "durasi_detik",
    "iterasi",
    "sideways_moves",
    "stop_reason",
    "state_awal_img",
    "state_akhir_img",
    "plot_obj_img",
]
with open(csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rekap)

print(f"Rekap disimpan ke {csv_path}")
```

Menyimpan **seluruh rekap eksperimen ke file CSV**.
1. Path file ditentukan: rekap_sideways.csv di folder output.
2. fieldnames menentukan urutan dan nama kolom.
3. File dibuka dengan newline="" dan encoding UTF-8 supaya tidak muncul baris kosong ekstra di Windows.
4. Header ditulis, lalu semua baris rekap ditulis sekaligus.
5. Lokasi file dicetak ke terminal.

```
if __name__ == "__main__":
    main()
```

Menjalankan **main() hanya kalau file dieksekusi langsung** (python exp_sideways.py), bukan saat diimpor sebagai modul.

2.2.3 *Stochastic Hill Climbing Algorithm*

2.2.3.1 stochastic.py

```
import random
import time
```

```
from neighbors import generate_neighbors
from objectives import calculate_objective
```

Menyiapkan **library dan modul pendukung** untuk Stochastic Hill Climbing.

1. random dipakai untuk memilih satu neighbor secara acak di antara neighbor yang lebih baik.
2. time dipakai untuk mengukur durasi eksekusi.
3. generate_neighbors dipakai untuk membangkitkan seluruh neighbor dari state saat ini.
4. calculate_objective dipakai untuk menghitung nilai tiap state.

```
def stochastic_hill_climbing(state, max_iterations=1000):
    start = time.time()

    current = state
    current_value = calculate_objective(current)

    value_history = [current_value]
    iteration = 0
    stop_reason = "max_iterations"
```

Menyiapkan **kondisi awal pencarian**.

1. Waktu mulai dicatat dengan time.time().
2. State awal dijadikan current, lalu nilai objective-nya dihitung.
3. value_history diisi nilai awal dan akan bertambah tiap iterasi, dipakai oleh plot_value.
4. iteration dimulai dari 0.
5. stop_reason diberi nilai awal "max_iterations", yang akan diganti kalau pencarian berhenti karena alasan lain.

```
    while iteration < max_iterations:
        neighbors = generate_neighbors(current)


        value_history.append(current      if not neighbors:
            stop_reason = "no_neighbor"
            break

        better = []
        for n in neighbors:
            v = calculate_objective(n)
            if v > current_value:
                better.append((n, v))
```

```
    if not better:
        stop_reason = "local_optimum"
        break

    current, current_value = random.choice(better)
_value)
    iteration += 1
```

Melakukan **pencarian iteratif** dengan berpindah ke salah satu neighbor yang lebih baik, dipilih secara acak.

1. Perulangan berjalan selama iteration belum mencapai max_iterations.
2. Seluruh neighbor dari current dibangkitkan dengan generate_neighbors.
3. Kalau tidak ada neighbor, pencarian berhenti dengan alasan "no_neighbor".
4. Nilai objective tiap neighbor dihitung. Hanya neighbor dengan nilai **lebih besar** dari current_value dimasukkan ke list better beserta nilainya.
5. Kalau better kosong, tidak ada perbaikan. Pencarian berhenti dengan alasan "local_optimum".
6. Kalau ada, satu pasangan (state, nilai) dipilih secara acak dari better sebagai current dan current_value baru.
7. Nilai baru ditambahkan ke value_history dan iteration bertambah satu.

```
  return {
    "initial_state": state,
    "final_state": current,
    "final_value": current_value,
    "value_history": value_history,
    "iterations": iteration,
    "stop_reason": stop_reason,
    "duration": time.time() - start,
  }
```

Mengembalikan **hasil pencarian dalam bentuk dictionary**.

1. initial_state dan final_state berisi state awal dan state hasil akhir.
2. final_value adalah nilai objective akhir.
3. value_history berisi riwayat nilai tiap iterasi.
4. iterations adalah jumlah iterasi yang benar-benar dijalankan.
5. stop_reason menjelaskan kenapa pencarian berhenti: max_iterations, no_neighbor, atau local_optimum.
6. duration adalah lama eksekusi dalam detik.

2.2.3.2 exp_stochastic.py

```
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from algorithms.stochastic import stochastic_hill_climbing
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data", "data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "output",
"stochastic")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Stochastic Hill Climbing.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. stochastic_hill_climbing, load_data, calculate_objective, serta plot_state dan plot_value diimpor sebagai komponen eksperimen. State tidak diimpor karena initial state diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/stochastic sebagai tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
    jumlah_percobaan = 3

    print("--- Eksperimen Stochastic Hill-Climbing ---")
```

Menyiapkan **data dan penampung hasil** sebelum eksperimen dijalankan.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. load_data membaca data.json menjadi objek kapal (ship) dan daftar kendaraan (vehicles).
3. Folder output dibuat kalau belum ada.
4. rekap disiapkan sebagai penampung hasil semua percobaan.

5. jumlah_percobaan = 3 berarti eksperimen diulang tiga kali dengan state awal acak berbeda.
6. Judul eksperimen dicetak ke layar.

```
for i in range(1, jumlah_percobaan + 1):
    initial_state = initial_states[i - 1].copy()
    initial_value = calculate_objective(initial_state)

    hasil = stochastic_hill_climbing(initial_state.copy(), max_iterations=1000)

    final_state = hasil["final_state"]
    final_value = hasil["final_value"]
    durasi = hasil["duration"]
    iterasi = hasil["iterations"]
    stop_reason = hasil["stop_reason"]
    history = hasil["value_history"]
```

Menjalankan **Stochastic Hill Climbing pada tiap percobaan**.

1. Perulangan berjalan dari percobaan 1 sampai jumlah_percobaan.
2. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Nilai objective-nya dihitung sebagai nilai_awal.
3. stochastic_hill_climbing dijalankan dengan salinan initial state dan max_iterations=1000.
4. Dari dictionary hasil diambil state akhir, nilai akhir, durasi, jumlah iterasi, alasan berhenti, dan riwayat nilai.

```
    path_init = os.path.join(OUTPUT_DIR, f"percobaan_{i}_initial.png")
    path_final = os.path.join(OUTPUT_DIR, f"percobaan_{i}_final.png")
    path_plot = os.path.join(OUTPUT_DIR, f"percobaan_{i}_objective.png")

    plot_state(initial_state, title=f"Percobaan {i} - Initial State",
save_path=path_init)
    plot_state(
        final_state,
        title=f"Percobaan {i} - Final State",
        save_path=path_final,
        iterations=hasil["iterations"],
        duration=hasil["duration"],
    )
    plot_value(history, title=f"Percobaan {i} - Objective vs Iterasi",
save_path=path_plot)
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state menggambar penempatan kendaraan pada kondisi awal,

lalu menyimpannya.
3. plot_state kembali dipakai untuk kondisi akhir.
4. plot_value menggambar grafik nilai objective terhadap iterasi dari value_history.

```
    rekap.append({
        "percobaan": i,
        "nilai_awal": initial_value,
        "nilai_akhir": final_value,
        "durasi_detik": f"{durasi:.4f}",
        "iterasi": iterasi,
        "stop_reason": stop_reason,
        "state_awal_img": f"percobaan_{i}_initial.png",
        "state_akhir_img": f"percobaan_{i}_final.png",
        "plot_obj_img": f"percobaan_{i}_objective.png",
    })

    print(f"Percobaan {i}: awal={initial_value}, akhir={final_value},
iterasi={iterasi}, berhenti={stop_reason}, waktu={durasi:.3f}s")
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.
1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state menggambar penempatan kendaraan pada kondisi awal, lalu menyimpannya.
3. plot_state kembali dipakai untuk kondisi akhir.
4. plot_value menggambar grafik nilai objective terhadap iterasi dari value_history.

```
csv_path = os.path.join(OUTPUT_DIR, "rekap_stochastic.csv")
fieldnames = [
    "percobaan",
    "nilai_awal",
    "nilai_akhir",
    "durasi_detik",
    "iterasi",
    "stop_reason",
    "state_awal_img",
    "state_akhir_img",
    "plot_obj_img",
]
with open(csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rekap)
```

print(f"Rekap disimpan ke {csv_path}")
Menyimpan **seluruh rekap eksperimen ke file CSV**. 1. Path file ditentukan: rekap_stochastic.csv di folder output. 2. fieldnames menentukan urutan dan nama kolom. 3. File dibuka dengan newline="" dan encoding UTF-8 supaya tidak muncul baris kosong ekstra di Windows. 4. Header ditulis, lalu semua baris rekap ditulis sekaligus. 5. Lokasi file dicetak ke terminal.
if __name__ == "__main__": main()
Menjalankan **main() hanya kalau file dieksekusi langsung** (python exp_stochastic.py), bukan saat diimpor sebagai modul.

2.2.4 *Random Restart Hill Climbing Algorithm*

2.2.4.1 random_restart.py

import random import time from neighbors import generate_neighbors from objectives import calculate_objective from state import State
Menyiapkan library dan modul pendukung untuk Random Restart Hill Climbing. random dipakai untuk memilih satu neighbor secara acak di antara neighbor terbaik yang nilainya sama. time dipakai untuk mengukur durasi eksekusi. generate_neighbors dipakai untuk membangkitkan seluruh neighbor dari state saat ini. calculate_objective dipakai untuk menghitung nilai tiap state. State dipakai untuk membangkitkan state acak dengan State.random pada restart berikutnya.
def random_restart_hill_climbing(state, max_restarts=10, max_iterations_per_restart=1000, vehicles=None): start_time = time.time() max_restarts = max(1, int(max_restarts)) max_iterations_per_restart = max(1, int(max_iterations_per_restart)) ship = state.ship

```
if vehicles is None:
    vehicles = [p.vehicle for p in state.placements]

best_state = state.copy()
best_value = calculate_objective(best_state)

value_history = []
best_value_history = []
restart_indices = []
iterations_per_restart = []
restart_details = []
```

1. Menyiapkan parameter dan penampung hasil.
2. Waktu mulai dicatat dengan time.time().
3. max_restarts dan max_iterations_per_restart diubah ke integer dan dipastikan minimal 1.
4. ship diambil dari state awal. Kalau vehicles tidak diberikan, daftar kendaraan diambil dari placements pada state awal.
5. best_state dan best_value menyimpan solusi terbaik dari semua restart, awalnya sama dengan state awal.
6. value_history mencatat nilai current tiap langkah di semua restart, dan best_value_history mencatat nilai terbaik sejauh ini di tiap langkah.
7. restart_indices mencatat posisi awal tiap restart di value_history, dipakai untuk garis vertikal putus-putus pada plot_value_rr.
8. iterations_per_restart dan restart_details menyimpan jumlah iterasi dan ringkasan tiap restart.

```
for r in range(max_restarts):
    if r == 0:
        current = state.copy()
    else:
        current = State.random(ship, vehicles)

    current_value = calculate_objective(current)

    restart_indices.append(len(value_history))
    value_history.append(current_value)

    if current_value > best_value:
        best_value = current_value
        best_state = current.copy()

    best_value_history.append(best_value)
```

1. Memulai tiap restart dari titik awal yang berbeda.

2. Restart pertama memakai salinan state awal dari program utama, restart berikutnya memakai State.random.
3. Nilai state awal restart dihitung, posisinya dicatat ke restart_indices, lalu nilainya ditambahkan ke value_history.
4. Kalau lebih baik dari best_value, best_state dan best_value diperbarui.
5. best_value_history ditambah satu nilai supaya panjangnya sama dengan value_history.

```
    restart_iteration = 0
    restart_stop_reason = "max_iterations"

    while restart_iteration < max_iterations_per_restart:
        neighbors = generate_neighbors(current)

        if not neighbors:
            restart_stop_reason = "no_neighbor"
            break

        values = [calculate_objective(n) for n in neighbors]
        best_neighbor_val = max(values)

        if best_neighbor_val <= current_value:
            restart_stop_reason = "local_optimum"
            break

        best_neighbors = [n for n, v in zip(neighbors, values) if v ==
best_neighbor_val]
        current = random.choice(best_neighbors)
        current_value = best_neighbor_val

        restart_iteration += 1
        value_history.append(current_value)

        if current_value > best_value:
            best_value = current_value
            best_state = current.copy()

        best_value_history.append(best_value)
```

1. Menjalankan hill climbing pada satu restart.
2. restart_iteration dimulai dari 0 dan restart_stop_reason diberi nilai awal "max_iterations".
3. Perulangan berjalan selama restart_iteration belum mencapai max_iterations_per_restart.

4. Kalau tidak ada neighbor, restart berhenti dengan alasan "no_neighbor".
5. Nilai semua neighbor dihitung, lalu nilai tertinggi diambil sebagai best_neighbor_val.
6. Kalau best_neighbor_val tidak lebih besar dari current_value, restart berhenti dengan alasan "local_optimum".
7. Kalau ada perbaikan, satu neighbor dipilih secara acak di antara neighbor dengan nilai tertinggi (steepest ascent dengan tie-break acak).
8. Nilai baru ditambahkan ke value_history, restart_iteration bertambah satu, dan best_state diperbarui kalau melampaui best_value.

```
    iterations_per_restart.append(restart_iteration)
    restart_details.append({
        "restart": r + 1,
        "final_value": current_value,
        "iterations": restart_iteration,
        "stop_reason": restart_stop_reason,
    })
```

1. Mencatat ringkasan tiap restart.
2. Jumlah iterasi ditambahkan ke iterations_per_restart.
3. restart_details diisi nomor restart, nilai akhir, jumlah iterasi, dan alasan berhenti restart tersebut.

```
stop_reason = "max_restarts"
  duration = time.time() - start_time

  return {
      "initial_state": state,
      "final_state": best_state,
      "final_value": best_value,
      "value_history": value_history,
      "best_value_history": best_value_history,
      "restart_indices": restart_indices,
      "restarts": max_restarts,
      "max_restarts": max_restarts,
      "iterations_per_restart": iterations_per_restart,
      "iterations": sum(iterations_per_restart),
      "stop_reason": stop_reason,
      "restart_details": restart_details,
      "duration": duration,
  }
```

```
random_restart = random_restart_hill_climbing
hill_climbing_random_restart = random_restart_hill_climbing
```

1. Mengembalikan hasil pencarian dalam bentuk dictionary.
2. stop_reason selalu "max_restarts" karena pencarian berhenti setelah semua restart selesai. Alasan berhenti tiap restart ada di restart_details.
3. final_state dan final_value adalah solusi terbaik dari semua restart, bukan state terakhir.
4. value_history, best_value_history, dan restart_indices dipakai oleh plot_value_rr.
5. iterations adalah total iterasi dari semua restart.
6. duration adalah lama eksekusi dalam detik.
7. random_restart dan hill_climbing_random_restart adalah alias ke fungsi yang sama.

2.2.4.2 exp_random_restart.py

```
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
"..")))

from algorithms.random_restart import random_restart_hill_climbing
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value_rr

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data", "data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "output",
"random_restart")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Random Restart Hill Climbing.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. random_restart_hill_climbing, load_data, calculate_objective, serta plot_state dan plot_value_rr diimpor sebagai komponen eksperimen. State tidak diimpor karena initial state diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/random_restart sebagai

tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    inp = input("Masukkan maximum restart (default 10): ").strip()
    try:
        max_restarts = int(inp) if inp else 10
        if max_restarts <= 0:
            print("Nilai harus > 0, menggunakan default 10.")
            max_restarts = 10
    except ValueError:
        print("Input tidak valid, menggunakan default 10.")
        max_restarts = 10

    max_iterations = 1000
```

Membaca **batas maksimum restart dari pengguna**.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. Program meminta nilai max_restarts lewat terminal.
3. Input kosong memakai default 10.
4. Kalau nilainya nol atau negatif, pesan ditampilkan dan nilai dikembalikan ke 10.
5. Kalau input tidak bisa diubah ke angka (ValueError), pesan ditampilkan dan nilai dikembalikan ke 10.
6. max_iterations = 1000 ditetapkan sebagai batas iterasi untuk tiap restart.

```
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
    jumlah_percobaan = 3

    print(f"\n--- Eksperimen Random Restart Hill-Climbing
(max_restart={max_restarts}) ---")
```

Menyiapkan **data dan penampung hasil** sebelum eksperimen dijalankan.

1. load_data membaca data.json menjadi objek kapal (ship) dan daftar kendaraan (vehicles).
2. Folder output dibuat kalau belum ada.
3. rekap disiapkan sebagai penampung hasil semua percobaan.
4. jumlah_percobaan = 3 berarti eksperimen diulang tiga kali dengan state awal acak berbeda.
5. Judul eksperimen beserta nilai max_restarts dicetak ke layar.

```
    for i in range(1, jumlah_percobaan + 1):
        initial_state = initial_states[i - 1].copy()
        initial_value = calculate_objective(initial_state)

        hasil = random_restart_hill_climbing(
            initial_state.copy(),
            max_restarts=max_restarts,
            max_iterations_per_restart=max_iterations,
            vehicles=vehicles,
        )

        final_state = hasil["final_state"]
        final_value = hasil["final_value"]
        durasi = hasil["duration"]
        iterasi = hasil["iterations"]
        restarts = hasil["restarts"]
        iter_per_restart = hasil["iterations_per_restart"]
        stop_reason = hasil["stop_reason"]
        history = hasil["value_history"]
        best_history = hasil["best_value_history"]
        restart_indices = hasil["restart_indices"]
```

Menjalankan **Random Restart Hill Climbing pada tiap percobaan** memakai initial state yang dibagikan.

1. Perulangan berjalan dari percobaan 1 sampai jumlah_percobaan.
2. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Nilai objective-nya dihitung sebagai nilai_awal.
3. random_restart_hill_climbing dijalankan dengan salinan initial state sebagai restart pertama. Restart berikutnya tetap memakai State.random dari vehicles, itu bagian dari algoritma.
4. Dari dictionary hasil diambil state akhir terbaik, nilai akhir, durasi, total iterasi, jumlah restart, iterasi per restart, alasan berhenti, riwayat nilai, riwayat nilai terbaik, dan indeks restart.

```
        plot_state(initial_state, title=f"Percobaan {i} - Initial State",
save_path=path_init)
        plot_state(
            final_state,
            title=f"Percobaan {i} - Final State",
            save_path=path_final,
            iterations=hasil["iterations"],
            duration=hasil["duration"],
            algorithm="random restart",
```

```
        max_restarts=max_restarts,
    )
    plot_value_rr(
        history,
        best_history=best_history,
        restart_indices=restart_indices,
        title=f"Percobaan {i} - Objective vs Iterasi (RRHC)",
        save_path=path_plot,
    )
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state menggambar penempatan kendaraan pada kondisi awal, lalu menyimpannya.
3. plot_state untuk final state menerima iterations, duration, algorithm="random restart", dan max_restarts, sehingga panel teks menampilkan jumlah iterasi, durasi, dan baris maximum restart.
4. plot_value_rr menggambar grafik nilai objective, garis nilai terbaik sejauh ini, dan garis vertikal putus-putus di tiap titik restart.

```
    rekap.append({
        "percobaan": i,
        "max_restart": max_restarts,
        "banyak_restart": restarts,
        "nilai_awal": initial_value,
        "nilai_akhir": final_value,
        "durasi_detik": f"{durasi:.4f}",
        "total_iterasi": iterasi,
        "iterasi_per_restart": str(iter_per_restart),
        "stop_reason": stop_reason,
        "state_awal_img": f"percobaan_{i}_initial.png",
        "state_akhir_img": f"percobaan_{i}_final.png",
        "plot_obj_img": f"percobaan_{i}_objective.png",
    })
    kondisi = "mencapai max_restart" if stop_reason == "max_restarts" else
stop_reason
    print(
        f"Percobaan {i}: awal={initial_value}, akhir={final_value}, "
        f"restart={restarts}/{max_restarts}, total_iterasi={iterasi}, "
        f"iterasi_per_restart={iter_per_restart}, berhenti={kondisi},
waktu={durasi:.3f}s"
    )
```

Mencatat **hasil tiap percobaan** ke rekap dan ke layar.

1. Satu baris dictionary ditambahkan ke rekap, berisi nomor percobaan,

max_restart, banyak restart yang terjadi, nilai awal dan akhir, durasi (4 desimal), total iterasi, iterasi per restart, alasan berhenti, dan nama file ketiga gambar.
2. iter_per_restart (berupa list) diubah jadi string dengan str() supaya bisa ditulis ke satu sel CSV.
3. Kalau berhenti karena max_restarts, teks alasan diganti menjadi “mencapai max_restart” agar lebih mudah dibaca.
4. Ringkasan dicetak ke terminal, termasuk rasio restart/max_restarts.

```
csv_path = os.path.join(OUTPUT_DIR, "rekap_random_restart.csv")
fieldnames = [
    "percobaan",
    "max_restart",
    "banyak_restart",
    "nilai_awal",
    "nilai_akhir",
    "durasi_detik",
    "total_iterasi",
    "iterasi_per_restart",
    "stop_reason",
    "state_awal_img",
    "state_akhir_img",
    "plot_obj_img",
]
with open(csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rekap)

print(f"\nRekap disimpan ke {csv_path}")
```

Menyimpan **seluruh rekap eksperimen ke file CSV**.

1. Path file ditentukan: rekap_random_restart.csv di folder output.
2. fieldnames menentukan urutan dan nama kolom.
3. File dibuka dengan newline="" dan encoding UTF-8 supaya tidak muncul baris kosong ekstra di Windows.
4. Header ditulis, lalu semua baris rekap ditulis sekaligus.
5. Lokasi file dicetak ke terminal.

```
if __name__ == "__main__":
    main()
```

Menjalankan **main() hanya kalau file dieksekusi langsung** (python exp_random_restart.py), bukan saat diimpor sebagai modul.

2.2.5 *Simulated Annealing Algorithm*

2.2.5.1 simulated_annealing.py

```
import math
import random
import time

from neighbors import random_neighbor
from objectives import calculate_objective
```

Menyiapkan **library dan modul pendukung** untuk Simulated Annealing.

1. math dipakai untuk menghitung probabilitas penerimaan e^(ΔE/T).
2. random dipakai untuk menentukan apakah state yang lebih buruk diterima.
3. time dipakai untuk mengukur durasi eksekusi.
4. random_neighbor dipakai untuk membangkitkan satu neighbor acak dari state saat ini.
5. calculate_objective dipakai untuk menghitung nilai tiap state.

```
def simulated_annealing(state, initial_temp=100.0, min_temp=1.0,
                        stuck_limit=50, max_iterations=1000):
    start = time.time()
    cooling_rate = (min_temp / initial_temp) ** (1.0 / max_iterations)

    current = state
    current_value = calculate_objective(current)
    best, best_value = current, current_value

    value_history = [current_value]
    probability_history = []

    temp = initial_temp
    iteration = 0
    no_improvement = 0
    stuck_count = 0
```

Menyiapkan **parameter, jadwal pendinginan, dan kondisi awal** pencarian.

1. Waktu mulai dicatat. Parameter initial_temp dan min_temp adalah suhu awal dan suhu minimum, stuck_limit adalah batas iterasi tanpa perbaikan untuk dianggap “stuck”, dan max_iterations adalah batas iterasi.
2. cooling_rate dihitung dengan rumus (min_temp / initial_temp)^(1/max_iterations). Ini pendinginan geometris yang membuat suhu turun dari initial_temp ke min_temp tepat setelah max_iterations iterasi.

3. State awal dijadikan current, lalu nilai objective-nya dihitung. best dan best_value menyimpan solusi terbaik yang pernah ditemukan, awalnya sama dengan state awal.
4. value_history menyimpan nilai state saat ini di tiap iterasi (dipakai plot_value_prob). probability_history menyimpan pasangan (iterasi, probabilitas) setiap kali state yang lebih buruk dipertimbangkan.
5. temp dimulai dari initial_temp.
6. iteration, no_improvement, dan stuck_count dimulai dari 0. no_improvement menghitung iterasi berturut-turut tanpa memperbaiki best, sedangkan stuck_count menghitung berapa kali kondisi “stuck” terjadi.

```
while temp > min_temp and iteration < max_iterations:
    neighbor = random_neighbor(current)

    if neighbor is not None:
        delta = calculate_objective(neighbor) - current_value
        if delta < 0:
            probability = math.exp(delta / temp)
            probability_history.append((iteration, probability))

            if random.random() < probability:
                current = neighbor
                current_value += delta
        else:
            current = neighbor
            current_value += delta
```

Memilih **satu neighbor acak lalu memutuskan diterima atau tidak**.

1. Perulangan berjalan selama suhu masih di atas min_temp dan iterasi belum mencapai max_iterations.
2. Satu neighbor acak dibangkitkan. Kalau tidak ada (None), langkah penerimaan dilewati dan iterasi tetap berjalan.
3. Selisih nilai delta dihitung dari nilai neighbor dikurangi nilai state saat ini.
4. Kalau delta < 0 (neighbor lebih buruk), probabilitas penerimaan dihitung dengan e^(delta/temp). Nilainya dicatat ke probability_history.
5. Angka acak 0 sampai 1 dibandingkan dengan probabilitas. Kalau lebih kecil, neighbor yang lebih buruk tetap diterima. Semakin tinggi suhu dan semakin kecil penurunannya, semakin besar peluang diterima.
6. Kalau delta >= 0 (lebih baik atau sama), neighbor langsung diterima.
7. Saat diterima, current diganti neighbor dan current_value ditambah delta.

```
    if current_value > best_value:
        best, best_value = current, current_value
        no_improvement = 0
    else:
        no_improvement += 1
        if no_improvement >= stuck_limit:
            stuck_count += 1
            no_improvement = 0

    value_history.append(current_value)
    temp *= cooling_rate
    iteration += 1
```

Memperbarui **solusi terbaik, penghitung stuck, dan suhu** di akhir tiap iterasi.

1. Kalau nilai saat ini melampaui best_value, solusi terbaik diperbarui dan no_improvement direset ke 0.
2. Kalau tidak, no_improvement bertambah satu. Saat mencapai stuck_limit, stuck_count bertambah satu dan no_improvement direset, jadi tiap rangkaian stuck_limit iterasi tanpa perbaikan dihitung sebagai satu kali stuck.
3. Nilai saat ini ditambahkan ke value_history.
4. Suhu diturunkan dengan dikalikan cooling_rate, lalu iteration bertambah satu.

```
  return {
    "initial_state": state,
    "final_state": best,
    "final_value": best_value,
    "value_history": value_history,
    "probability_history": probability_history,
    "duration": time.time() - start,
    "iterations": iteration,
    "stuck_count": stuck_count,
  }
```

Mengembalikan **hasil pencarian dalam bentuk dictionary**.

1. initial_state adalah state awal, sedangkan final_state dan final_value adalah **solusi terbaik yang pernah ditemukan**, bukan state terakhir.
2. value_history dan probability_history dipakai oleh plot_value_prob untuk dua grafik: nilai objective dan e^(ΔE/T).
3. duration adalah lama eksekusi dalam detik.
4. iterations adalah jumlah iterasi yang benar-benar dijalankan.
5. stuck_count adalah jumlah kejadian stuck, yang cocok dengan parameter stuck_frequency di plot_state.

2.2.5.2 exp_simulated_annealing.py

```
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
"..")))

from algorithms.simulated_annealing import simulated_annealing
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value_prob

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data", "data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "output",
"simulated_annealing")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Simulated Annealing.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. simulated_annealing, load_data, calculate_objective, serta plot_state dan plot_value_prob diimpor sebagai komponen eksperimen. State tidak diimpor karena initial state diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/simulated_annealing sebagai tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
    jumlah_percobaan = 3

    print("--- Eksperimen Simulated Annealing ---")
```

Menyiapkan **data dan penampung hasil** sebelum eksperimen dijalankan.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. load_data membaca data.json menjadi objek kapal (ship) dan daftar

kendaraan (vehicles).
3. Folder output dibuat kalau belum ada.
4. rekap disiapkan sebagai penampung hasil, dan jumlah_percobaan = 3 berarti eksperimen diulang tiga kali.
5. Judul eksperimen dicetak ke layar.

```
for i in range(1, jumlah_percobaan + 1):
    initial_state = initial_states[i - 1].copy()
    initial_value = calculate_objective(initial_state)

    hasil = simulated_annealing(
        initial_state.copy(),
        initial_temp=100.0,
        min_temp=1.0,
        stuck_limit=50,
        max_iterations=1000
    )

    final_state = hasil["final_state"]
    final_value = hasil["final_value"]
    durasi = hasil["duration"]
    iterasi = hasil["iterations"]
    stuck_count = hasil["stuck_count"]
    history_value = hasil["value_history"]
    history_prob = hasil["probability_history"]
```

Menjalankan **Simulated Annealing pada tiap percobaan** memakai initial state yang dibagikan.

1. Perulangan berjalan dari percobaan 1 sampai jumlah_percobaan.
2. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Nilai objective-nya dihitung sebagai nilai_awal.
3. simulated_annealing dijalankan dengan salinan initial state dan parameter tetap: suhu awal 100, suhu minimum 1, batas stuck 50, dan maksimum 1000 iterasi.
4. Dari dictionary hasil diambil state akhir terbaik, nilai akhir, durasi, jumlah iterasi, jumlah stuck, riwayat nilai, dan riwayat probabilitas penerimaan.

```
    plot_state(initial_state, title=f"Percobaan {i} - Initial State",
save_path=path_init)
    plot_state(
        final_state,
        title=f"Percobaan {i} - Final State",
        save_path=path_final,
```

```
        iterations=hasil["iterations"],
        duration=hasil["duration"],
        algorithm="simulated annealing",
        stuck_frequency=stuck_count,
    )
    plot_value_prob(
        history_value,
        history_prob,
        title=f"Percobaan {i} - SA: Objective & e^(ΔE/T) vs Iterasi",
        save_path=path_plot_obj,
    )
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state untuk initial state dipanggil tanpa parameter tambahan, jadi baris Iteration dan Duration tetap - karena belum ada proses pencarian.
3. plot_state untuk final state menerima iterations, duration, algorithm="simulated annealing", dan stuck_frequency, sehingga panel teks menampilkan jumlah iterasi, durasi, dan baris stuck frequency.
4. plot_value_prob menggambar dua grafik bertumpuk: nilai objective terhadap iterasi dan probabilitas penerimaan e^(ΔE/T).

```
    rekap.append({
        "percobaan": i,
        "nilai_awal": initial_value,
        "nilai_akhir": final_value,
        "durasi_detik": f"{durasi:.4f}",
        "iterasi": iterasi,
        "frekuensi_stuck": stuck_count,
        "state_awal_img": f"percobaan_{i}_initial.png",
        "state_akhir_img": f"percobaan_{i}_final.png",
        "plot_obj_img": f"percobaan_{i}_objective.png",
    })

    print(f"Percobaan {i}: awal={initial_value}, akhir={final_value},
iterasi={iterasi}, stuck={stuck_count}, waktu={durasi:.3f}s")
```

Mencatat **hasil tiap percobaan** ke rekap dan ke layar.

1. Satu baris dictionary ditambahkan ke rekap, berisi nomor percobaan, nilai awal dan akhir, durasi (4 desimal), jumlah iterasi, frekuensi stuck, dan nama file ketiga gambar.
2. Ringkasan percobaan dicetak ke terminal supaya kemajuan eksperimen terlihat.

```
    csv_path = os.path.join(OUTPUT_DIR, "rekap_simulated_annealing.csv")
    fieldnames = [
        "percobaan",
        "nilai_awal",
        "nilai_akhir",
        "durasi_detik",
        "iterasi",
        "frekuensi_stuck",
        "state_awal_img",
        "state_akhir_img",
        "plot_obj_img",
    ]
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rekap)

    print(f"Rekap disimpan ke {csv_path}")
```

Menyimpan **seluruh rekap eksperimen ke file CSV**.

1. Path file ditentukan: rekap_simulated_annealing.csv di folder output.
2. fieldnames menentukan urutan dan nama kolom.
3. File dibuka dengan mode "w", newline="", dan encoding UTF-8, sehingga isi lama ditimpa dan tidak muncul baris kosong ekstra di Windows.
4. Header ditulis, lalu semua baris rekap ditulis sekaligus.
5. Lokasi file dicetak ke terminal.

```
if __name__ == "__main__":
    main()
```

Menjalankan **main() hanya kalau file dieksekusi langsung**, bukan saat diimpor sebagai modul.

2.2.6 *Genetic Algorithm*

2.2.6.1 genetic.py

```
import random
import time

from constraints import is_valid
from neighbors import random_neighbor
from objectives import calculate_objective
from state import State, Placement
```

Menyiapkan **library dan modul pendukung** untuk Genetic Algorithm.

1. random dipakai untuk pengacakan (urutan, seleksi, probabilitas mutasi).
2. time dipakai untuk mengukur durasi eksekusi.
3. is_valid dipakai untuk memastikan hasil crossover tetap memenuhi constraint.
4. random_neighbor dipakai sebagai operator mutasi.
5. calculate_objective dipakai sebagai nilai fitness.
6. State dan Placement dipakai untuk membangun individu (child) baru.

```
def crossover(p1, p2):
    n = len(p1.placements)
    child = State(p1.ship, [Placement(pl.vehicle, pl.Orientation) for
pl in p1.placements])
    order = list(range(n))
    random.shuffle(order)
    for i in order:
        parents = [p1, p2]
        random.shuffle(parents)
        for parent in parents:
            src = parent.placements[i]
            if not src.on_ship:
                continue
            dst = child.placements[i]
            dst.Orientation, dst.x, dst.y, dst.on_ship =
src.Orientation, src.x, src.y, True
            if is_valid(child):
                break
            dst.on_ship, dst.x, dst.y = False, None, None
    return child
```

Membuat **child dari gabungan dua parent** dengan tetap menjaga state valid.

1. Child dibuat dengan kapal yang sama dan daftar Placement kosong (belum di kapal) untuk tiap kendaraan.
2. Urutan indeks kendaraan diacak supaya tidak ada kendaraan yang selalu diproses lebih dulu.
3. Untuk tiap kendaraan, urutan parent (p1, p2) diacak, sehingga pewarisan posisi bersifat acak.
4. Kalau kendaraan itu tidak dimuat di parent yang sedang dicek, parent tersebut dilewati.
5. Kalau dimuat, orientasi dan posisi (x, y) parent disalin ke child dengan on_ship = True.
6. Kalau child masih valid (is_valid), penempatan dipertahankan dan loop parent berhenti (break).
7. Kalau tidak valid (misal overlap dengan kendaraan lain yang sudah diwariskan), penempatan dibatalkan (on_ship = False, x, y = None), lalu dicoba parent satunya.
8. Setelah semua kendaraan diproses, child dikembalikan. Hasilnya selalu valid, tapi bisa memuat lebih sedikit kendaraan.

```
def mutate(state, mutation_rate):
    if random.random() < mutation_rate:
        neighbor = random_neighbor(state)
        if neighbor is not None:
            return neighbor
    return state
```

Melakukan **mutasi pada individu** dengan peluang tertentu.

1. Angka acak 0 sampai 1 dibandingkan dengan mutation_rate. Mutasi hanya terjadi kalau angkanya lebih kecil.
2. Kalau terpilih, dibangkitkan satu neighbor acak lewat random_neighbor(state).
3. Kalau neighbor berhasil didapat (tidak None), neighbor itu dikembalikan sebagai hasil mutasi.
4. Kalau tidak terpilih atau neighbor tidak ada, state dikembalikan apa adanya.

```
def tournament(population, fitness, k=3):
    idx = random.sample(range(len(population)), min(k,
len(population)))
    return population[max(idx, key=lambda i: fitness[i])]
```

Memilih **satu parent dengan random**.

1. Diambil k indeks individu secara acak tanpa pengulangan (k dibatasi agar tidak melebihi ukuran populasi).
2. Dari kandidat tersebut, dipilih indeks dengan fitness tertinggi.
3. Individu pada indeks itu dikembalikan sebagai parent.

```
def genetic_algorithm(state, population_size=50,
max_iterations=1000,mutation_rate=0.3, elite_count=1):
    start = time.time()
    vehicles = [p.vehicle for p in state.placements]
    population = [state.copy()]
    while len(population) < population_size:
        population.append(State.random(state.ship, vehicles))

    fitness = [calculate_objective(s) for s in population]

    best_idx = max(range(len(population)), key=lambda i: fitness[i])
    best, best_value = population[best_idx].copy(), fitness[best_idx]

    max_history = [max(fitness)]
    avg_history = [sum(fitness) / len(fitness)]

    for _ in range(max_iterations):
    ranked = sorted(range(len(population)), key=lambda i: fitness[i],
reverse=True)
        new_pop = [population[i].copy() for i in ranked[:elite_count]]

        while len(new_pop) < population_size:
```

```
            p1 = tournament(population, fitness)
            p2 = tournament(population, fitness)
            child = crossover(p1, p2)
            child = mutate(child, mutation_rate)
            new_pop.append(child)

        population = new_pop
        fitness = [calculate_objective(s) for s in population]

        gen_best = max(range(len(population)), key=lambda i:
fitness[i])
        if fitness[gen_best] > best_value:
            best, best_value = population[gen_best].copy(),
fitness[gen_best]

        max_history.append(max(fitness))
        avg_history.append(sum(fitness) / len(fitness))

    return {
        "initial_state": state,
        "final_state": best,
        "final_value": best_value,
        "max_history": max_history,
        "avg_history": avg_history,
        "population_size": population_size,
        "iterations": max_iterations,
        "duration": time.time() - start,
    }
```

Menjalankan **Genetic Algorithm** dari state awal dan mengembalikan solusi terbaik beserta riwayat prosesnya.

1. Waktu mulai dicatat, lalu daftar kendaraan diambil dari state.placements.
2. Populasi awal berisi salinan state ditambah state acak (State.random) sampai berjumlah population_size.
3. Fitness tiap individu dihitung dengan calculate_objective.
4. Individu terbaik awal disimpan sebagai best dan best_value, dan riwayat awal dicatat di max_history (maksimum) dan avg_history (rata-rata).
5. Pada tiap generasi (sebanyak max_iterations), individu diurutkan berdasarkan fitness, lalu elite_count terbaik disalin langsung ke populasi baru (elitisme).
6. Sisa populasi baru diisi berulang: pilih dua parent dengan tournament, hasilkan child lewat crossover, lalu terapkan mutate.
7. Populasi diganti dengan populasi baru, lalu fitness dihitung ulang.
8. Kalau individu terbaik generasi ini lebih baik dari best_value, best diperbarui.
9. Nilai maksimum dan rata-rata generasi ditambahkan ke max_history dan avg_history.

10. Setelah semua generasi selesai, dikembalikan dictionary berisi initial_state, final_state, final_value, max_history, avg_history, population_size, iterations, dan duration. Riwayat dipakai oleh plot_value_ga.

2.2.6.2 exp_genetic.py

```
import csv
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
"..")))

from algorithms.genetic import genetic_algorithm
from data.loader import load_data
from objectives import calculate_objective
from visualisasi import plot_state, plot_value_ga

DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data",
"data.json")
OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "..",
"output", "genetic")
```

Menyiapkan **library, path modul, dan lokasi data/output** untuk eksperimen Genetic Algorithm.

1. csv dipakai untuk menulis rekap hasil, os untuk path dan folder, sys untuk mengatur path impor.
2. sys.path.append(...) menambahkan folder induk ke path impor, supaya file di folder experiment bisa mengimpor modul lain di proyek.
3. genetic_algorithm, load_data, calculate_objective, serta plot_state dan plot_value_ga diimpor sebagai komponen eksperimen. State ikut diimpor tetapi belum terpakai, karena initial state kini diterima dari program utama.
4. DATA_PATH menunjuk ke data/data.json (data kapal dan kendaraan).
5. OUTPUT_DIR menunjuk ke folder output/genetic sebagai tempat semua gambar dan CSV disimpan.

```
def main(initial_states=None):
    try:
        inp = input("Masukkan jumlah populasi kontrol untuk exp1
(default 50): ").strip()
        pop_kontrol = int(inp) if inp else 50
        inp = input("Masukkan 3 variasi iterasi, pisah koma (default
50,100,200): ").strip()
        var_iter = [int(x) for x in inp.split(",")] if inp else [50,
```

```
100, 200]
        inp = input("Masukkan banyak iterasi kontrol untuk exp2
(default 100): ").strip()
        iter_kontrol = int(inp) if inp else 100
        inp = input("Masukkan 3 variasi populasi, pisah koma (default
20,50,100): ").strip()
        var_pop = [int(x) for x in inp.split(",")] if inp else [20, 50,
100]
    except ValueError:
        print("Input tidak valid, menggunakan semua nilai default.")
        pop_kontrol, var_iter = 50, [50, 100, 200]
        iter_kontrol, var_pop = 100, [20, 50, 100]
```

Membaca **parameter eksperimen dari pengguna**, dengan nilai default kalau input kosong atau salah.

1. main menerima initial_states, yaitu daftar tiga initial state dari program utama, supaya semua algoritma memakai state awal yang sama pada percobaan yang sama.
2. Meminta jumlah populasi kontrol untuk eksperimen 1 (default 50).
3. Meminta tiga variasi jumlah iterasi untuk eksperimen 1, dipisah koma (default 50, 100, 200).
4. Meminta jumlah iterasi kontrol untuk eksperimen 2 (default 100).
5. Meminta tiga variasi jumlah populasi untuk eksperimen 2 (default 20, 50, 100).
6. Input kosong memakai default. Kalau input tidak bisa diubah ke angka (ValueError), pesan ditampilkan dan **semua** parameter dikembalikan ke default.

```
    ship, vehicles = load_data(DATA_PATH)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    rekap = []
    jumlah_percobaan = 3

    konfigurasi = [("exp1", pop_kontrol, it) for it in var_iter]
    konfigurasi += [("exp2", pop, iter_kontrol) for pop in var_pop]
```

Menyiapkan **data, folder output, dan daftar konfigurasi eksperimen**.

1. load_data membaca data.json menjadi objek kapal (ship) dan daftar kendaraan (vehicles).
2. Folder output dibuat kalau belum ada.
3. rekap disiapkan sebagai penampung hasil, dan jumlah_percobaan = 3 berarti tiap konfigurasi diulang tiga kali.
4. Eksperimen 1 memvariasikan **iterasi** dengan populasi tetap

(pop_kontrol).

5. Eksperimen 2 memvariasikan **populasi** dengan iterasi tetap (iter_kontrol).
6. Hasilnya 6 konfigurasi dalam bentuk (nama, populasi, iterasi).

```
    for nama, populasi, iterasi in konfigurasi:
        folder = f"{nama}_pop{populasi}_iter{iterasi}"
        os.makedirs(os.path.join(OUTPUT_DIR, folder), exist_ok=True)

        print(f"\n--- Eksperimen Genetic Algorithm ({nama}:
populasi={populasi}, iterasi={iterasi}) ---")

        for i in range(1, jumlah_percobaan + 1):
            initial_state = initial_states[i - 1].copy()
            initial_value = calculate_objective(initial_state)

            hasil = genetic_algorithm(initial_state.copy(),
population_size=populasi, max_iterations=iterasi)

            final_state = hasil["final_state"]
            final_value = hasil["final_value"]
            durasi = hasil["duration"]
```

Menjalankan **Genetic Algorithm untuk tiap konfigurasi dan percobaan** memakai initial state yang dibagikan.

1. Untuk tiap konfigurasi, dibuat subfolder bernama sesuai parameter, misalnya exp1_pop50_iter100, lalu judul eksperimen dicetak.
2. Tiap konfigurasi diulang sebanyak jumlah_percobaan.
3. Initial state percobaan ke-i diambil dari initial_states[i - 1] dan disalin dengan .copy(), bukan dibangkitkan acak. Semua konfigurasi dan semua algoritma lain jadi memakai state awal yang sama pada percobaan yang sama.
4. Nilai objective initial state dihitung sebagai nilai_awal.
5. genetic_algorithm dijalankan dengan salinan initial state supaya state asli tidak berubah.
6. State akhir terbaik, nilai akhir, dan durasi diambil dari dictionary hasil.

```
            plot_state(initial_state, title=f"Percobaan {i} - Initial
State", save_path=path_init)
            plot_state(
                final_state,
                title=f"Percobaan {i} - Final State",
                save_path=path_final,
                iterations=hasil["iterations"],
                duration=hasil["duration"],
                algorithm="genetic",
```

```
                population_size=populasi,
            )
            plot_value_ga(
                hasil["max_history"],
                hasil["avg_history"],
                title=f"Percobaan {i} - Objective vs Iterasi",
                save_path=path_plot,
            )
```

Menyimpan **tiga gambar visualisasi** untuk tiap percobaan.

1. Ditentukan tiga path file: state awal, state akhir, dan grafik objective.
2. plot_state untuk initial state dipanggil tanpa parameter tambahan, jadi baris Iteration dan Duration di gambar tetap - karena belum ada proses pencarian.
3. plot_state untuk final state menerima iterations, duration, algorithm="genetic", dan population_size, sehingga panel teks menampilkan jumlah iterasi (generasi), durasi, dan baris jumlah populasi.
4. plot_value_ga menggambar grafik nilai maksimum dan rata-rata populasi terhadap iterasi dari max_history dan avg_history.

```
            rekap.append({
                "eksperimen": nama,
                "percobaan": i,
                "populasi": populasi,
                "iterasi": iterasi,
                "nilai_awal": initial_value,
                "nilai_akhir": final_value,
                "durasi_detik": f"{durasi:.4f}",
                "state_awal_img":
f"{folder}/percobaan_{i}_initial.png",
                "state_akhir_img": f"{folder}/percobaan_{i}_final.png",
                "plot_obj_img":
f"{folder}/percobaan_{i}_objective.png",
            })

            print(f"Percobaan {i}: awal={initial_value},
akhir={final_value}, populasi={populasi}, iterasi={iterasi},
waktu={durasi:.3f}s")
```

Mencatat **hasil tiap percobaan** ke rekap dan ke layar.

1. Satu baris dictionary ditambahkan ke rekap, berisi nama eksperimen, nomor percobaan, parameter, nilai awal dan akhir, durasi (4 desimal), serta path relatif ketiga gambar.
2. Ringkasan percobaan dicetak ke terminal supaya kemajuan

eksperimen terlihat.

```
csv_path = os.path.join(OUTPUT_DIR, "rekap_genetic.csv")
 fieldnames = [
     "eksperimen",
     "percobaan",
     "populasi",
     "iterasi",
     "nilai_awal",
     "nilai_akhir",
     "durasi_detik",
     "state_awal_img",
     "state_akhir_img",
     "plot_obj_img",
 ]
 with open(csv_path, "w", newline="", encoding="utf-8") as f:
     writer = csv.DictWriter(f, fieldnames=fieldnames)
     writer.writeheader()
     writer.writerows(rekap)

 print(f"Rekap disimpan ke {csv_path}")
```

Menyimpan **seluruh rekap eksperimen ke file CSV**.

1. Path file ditentukan: rekap_genetic.csv di folder output.
2. fieldnames menentukan urutan dan nama kolom.
3. File dibuka dengan mode "w", newline="", dan encoding UTF-8, sehingga isi lama ditimpa dan tidak muncul baris kosong ekstra di Windows.
4. Header ditulis, lalu semua baris rekap ditulis sekaligus.
5. Lokasi file dicetak ke terminal.

```
if __name__ == "__main__":
    main()
```

Menjalankan **main() hanya kalau file dieksekusi langsung** (python exp_genetic.py), bukan saat diimpor sebagai m
odul.

2.3 Menjalankan Program

main.py

```
import os
import sys
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
if os.path.basename(BASE_DIR) == "src":
    SRC_DIR = BASE_DIR
    BASE_DIR = os.path.dirname(BASE_DIR)
else:
    SRC_DIR = os.path.join(BASE_DIR, "src")

EXP_DIR = os.path.join(SRC_DIR, "experiment")

if SRC_DIR not in sys.path:
    sys.path.insert(0, SRC_DIR)
if EXP_DIR not in sys.path:
    sys.path.insert(0, EXP_DIR)
```

Menentukan **lokasi folder proyek dan mengatur path impor**, supaya file ini bisa dijalankan dari root proyek maupun dari dalam src.

1. BASE_DIR diisi folder tempat file ini berada.
2. Kalau folder itu bernama src, maka SRC_DIR adalah folder tersebut dan BASE_DIR naik satu tingkat ke root proyek.
3. Kalau bukan, SRC_DIR diasumsikan berada di dalam BASE_DIR (BASE_DIR/src).
4. EXP_DIR menunjuk ke src/experiment.
5. SRC_DIR dan EXP_DIR ditambahkan ke awal sys.path (kalau belum ada), sehingga modul seperti state, data.loader, dan exp_* bisa diimpor langsung.

```
from data.loader import load_data
from objectives import calculate_objective
from state import State

import exp_steepest_ascent
import exp_sideways
import exp_stochastic
import exp_random_restart
import exp_simulated_annealing
import exp_genetic

DATA_PATH = os.path.join(SRC_DIR, "data", "data.json")
```

Mengimpor **komponen inti dan keenam modul eksperimen**.

1. load_data membaca data kapal dan kendaraan.
2. calculate_objective dipakai untuk menampilkan nilai awal tiap initial state.
3. State dipakai untuk membangkitkan initial state acak.
4. Enam modul exp_* diimpor supaya fungsi main masing-masing bisa dipanggil dari menu.

5. DATA_PATH menunjuk ke src/data/data.json.

```
DAFTAR_ALGORITMA = {
    1: ("Steepest Ascent Hill-Climbing", exp_steepest_ascent.main),
    2: ("Sideways Move Hill-Climbing", exp_sideways.main),
    3: ("Stochastic Hill-Climbing", exp_stochastic.main),
    4: ("Random Restart Hill-Climbing", exp_random_restart.main),
    5: ("Simulated Annealing", exp_simulated_annealing.main),
    6: ("Genetic Algorithm", exp_genetic.main),
}
```

Menyimpan **daftar algoritma beserta fungsi eksekusinya** dalam satu dictionary.

1. Kunci berupa nomor menu (1 sampai 6).
2. Nilai berupa pasangan (nama algoritma, fungsi main).
3. Dengan begitu menu dan pemanggilan algoritma cukup memakai satu struktur data, tanpa if-elif panjang.

```
def main():
    print("\n" + "=" * 14)
    print("Program Utama")
    print("=" * 14)

    ship, vehicles = load_data(DATA_PATH)
    initial_states = [State.random(ship, vehicles) for _ in range(3)]

    print("\n3 Random Initial State :")
    for i, st in enumerate(initial_states, 1):
        print(f"  * Percobaan {i} : Nilai Objective Awal =
{calculate_objective(st)}")

    sudah_dijalankan = set()
```

Menyiapkan **initial state terpusat** yang dipakai bersama oleh semua algoritma.

1. Judul Program Utama dicetak ke terminal.
2. load_data membaca data.json menjadi ship dan vehicles.
3. Tiga initial state acak dibangkitkan **sekali di awal** dengan State.random, lalu disimpan di list initial_states. Percobaan ke-i selalu memakai state yang sama di semua algoritma.
4. Nilai objective awal tiap percobaan ditampilkan di bawah judul 3 Random Initial State :.
5. sudah_dijalankan (set) disiapkan untuk mencatat algoritma yang sudah dijalankan.

```
 while True:
        belum_dicoba = [k for k in DAFTAR_ALGORITMA if k not in
sudah_dijalankan]
        print("\n" + "=" * 18)
        print("PILIHAN ALGORITMA")
        print("=" * 18)
        for num, (nama, _) in DAFTAR_ALGORITMA.items():
            status = "[SUDAH DIJALANKAN]" if num in sudah_dijalankan else
"[BELUM DICOBA]"
            print(f"  {num}. {nama:<35} {status}")
        print("  7. Jalankan SEMUA Algoritma yang Belum Dicoba")
        print("  0. Selesai (Keluar)")
        pilihan = input("\nMasukkan nomor pilihan Anda: ").strip()
```

Menampilkan **menu pilihan algoritma** berulang sampai pengguna keluar.

1. Perulangan while True membuat menu muncul lagi setelah tiap pilihan selesai.
2. Header PILIHAN ALGORITMA dicetak di antara dua garis pemisah.
3. Tiap algoritma dicetak dengan nomor, nama, dan status [SUDAH DIJALANKAN] atau [BELUM DICOBA].
4. Opsi 7 menjalankan semua algoritma yang belum dicoba, dan opsi 0 keluar.
5. Pilihan pengguna dibaca dan dibersihkan spasinya dengan strip().

```
    if pilihan in ["0", "selesai", "exit", "keluar"]:
            print("\nbye :)))) stop nubes")
            break
        elif pilihan == "7":
            targets = [k for k in range(1, 7) if k not in sudah_dijalankan]
            if not targets:
                print("Semua algoritma sudah pernah dijalankan.")
                continue
            for k in targets:
                nama, func = DAFTAR_ALGORITMA[k]
                print(f"\n>>> Memulai: {nama} <<<")
                func(initial_states=initial_states)
                sudah_dijalankan.add(k)
        elif pilihan.isdigit() and int(pilihan) in DAFTAR_ALGORITMA:
            pilih_num = int(pilihan)
            nama, func = DAFTAR_ALGORITMA[pilih_num]
            func(initial_states=initial_states)
            sudah_dijalankan.add(pilih_num)
        else:
            print("Pilihan tidak valid, silakan masukkan nomor menu yang
benar."
```

Memproses **pilihan pengguna** dan menjalankan algoritma yang dipilih.

1. Kalau pilihan 0, selesai, exit, atau keluar, pesan penutup dicetak dan perulangan berhenti (break).
2. Kalau pilihan 7, dikumpulkan semua algoritma yang belum dijalankan (targets). Kalau kosong, pesan ditampilkan dan menu diulang (continue).
3. Untuk tiap target, nama algoritma dicetak, lalu fungsi main dipanggil dengan initial_states=initial_states, dan nomornya dicatat ke sudah_dijalankan.
4. Kalau pilihan berupa angka 1 sampai 6, hanya algoritma itu yang dijalankan dengan cara yang sama.
5. Selain itu, ditampilkan pesan bahwa pilihan tidak valid dan menu diulang.
6. Input tambahan (misalnya populasi GA atau max_sideways) tidak diminta di file ini, tapi di dalam main tiap exp_* saat algoritma itu dijalankan.

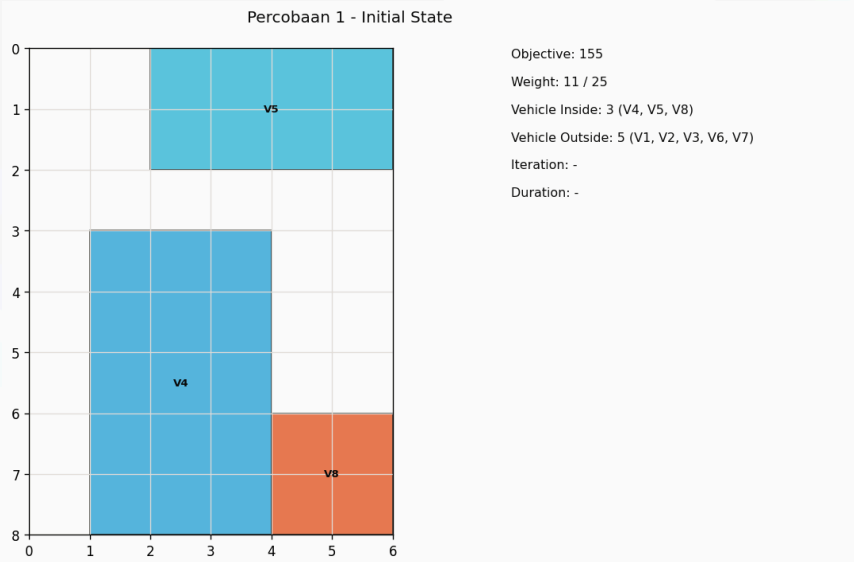
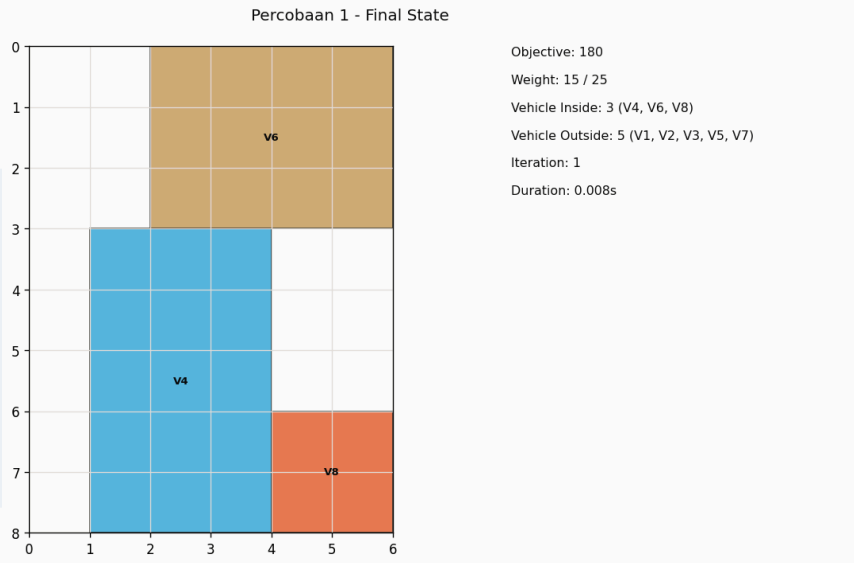
```
if __name__ == "__main__":
    main()
```

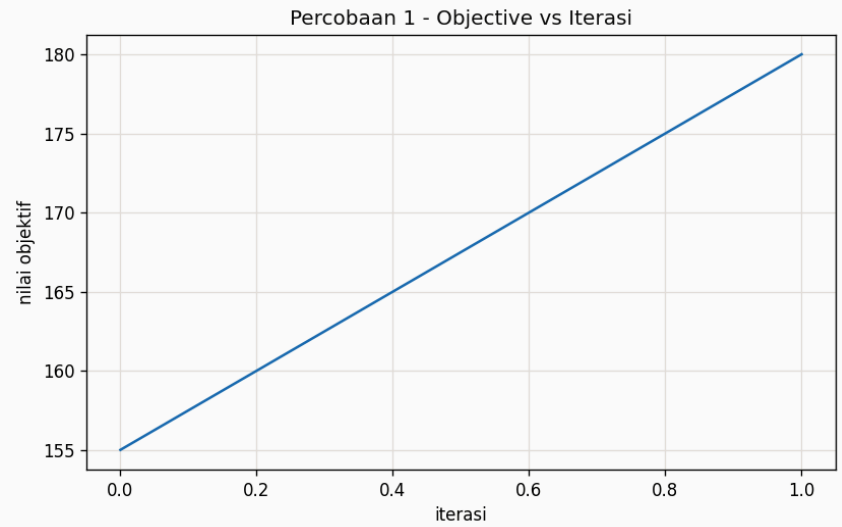
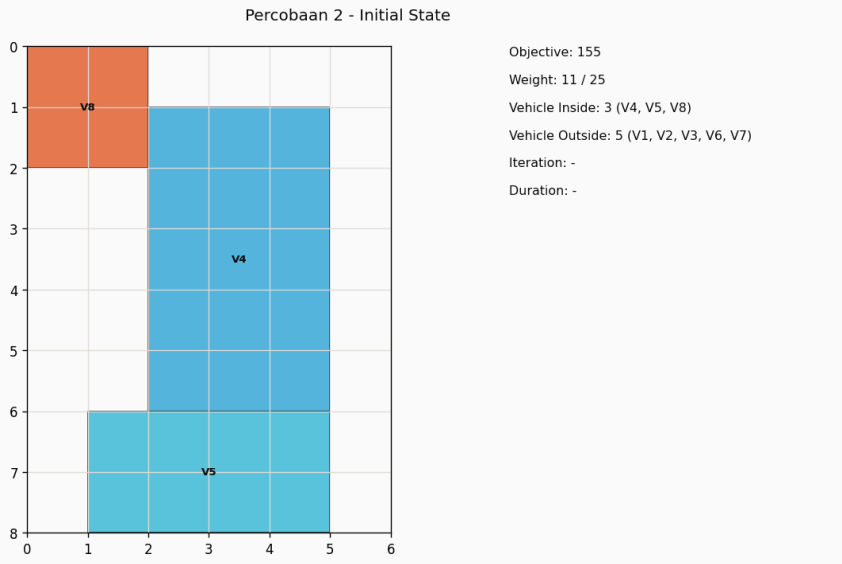
Menjalankan **main() hanya kalau file dieksekusi langsung**, bukan saat diimpor sebagai modul.

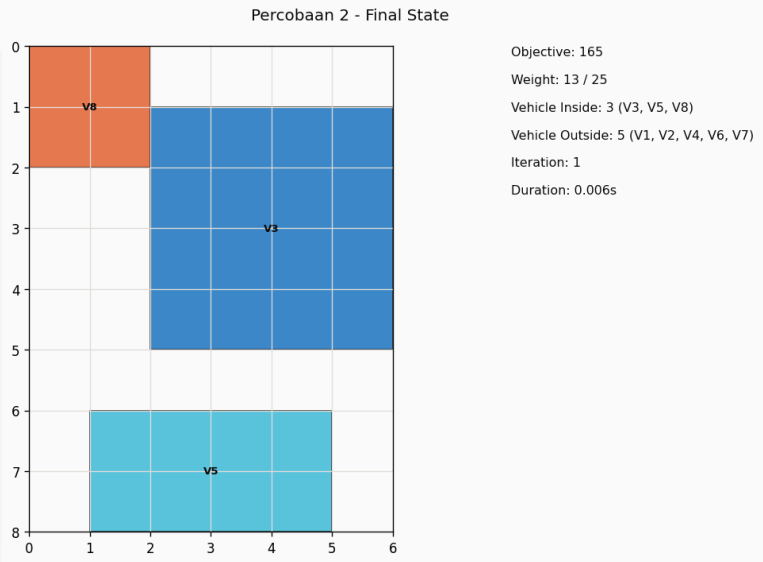
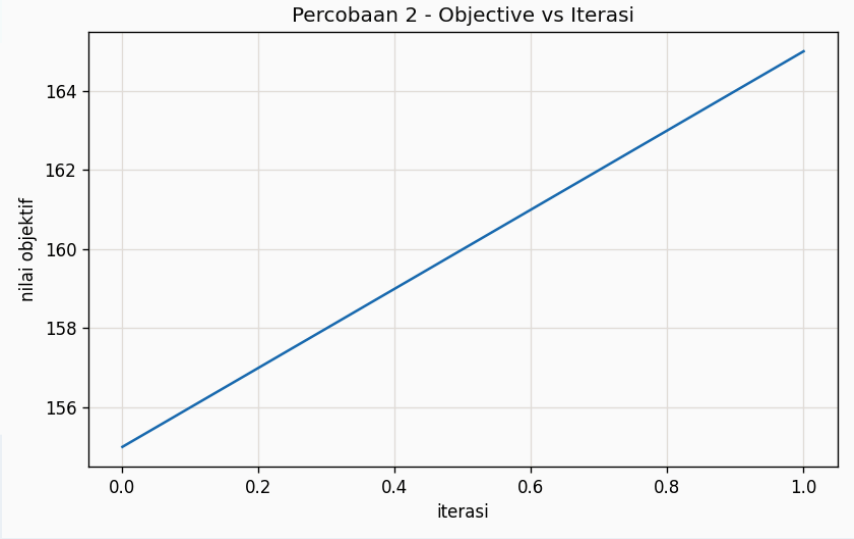
BAB III
Hasil dan Analisis Eksperimen

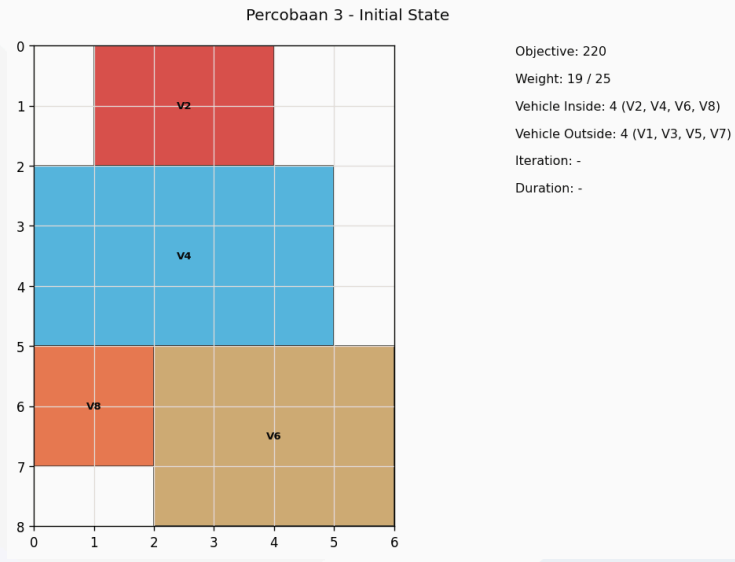
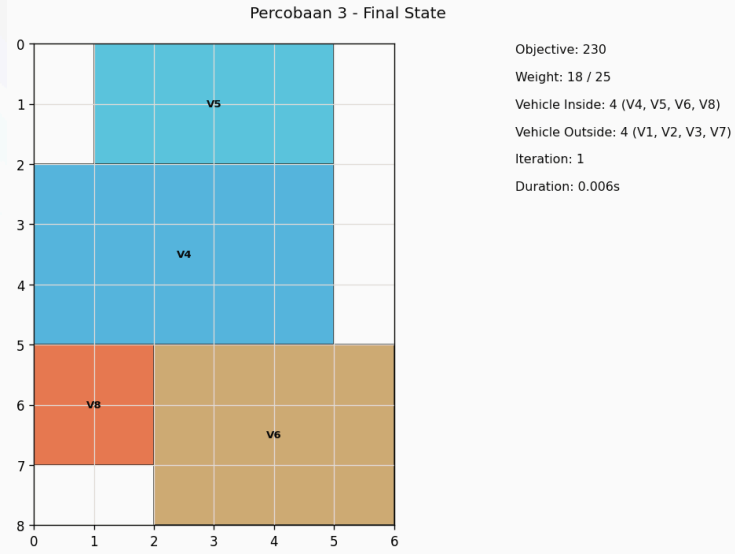
3.1 Hasil Eksperimen

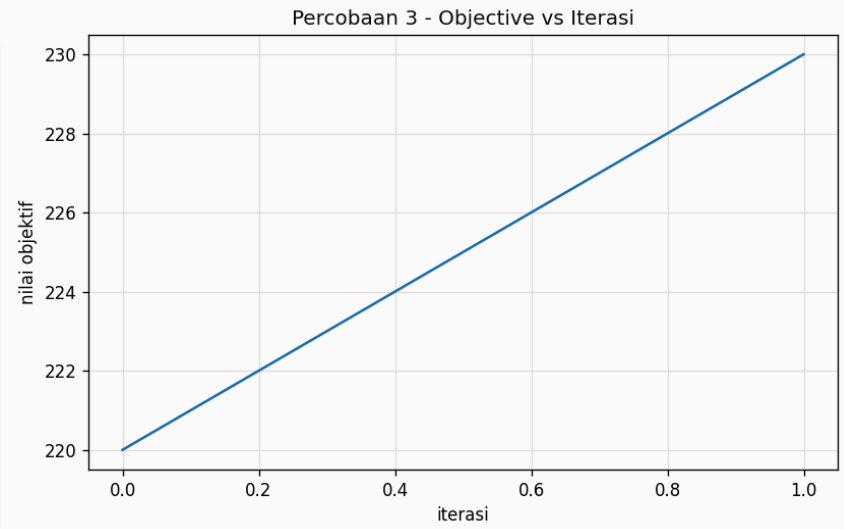
3.1.1 Steepest Ascent Eksperimen Hill Climbing

Percobaan 1	
State Awal	
State Akhir	
Nilai Objective Function Terakhir	180

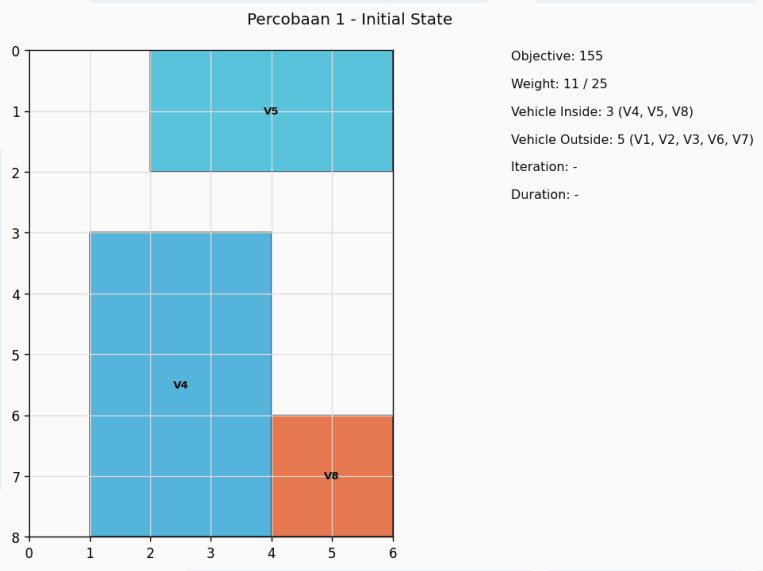
Plot Objective Function vs Iterasi	
Durasi	0.008s
Total Iterasi yang dilakukan sampai Program berhenti	1
Percobaan 2	
State Awal	

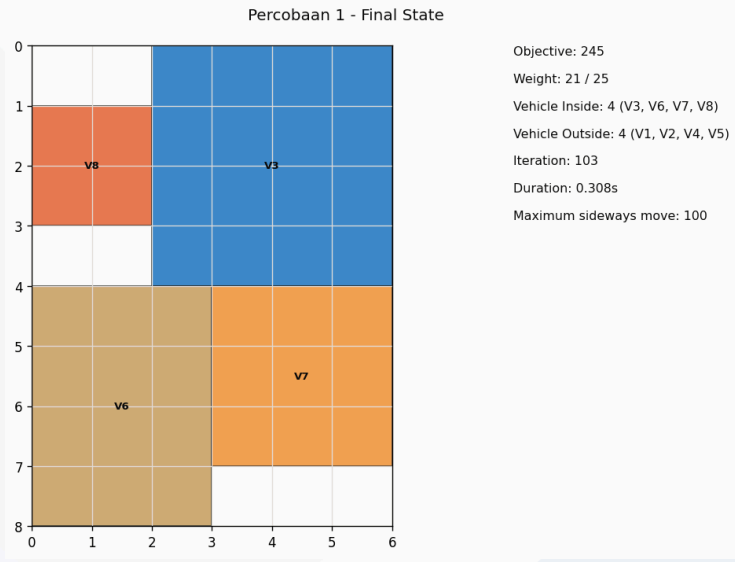
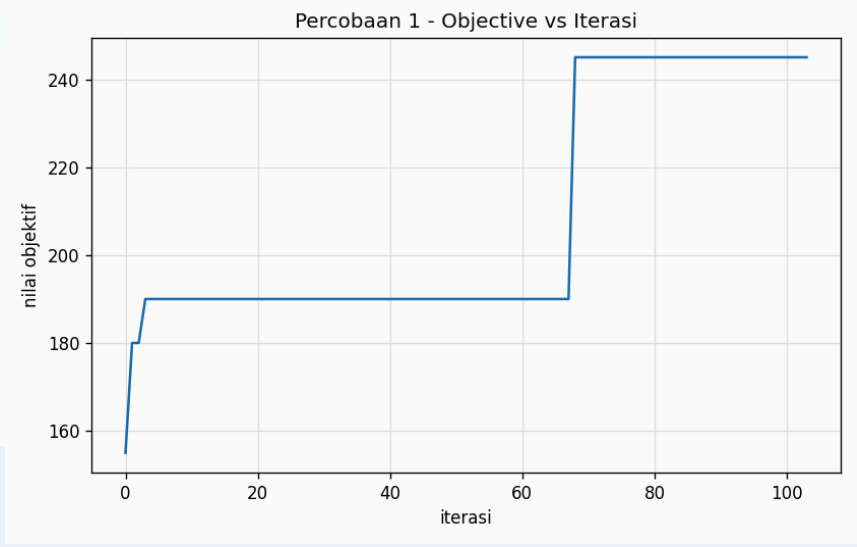
State Akhir	
Nilai Objective Function Terakhir	165
Plot Objective Function vs Iterasi	
Durasi	0.006s
Total Iterasi yang dilakukan sampai Program berhenti	1
Percobaan 3	

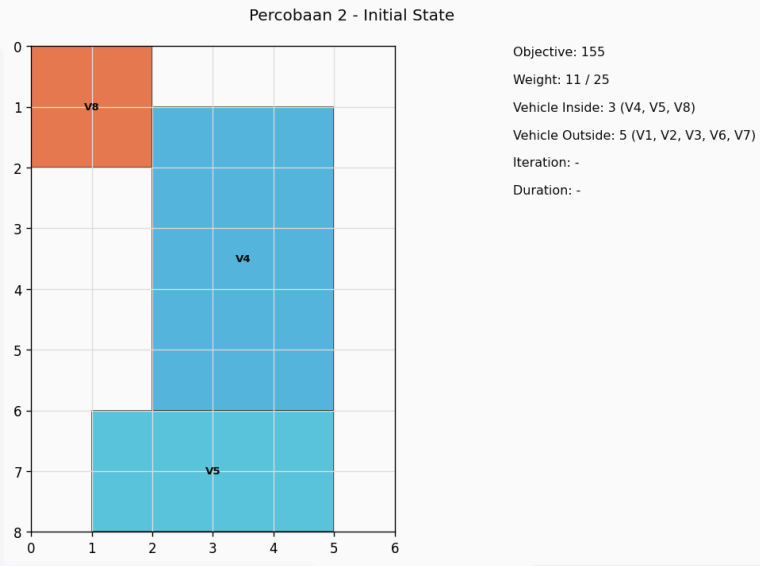
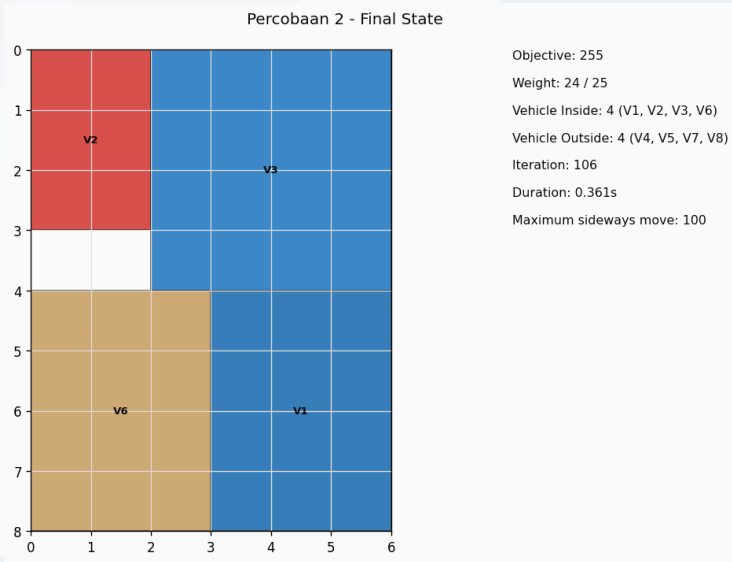
State Awal	
State Akhir	
Nilai Objective Function Terakhir	230

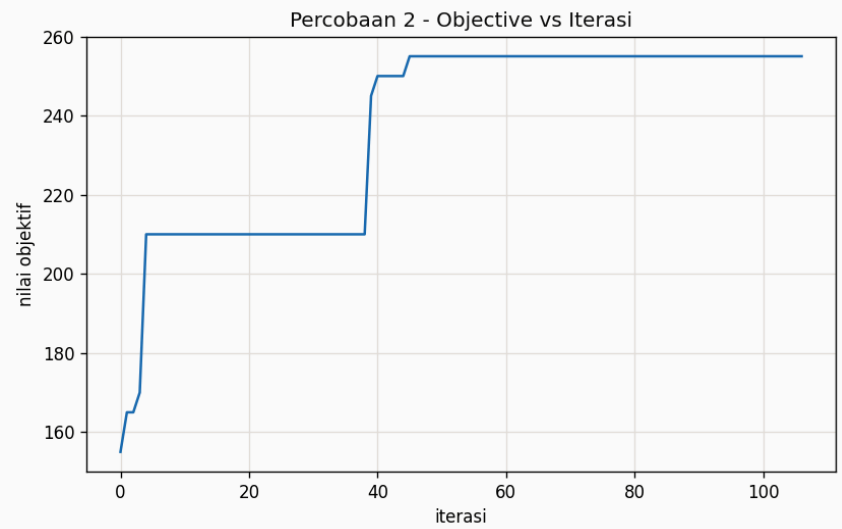
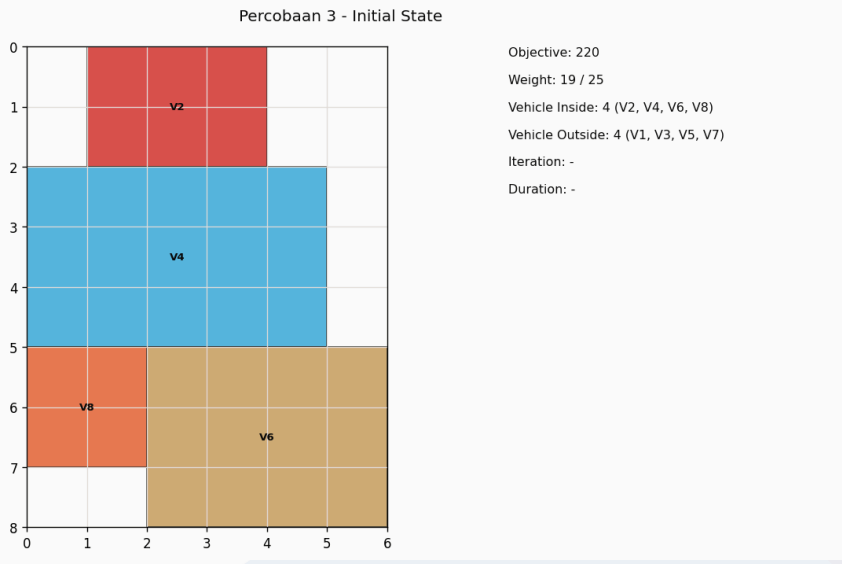
Plot Objective Function vs Iterasi	
Durasi	0.006s
Total Iterasi yang dilakukan sampai Program berhenti	1

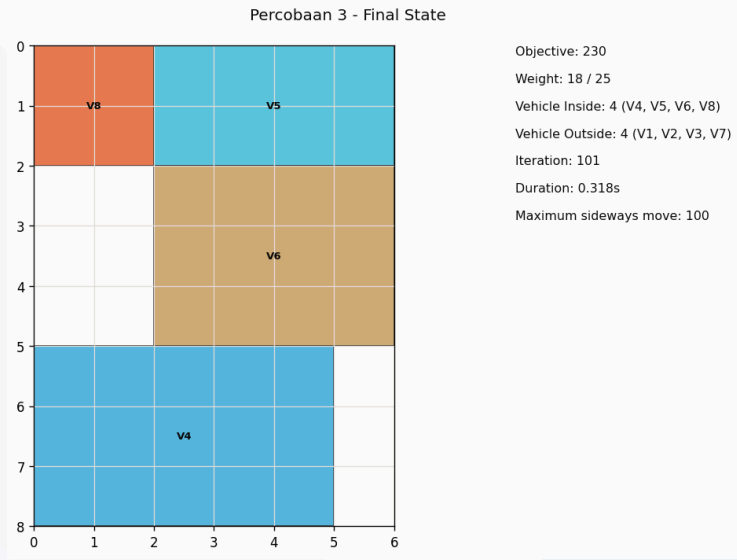
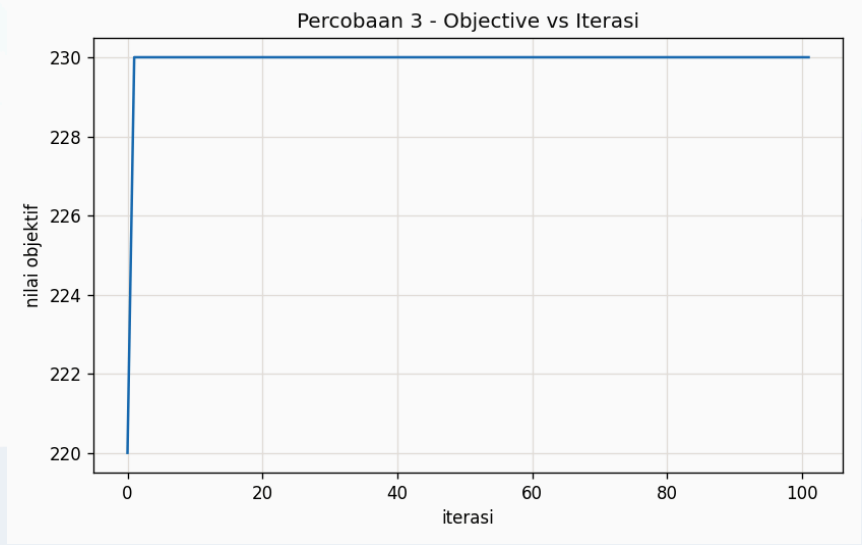
3.1.2 Sideways Move Hill Climbing Algorithm

Percobaan 1	
State Awal	

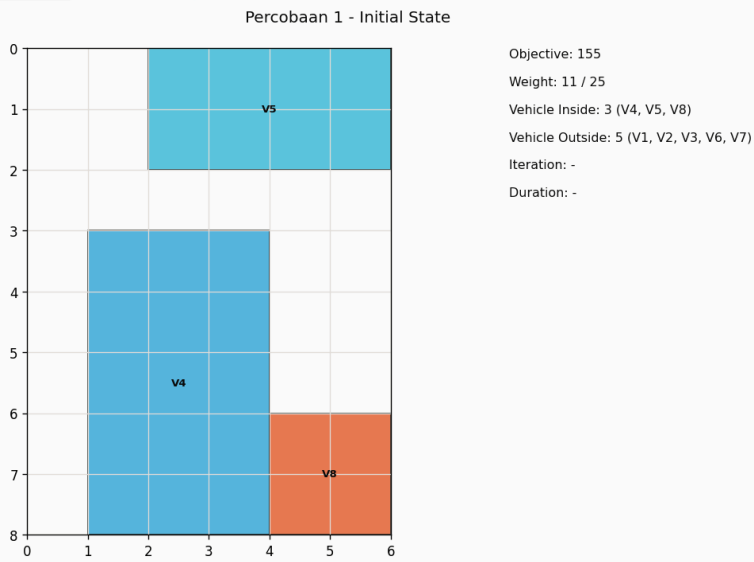
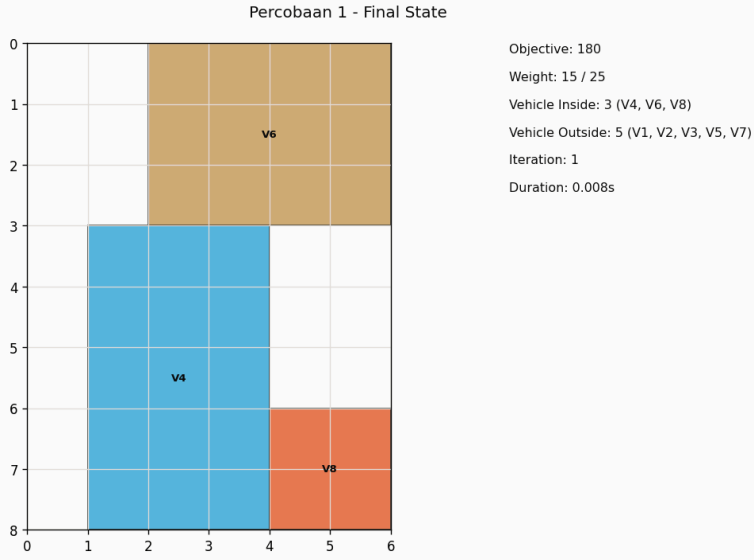
State Akhir	
Nilai Objective Function Terakhir	245
Plot Objective Function vs Iterasi	
Durasi	0.308s
Total Iterasi yang dilakukan sampai Program berhenti	103
Maximum sideways move	100
Percobaan 2	

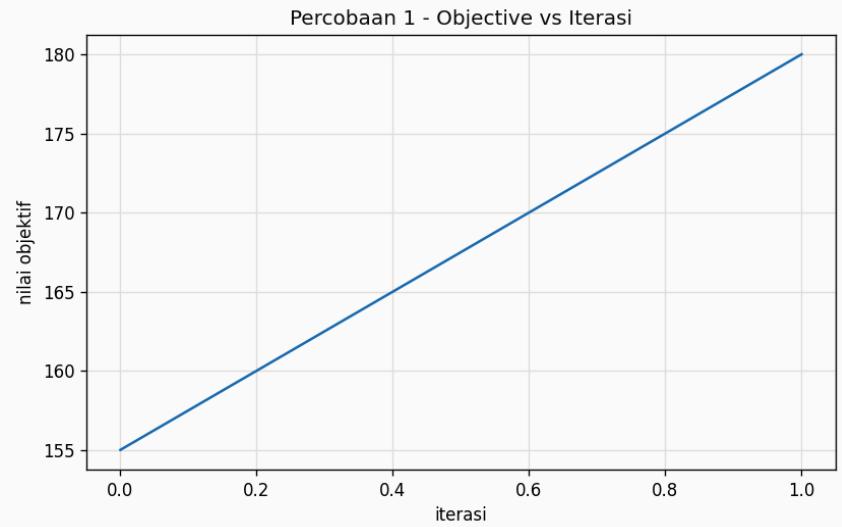
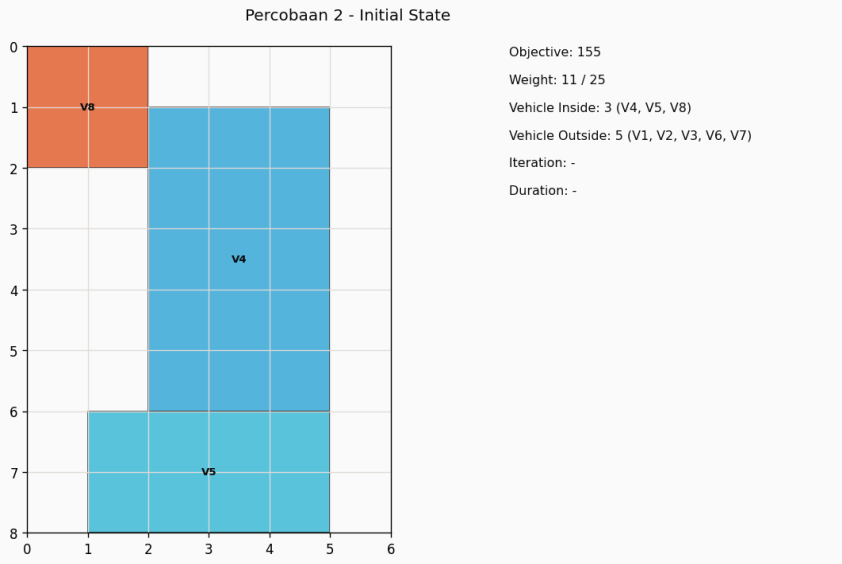
State Awal	
State Akhir	
Nilai Objective Function Terakhir	255

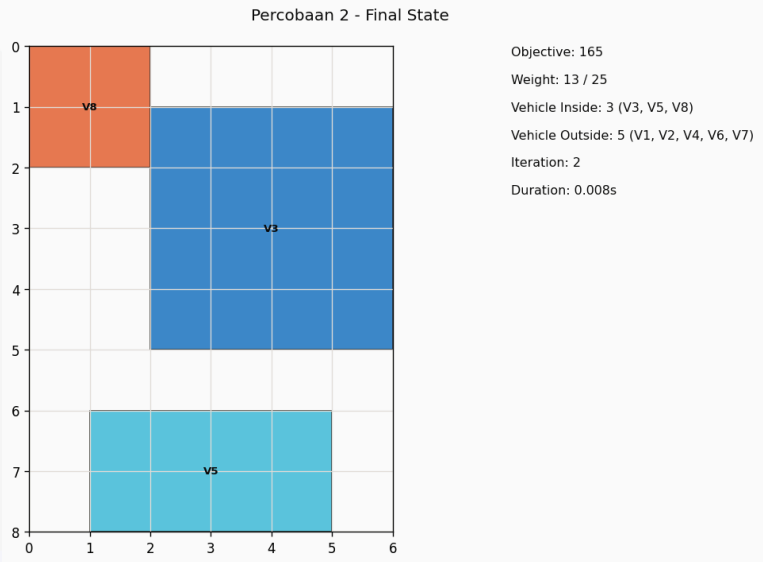
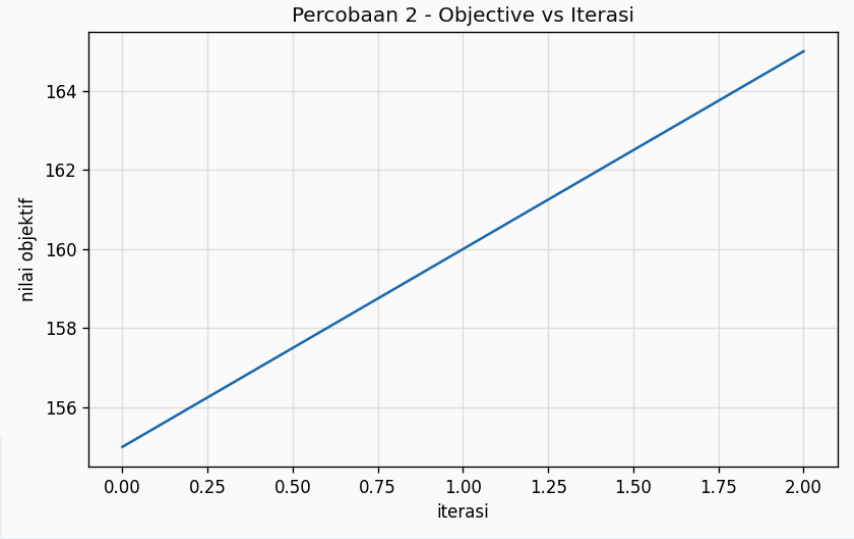
Plot Objective Function vs Iterasi	
Durasi	0.361s
Total Iterasi yang dilakukan sampai Program berhenti	106
Maximum sideways move	100
Percobaan 3	
State Awal	

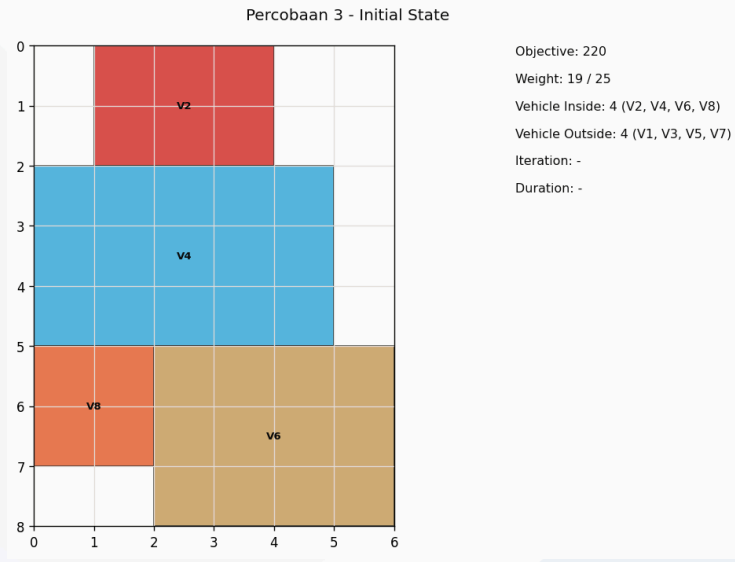
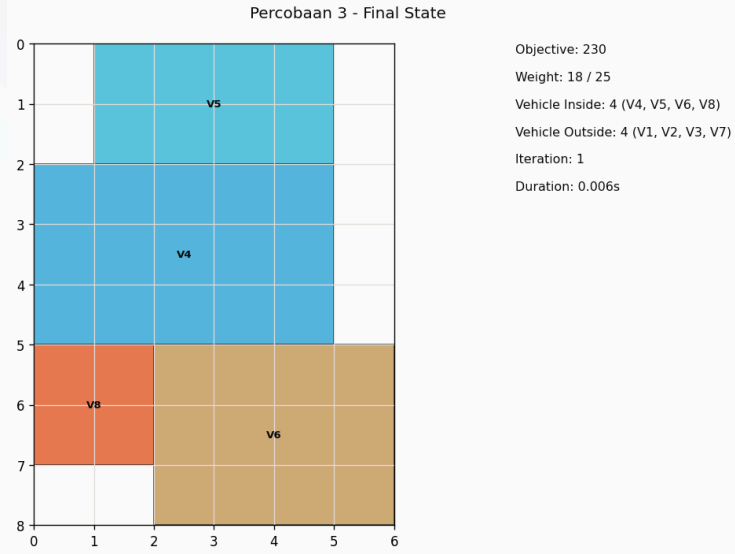
State Akhir	
Nilai Objective Function Terakhir	230
Plot Objective Function vs Iterasi	
Durasi	0.318s
Total Iterasi yang dilakukan sampai Program berhenti	101
Maximum sideways move	100

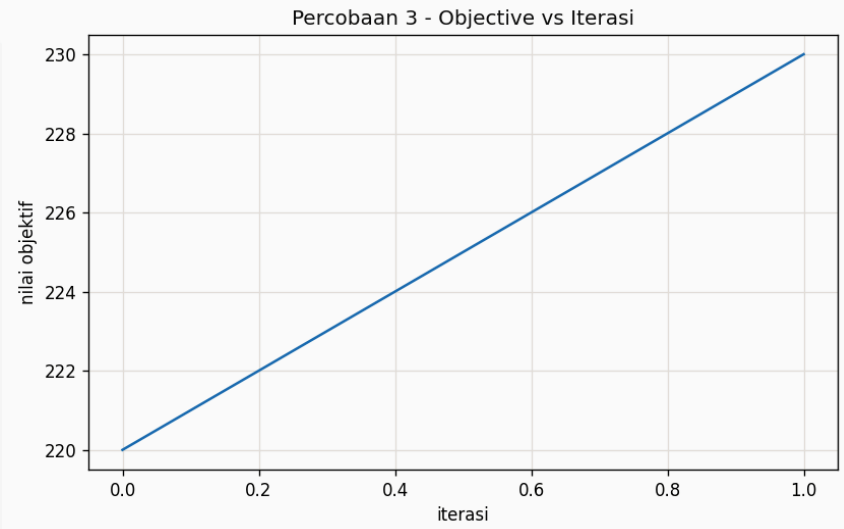
3.1.3 Stochastic Hill Climbing

Percobaan 1	
State Awal	
State Akhir	
Nilai Objective Function Terakhir	180

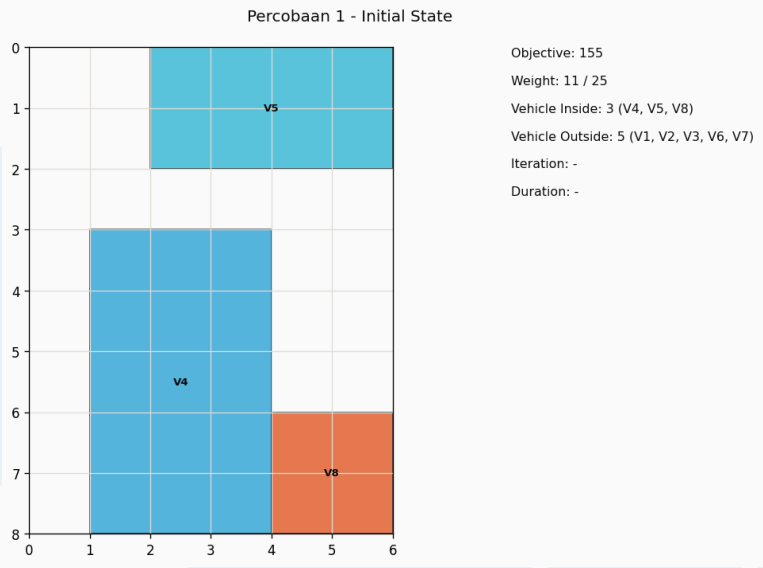
Plot Objective Function vs Iterasi	
Durasi	0.008s
Total Iterasi yang dilakukan sampai Program berhenti	1
Percobaan 2	
State Awal	

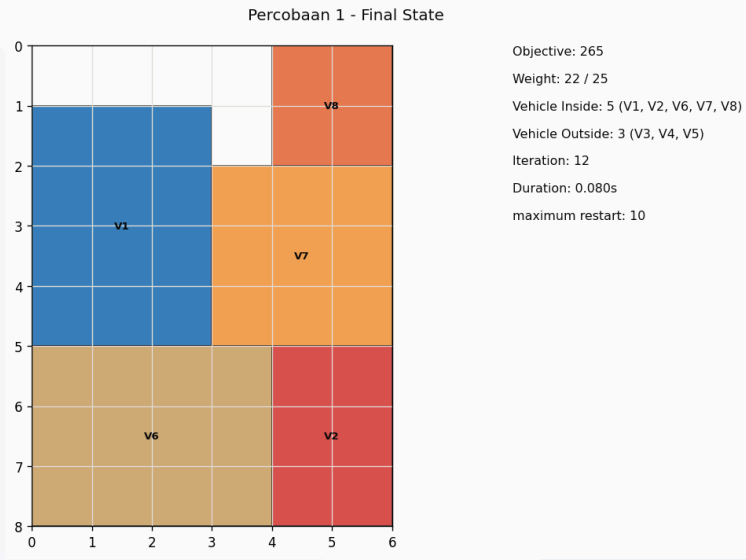
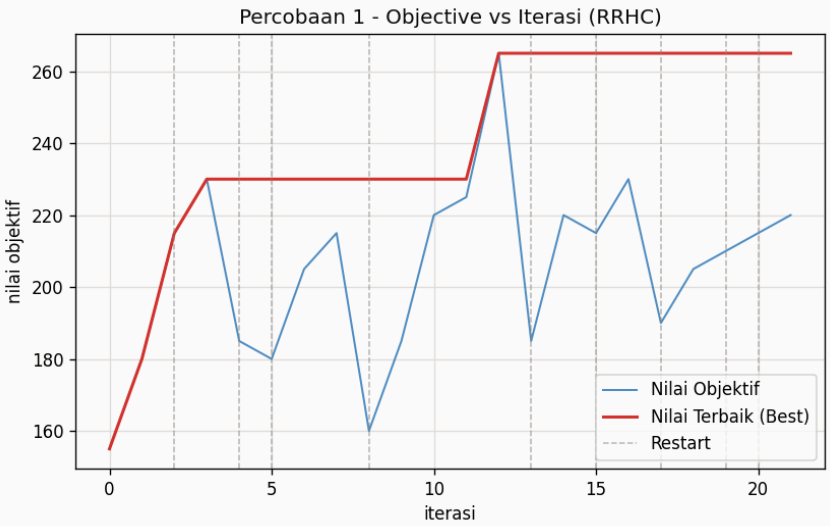
State Akhir	
Nilai Objective Function Terakhir	165
Plot Objective Function vs Iterasi	
Durasi	0.008s
Total Iterasi yang dilakukan sampai Program berhenti	2
Percobaan 3	

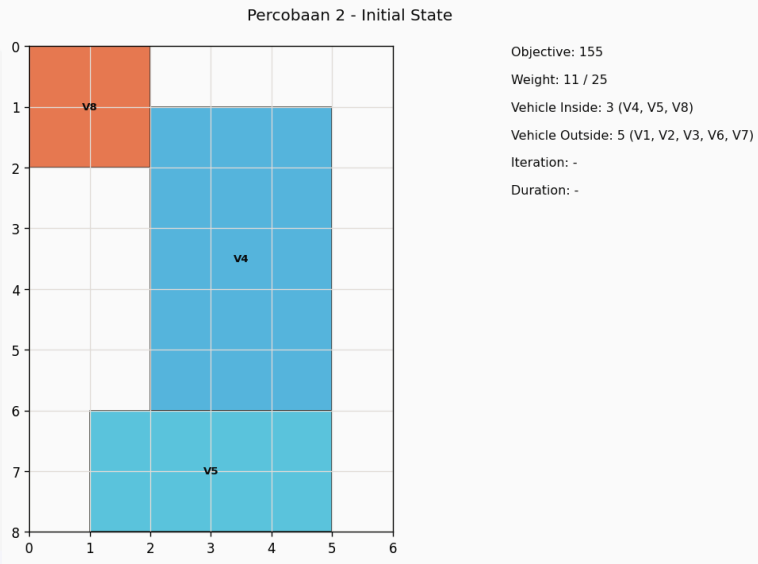
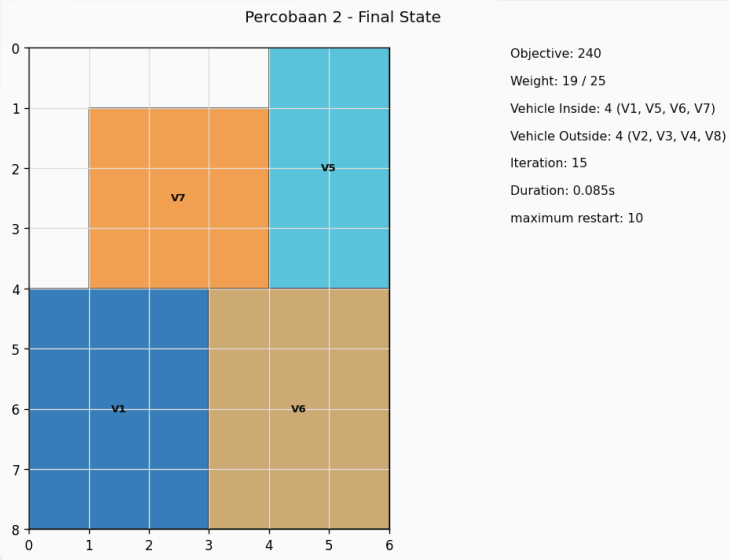
State Awal	
State Akhir	
Nilai Objective Function Terakhir	230

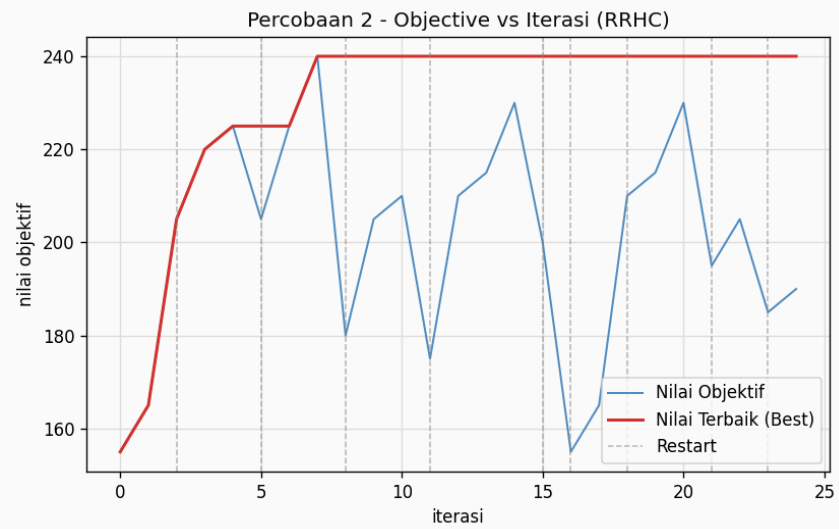
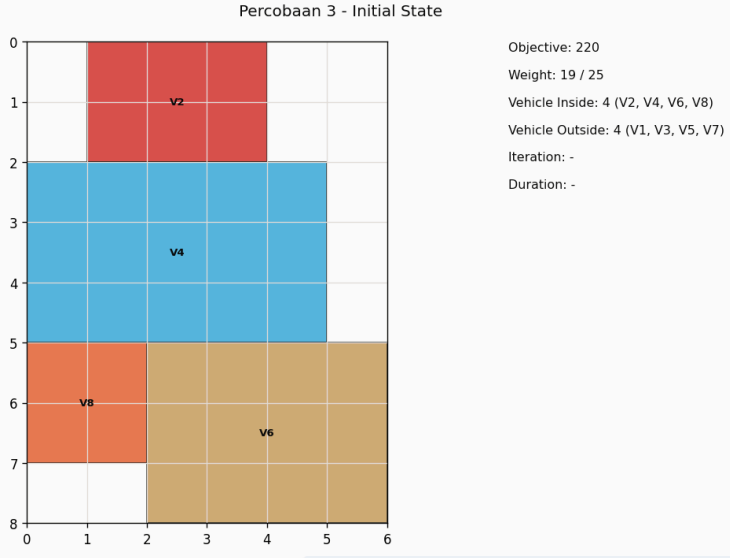
Plot Objective Function vs Iterasi	
Durasi	0.006s
Total Iterasi yang dilakukan sampai Program berhenti	1

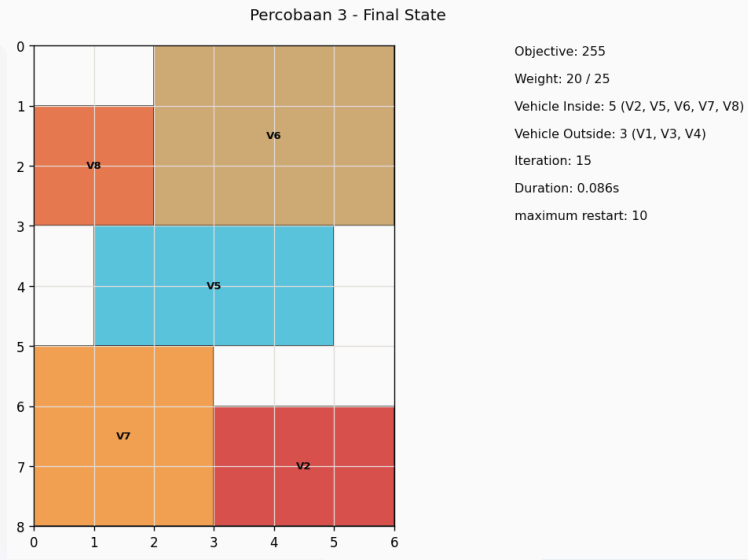
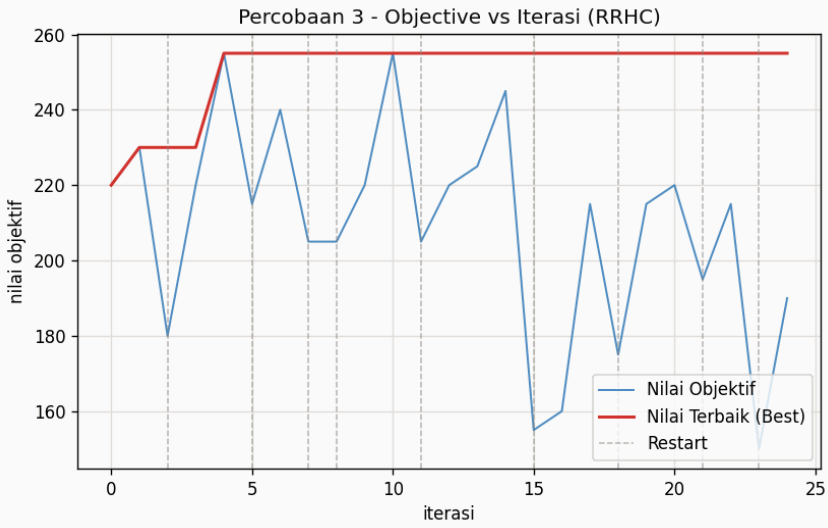
3.1.4 Random Restart Hill Climbing

Percobaan 1	
State Awal	

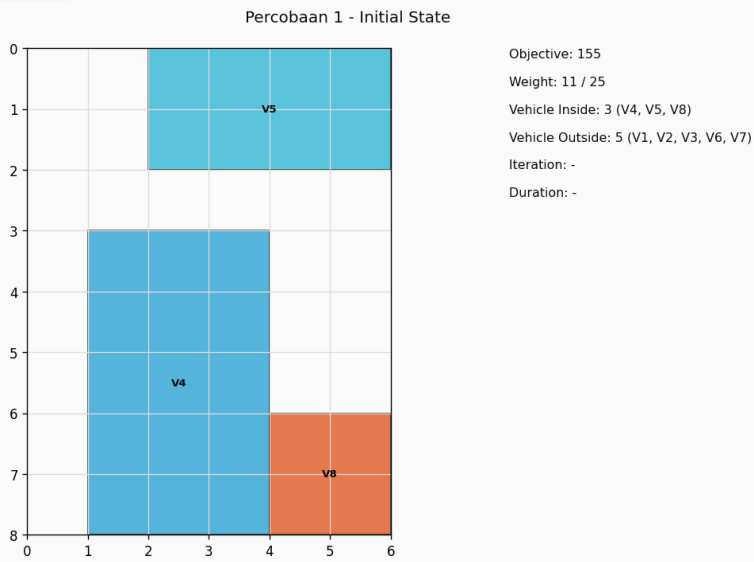
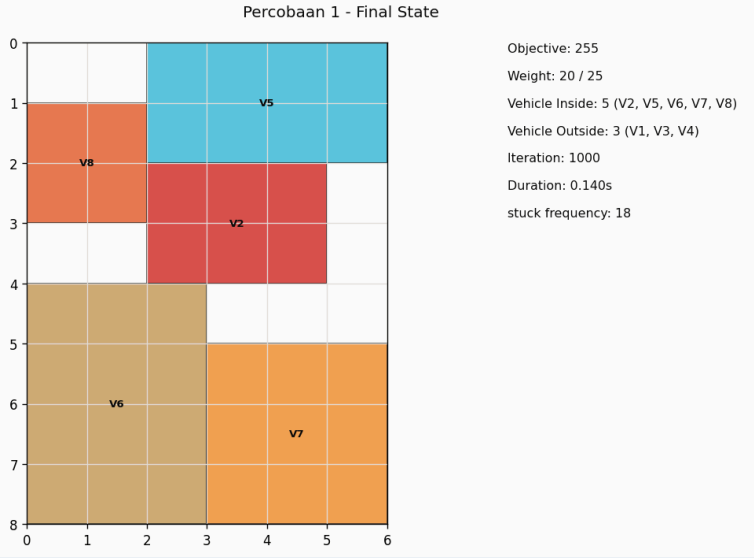
State Akhir	
Nilai Objective Function Terakhir	265
Plot Objective Function vs Iterasi	
Durasi	0.080s
Total Iterasi yang dilakukan sampai Program berhenti	12 Iterasi per restart = [1, 1, 0, 2, 4, 1, 1, 1, 0, 1]
Maximum Restart	10
Percobaan 2	

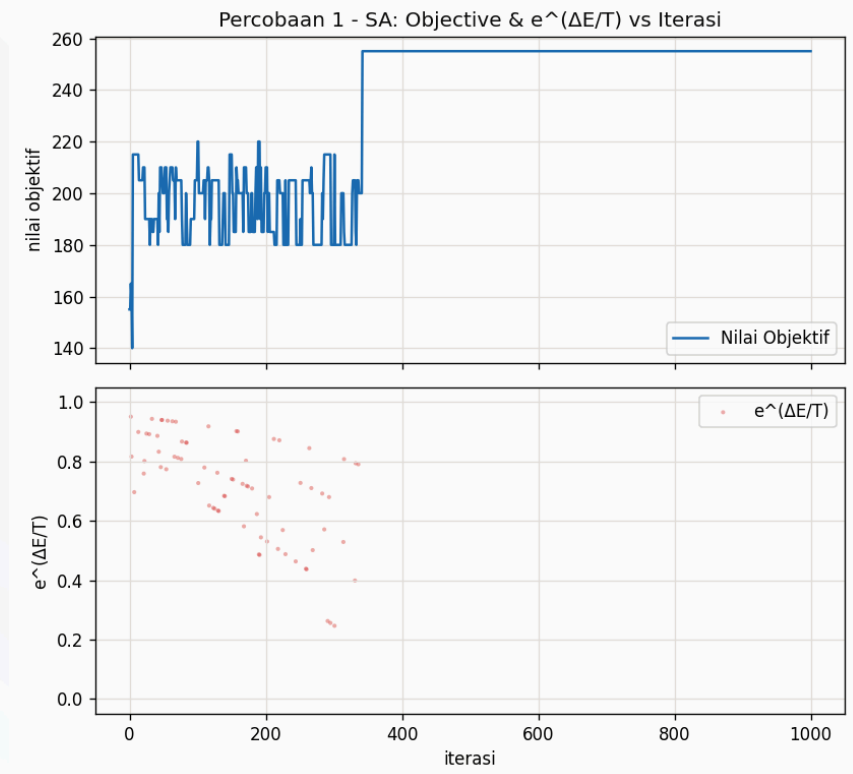
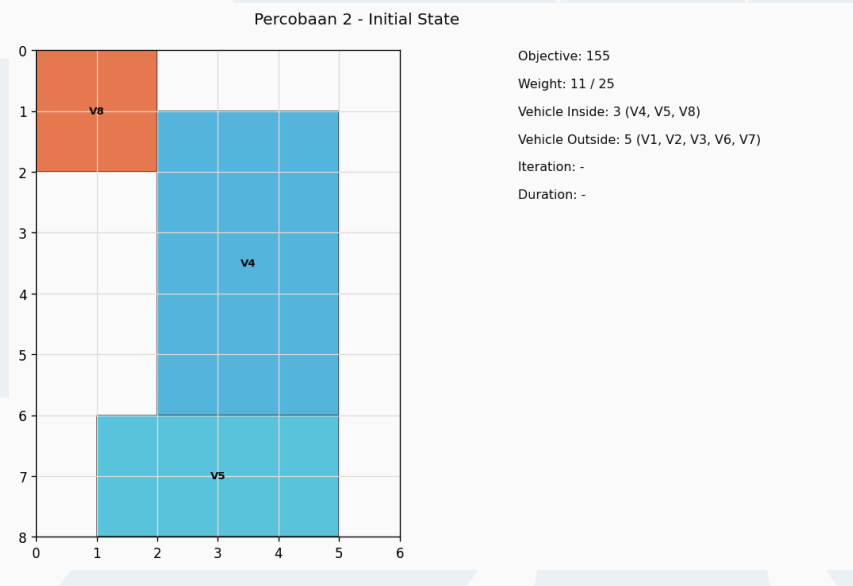
State Awal	
State Akhir	
Nilai Objective Function Terakhir	240

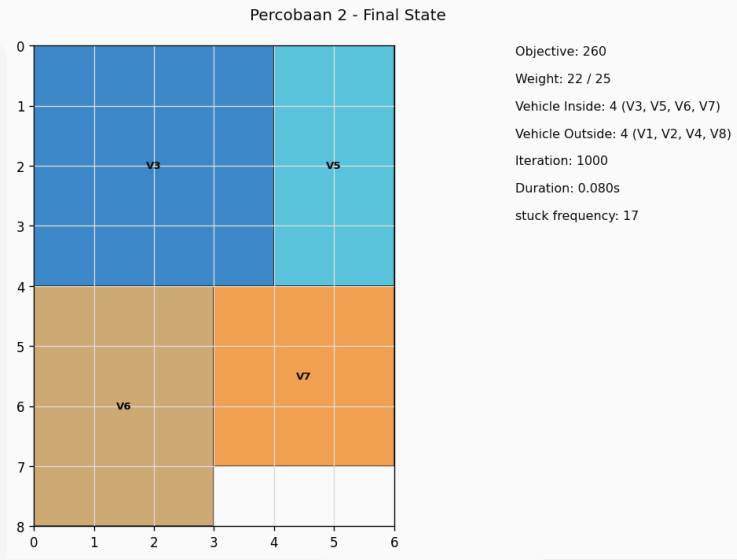
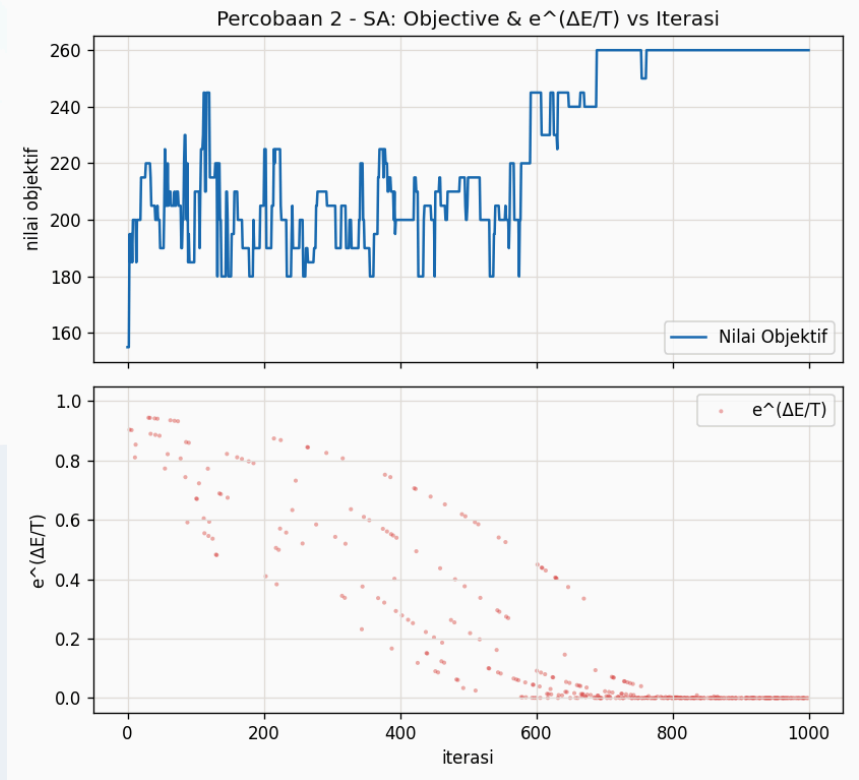
Plot Objective Function vs Iterasi	
Durasi	0.085s
Total Iterasi yang dilakukan sampai Program berhenti	15 Iterasi per restart = [1, 2, 2, 2, 3, 0, 1, 2, 1, 1]
Maximum Restart	10
Percobaan 3	
State Awal	

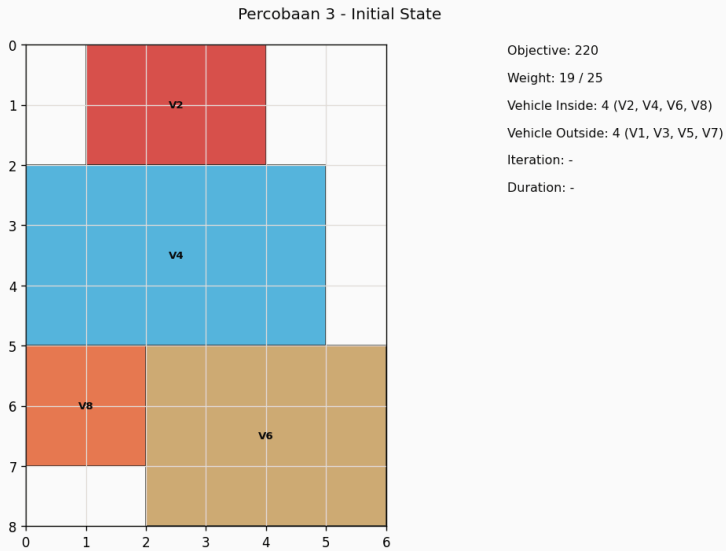
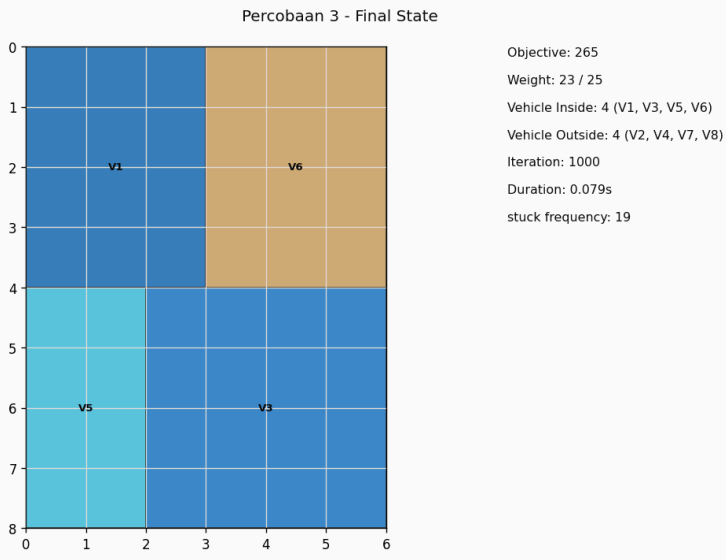
State Akhir	
Nilai Objective Function Terakhir	255
Plot Objective Function vs Iterasi	
Durasi	0.086s
Total Iterasi yang dilakukan sampai Program berhenti	15 Iterasi per restart = [1, 2, 1, 0, 2, 3, 2, 2, 1, 1]
Maximum Restart	10

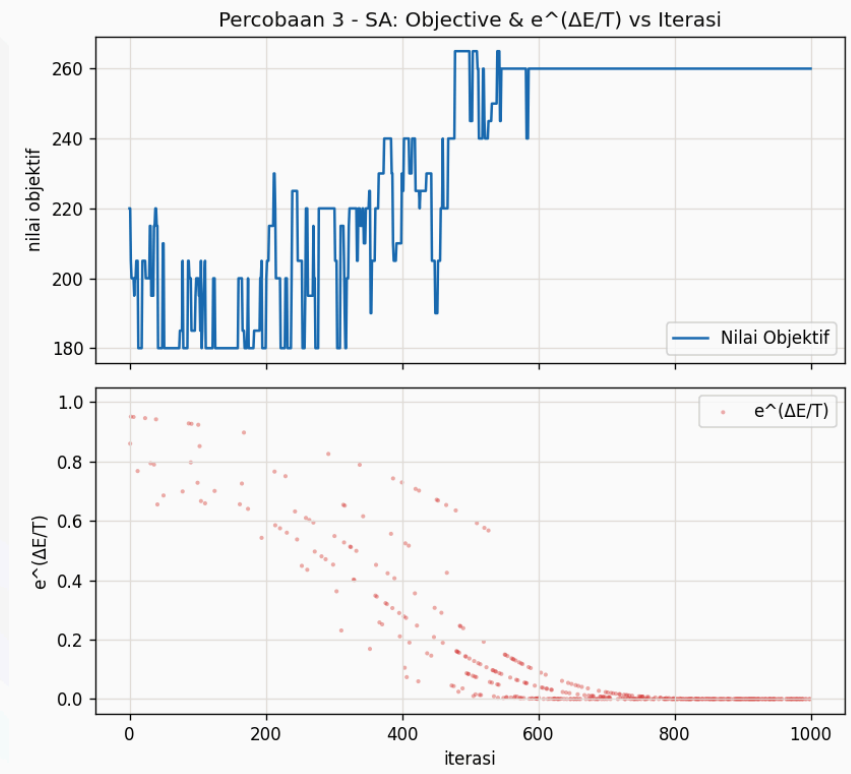
3.1.5 Simulated Annealing

Percobaan 1	
State Awal	
State Akhir	
Nilai Objective Function Terakhir	255

Plot Objective Function and Probability vs Iterasi	
Durasi	0.140s
Total Iterasi	1000
Frekuensi Stuck	18
Percobaan 2	
State Awal	

State Akhir	
Nilai Objective Function Terakhir	260
Plot Objective Function and Probability vs Iterasi	
Durasi	0.080s
Total Iterasi	1000

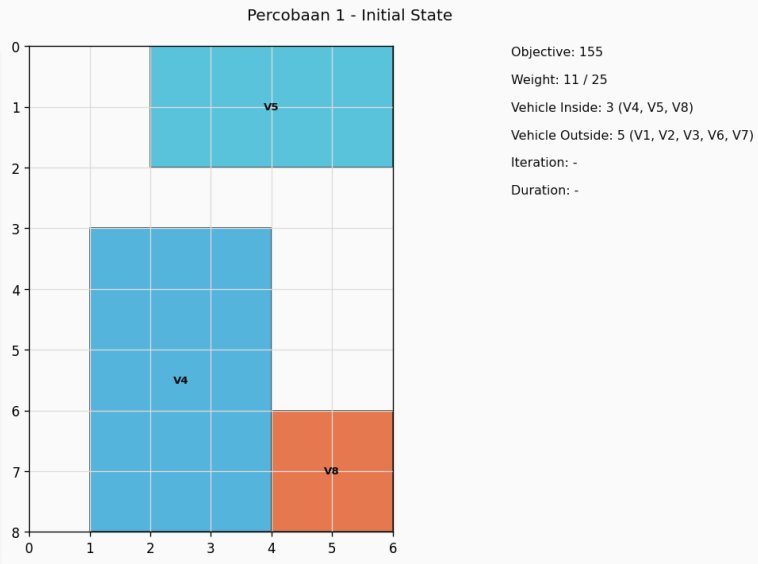
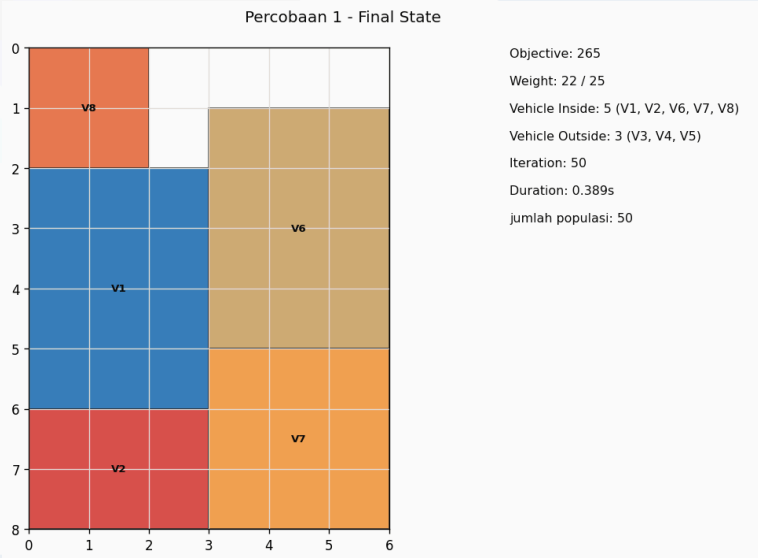
Frekuensi Stuck	17
Percobaan 3	
State Awal	
State Akhir	
Nilai Objective Function Terakhir	265

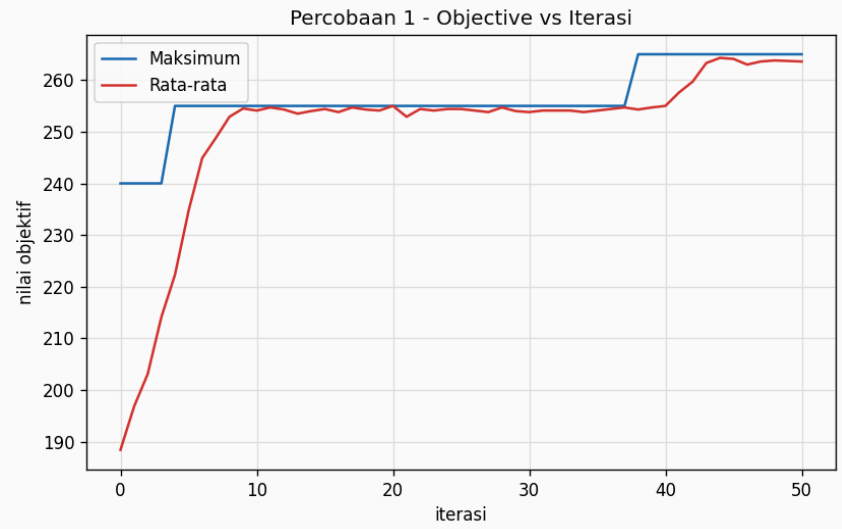
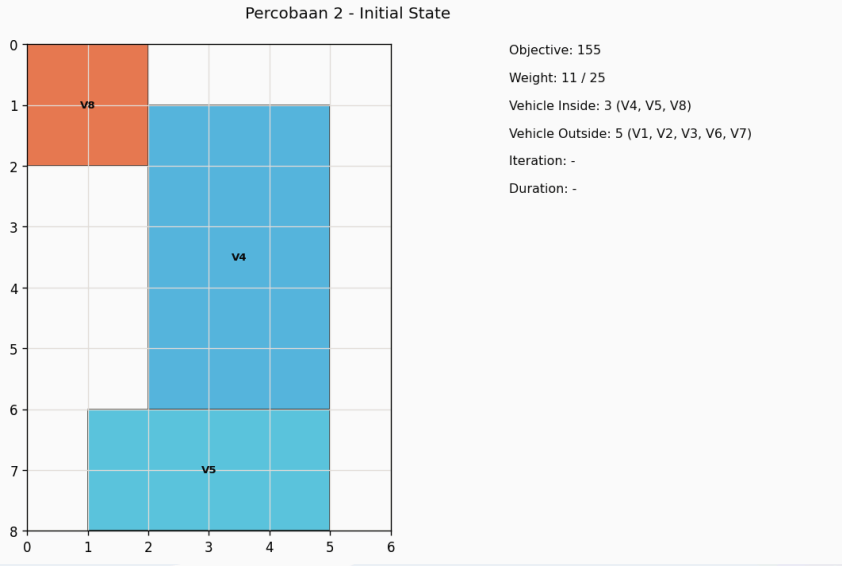
Plot Objective Function and Probability vs Iterasi	
Durasi	0.079s
Total Iterasi	1000
Frekuensi Stuck	19

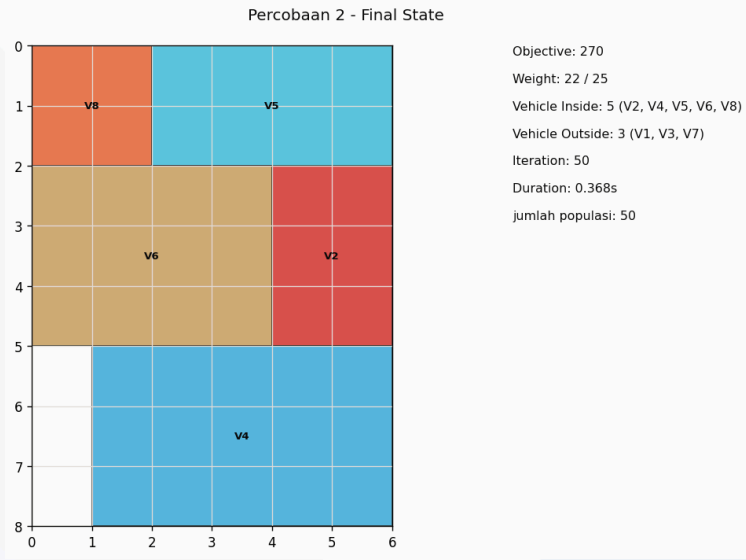
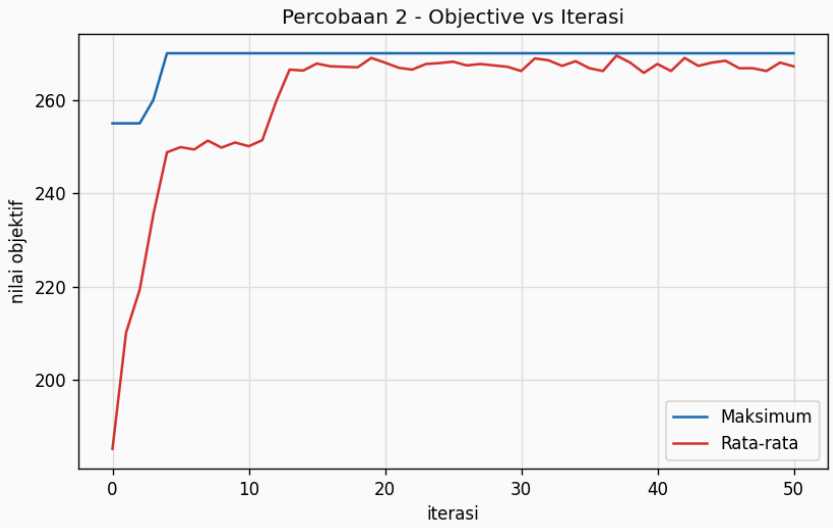
3.1.6 Genetic Algorithm

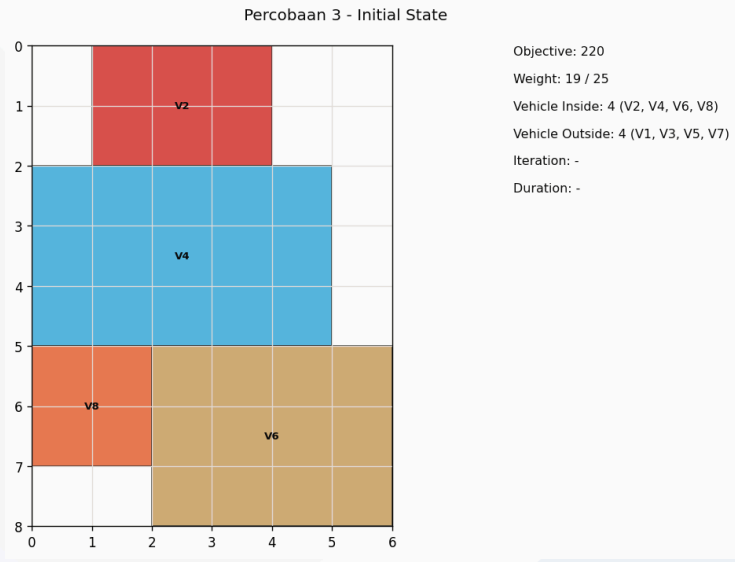
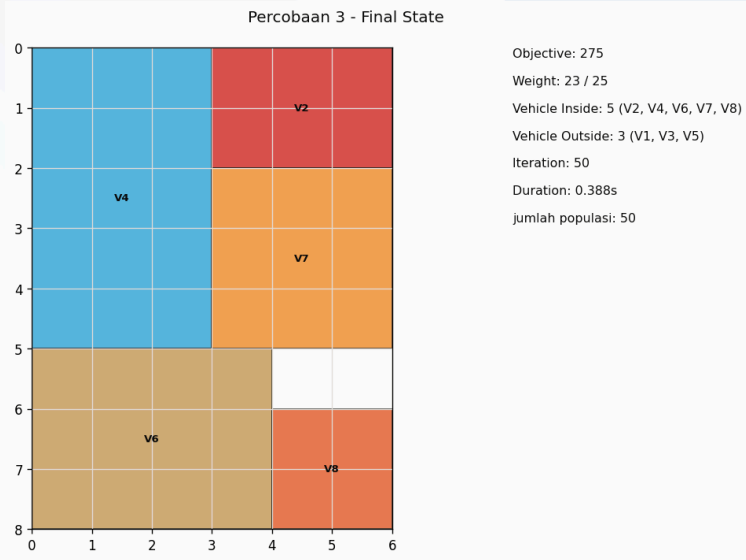
3.1.6.1 Eksperimen Populasi Kontrol = 50 dan Iterasi = 50

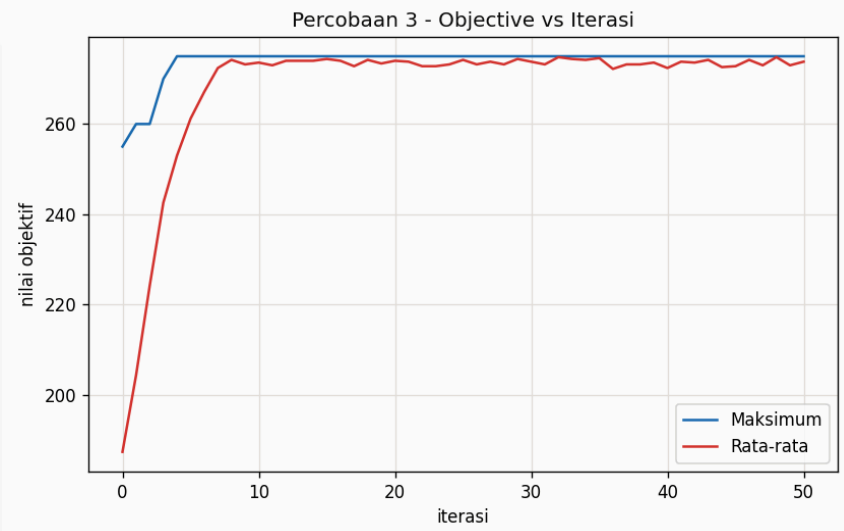
Percobaan 1

State Awal	
State Akhir	
Nilai Objective Function Terakhir	265

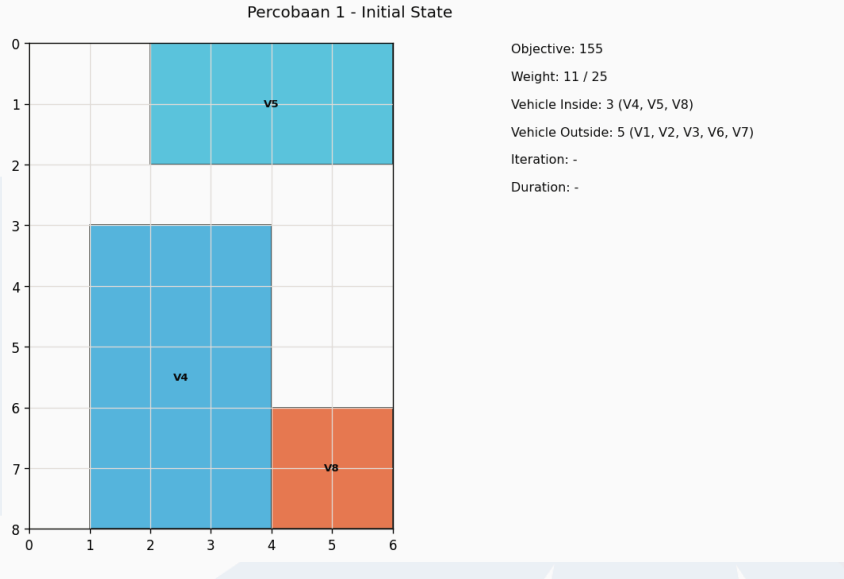
Plot Objective Function and Probability vs Iterasi	
Durasi	0.389s
Total Iterasi	50
Total Jumlah Populasi	50
Percobaan 2	
State Awal	

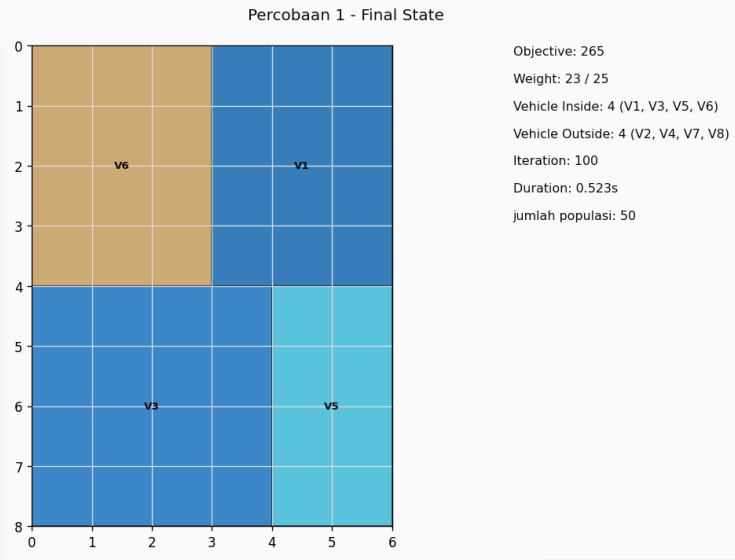
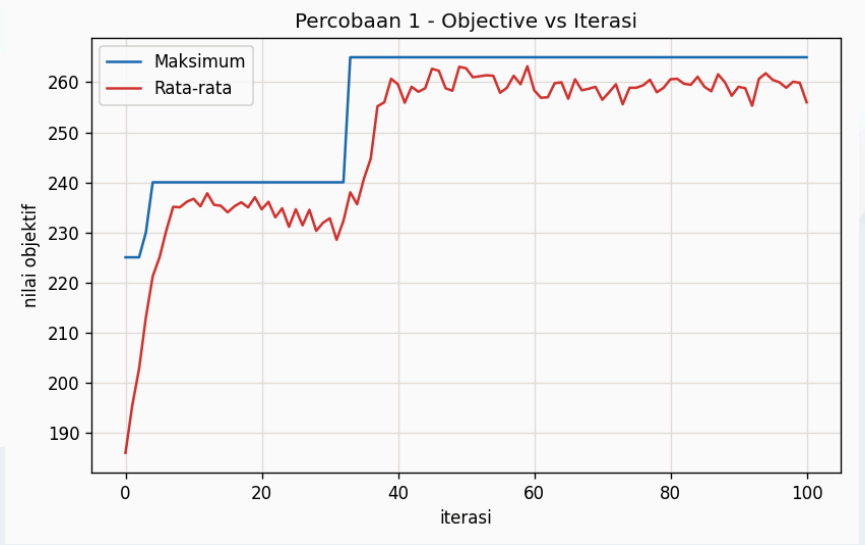
State Akhir	
Nilai Objective Function Terakhir	270
Plot Objective Function and Probability vs Iterasi	
Durasi	0.368s
Total Iterasi	50
Total Jumlah Populasi	50
Percobaan 3	

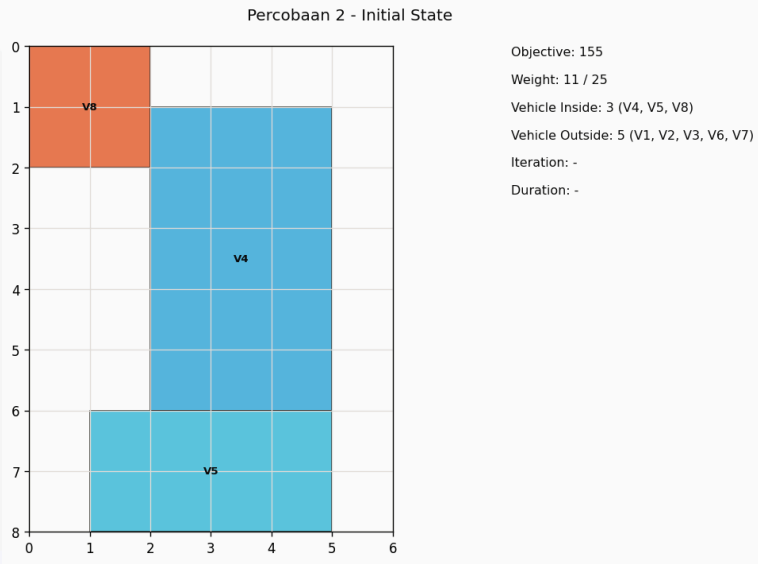
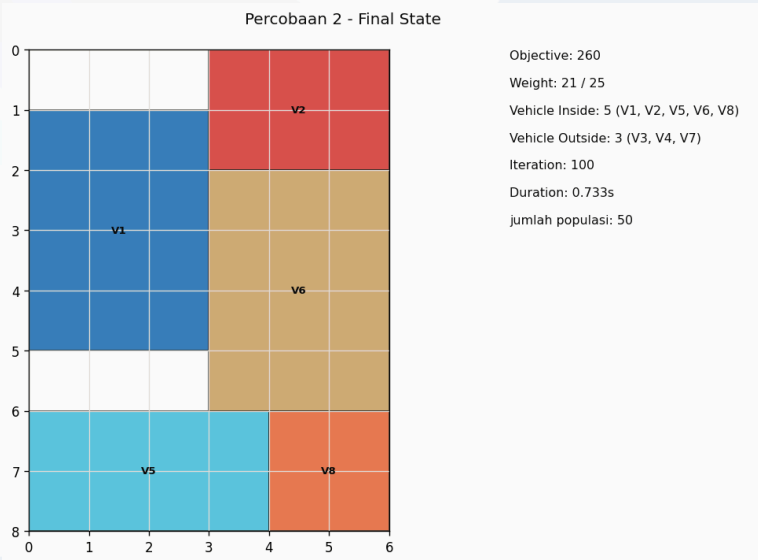
State Awal	
State Akhir	
Nilai Objective Function Terakhir	275

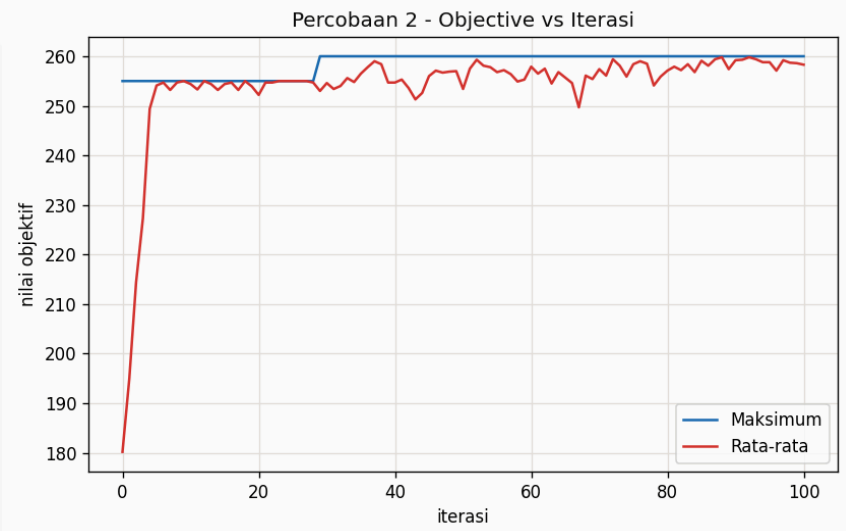
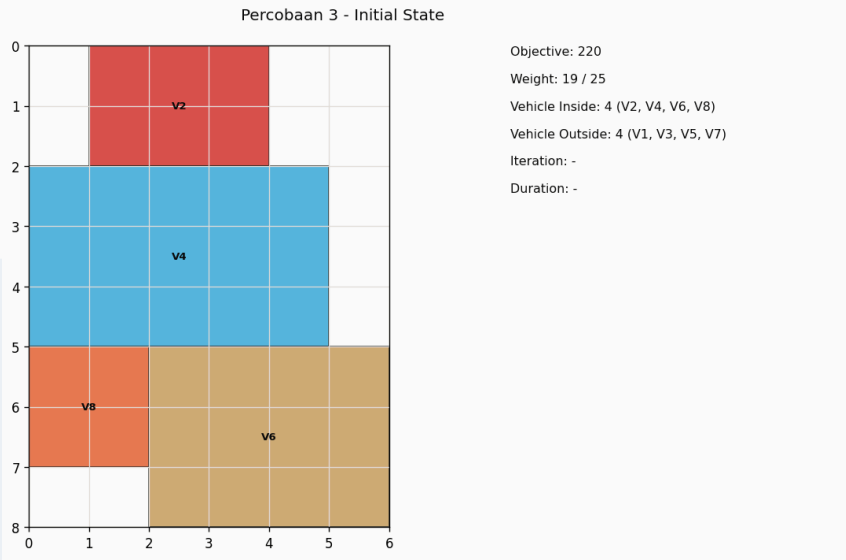
Plot Objective Function and Probability vs Iterasi	
Durasi	0.388s
Total Iterasi	50
Total Jumlah Populasi	50

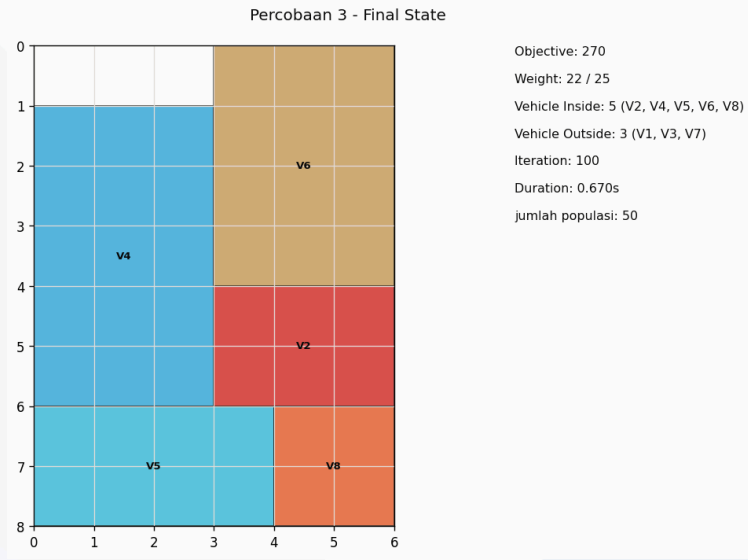
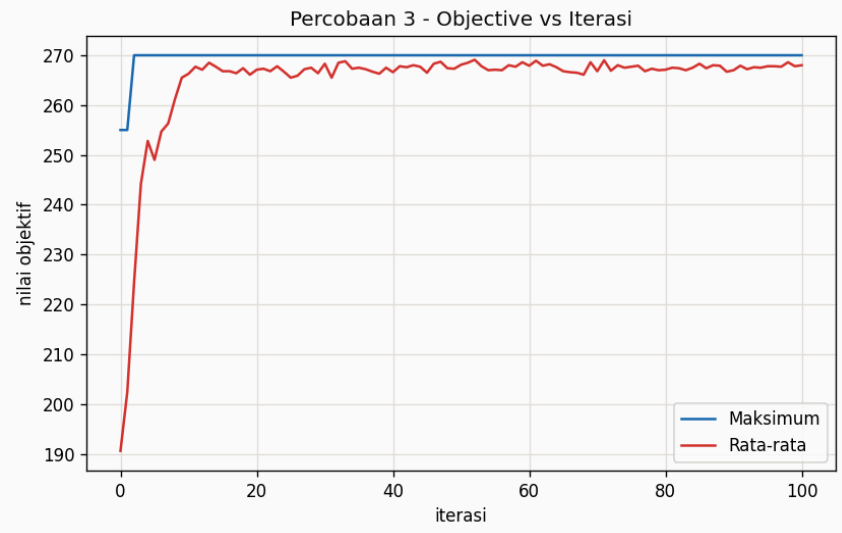
3.1.6.2 Eksperimen Populasi Kontrol = 50 dan Iterasi = 100

Percobaan 1	
State Awal	

State Akhir	
Nilai Objective Function Terakhir	265
Plot Objective Function and Probability vs Iterasi	
Durasi	0.523s
Total Iterasi	100
Total Jumlah Populasi	50
Percobaan 2	

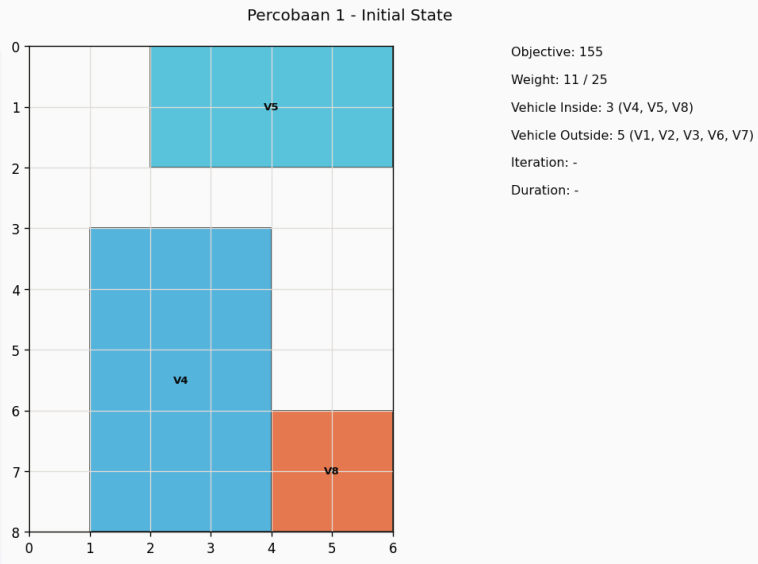
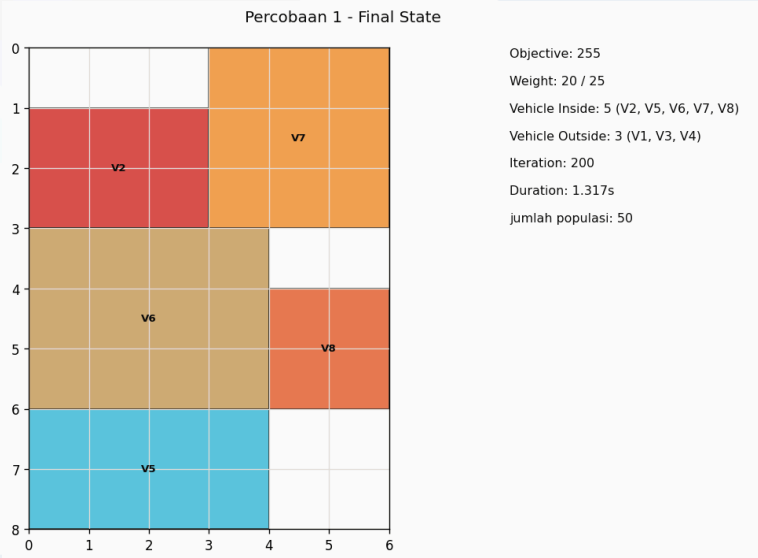
State Awal	
State Akhir	
Nilai Objective Function Terakhir	260

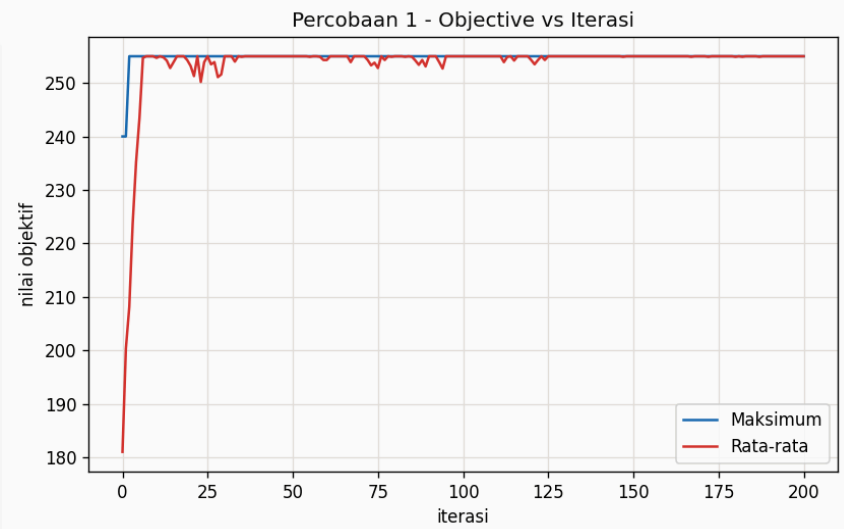
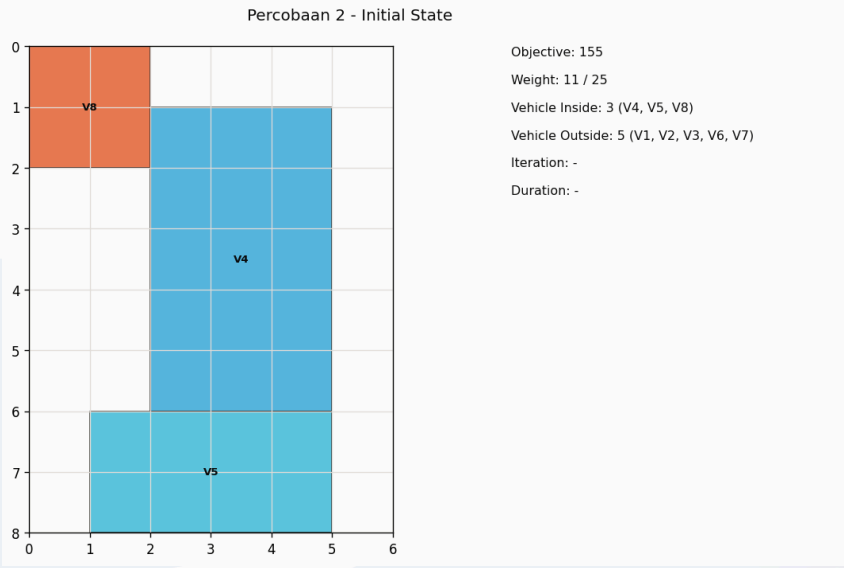
Plot Objective Function and Probability vs Iterasi	
Durasi	0.773s
Total Iterasi	100
Total Jumlah Populasi	50
Percobaan 3	
State Awal	

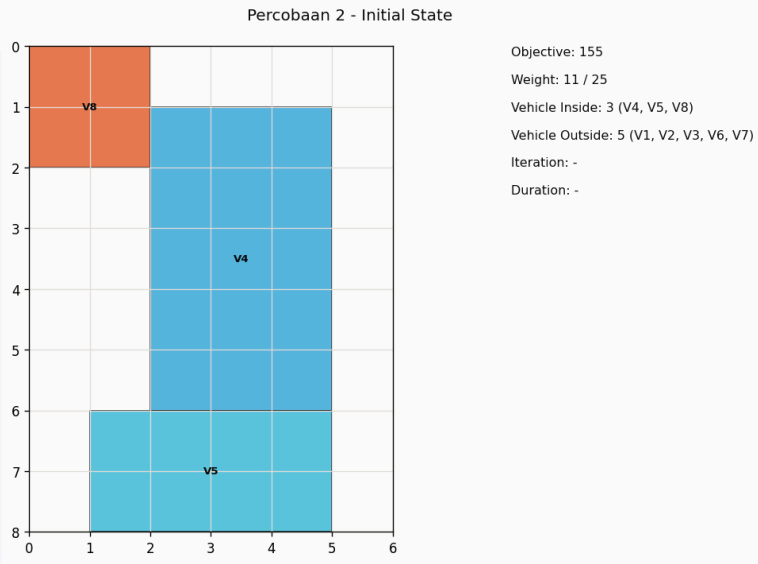
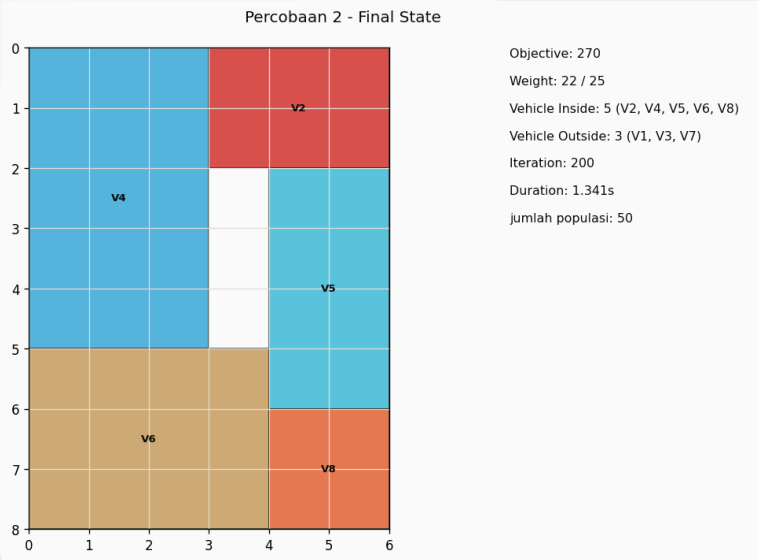
State Akhir	
Nilai Objective Function Terakhir	270
Plot Objective Function and Probability vs Iterasi	
Durasi	0.670s
Total Iterasi	100
Total Jumlah Populasi	50

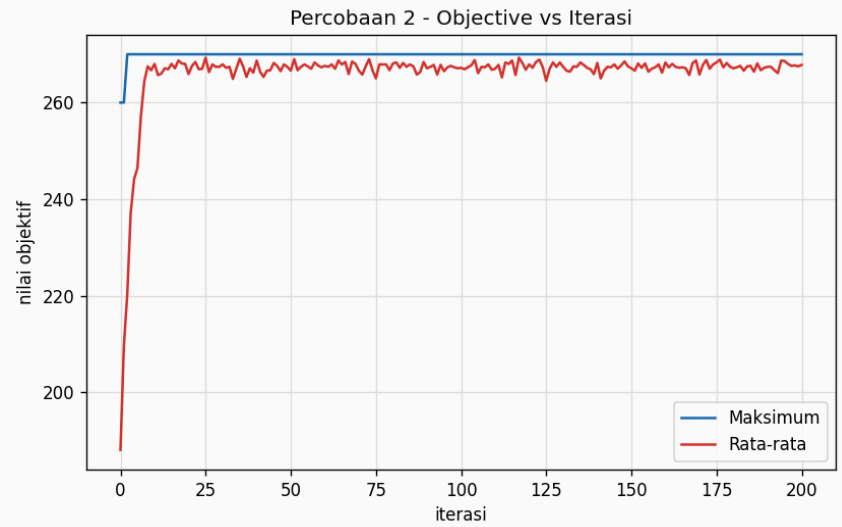
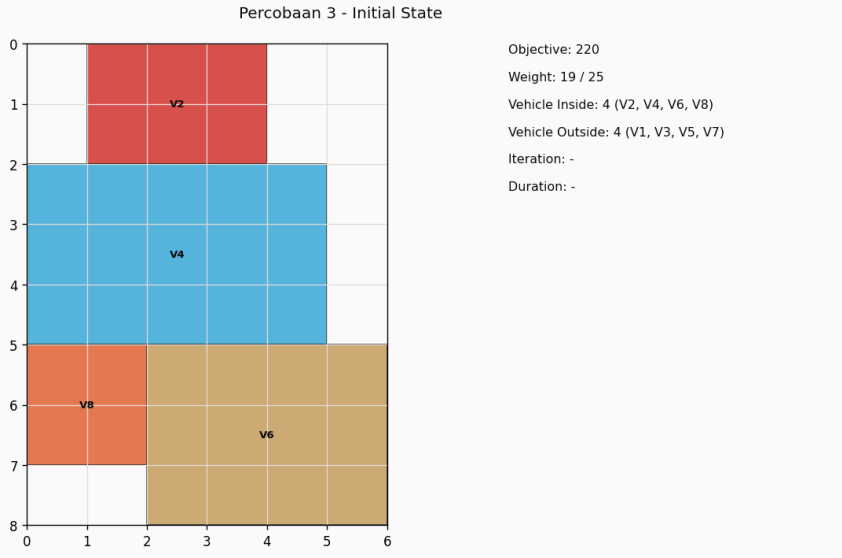
3.1.6.3 Eksperimen Populasi Kontrol = 50 dan Iterasi = 200

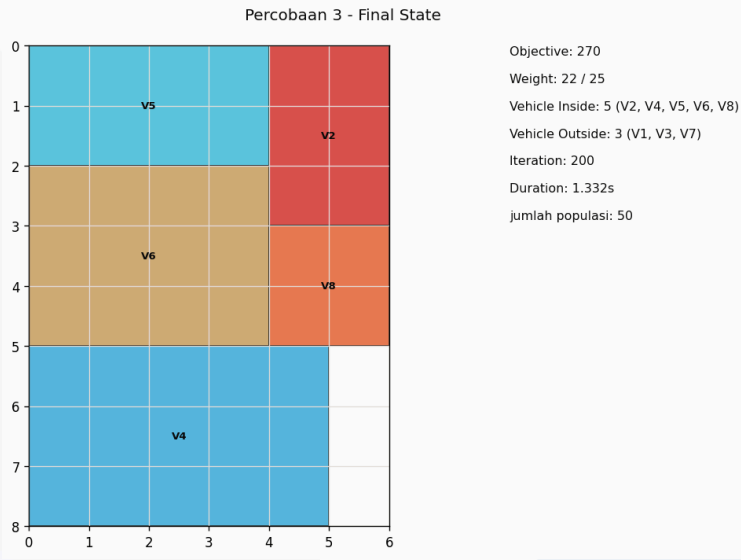
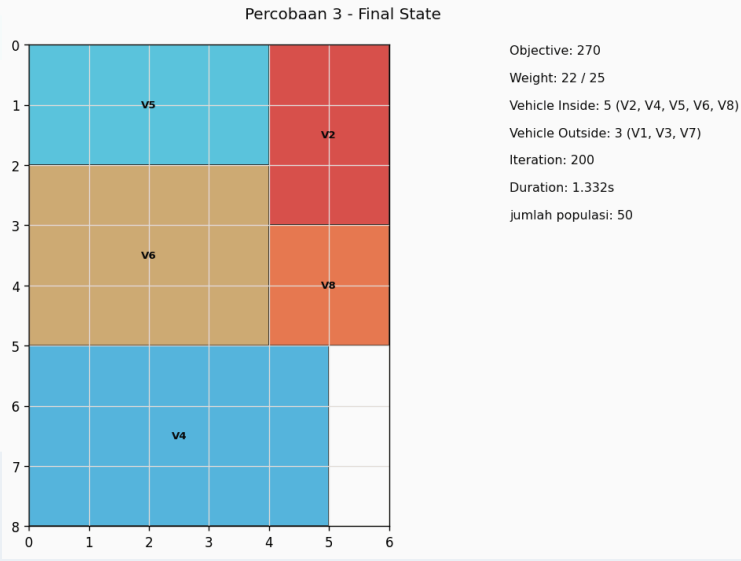
Percobaan 1

State Awal	
State Akhir	
Nilai Objective Function Terakhir	255

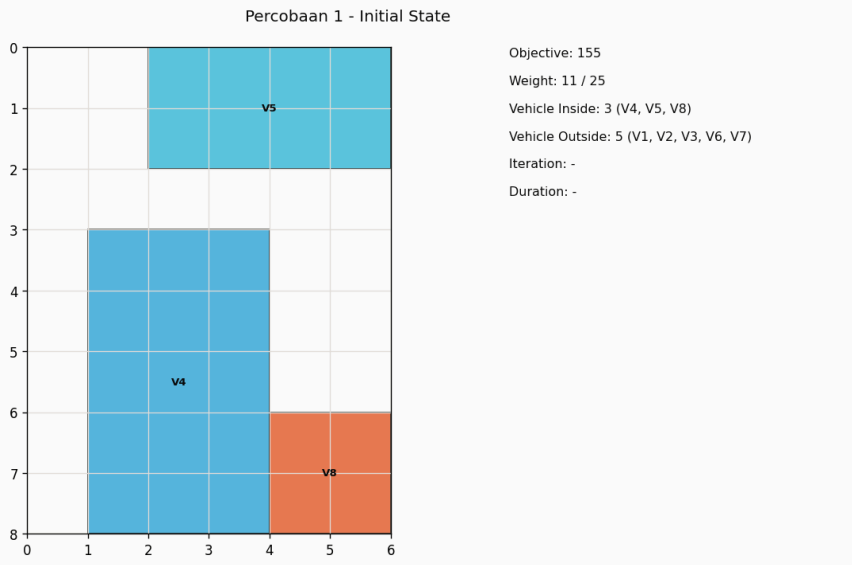
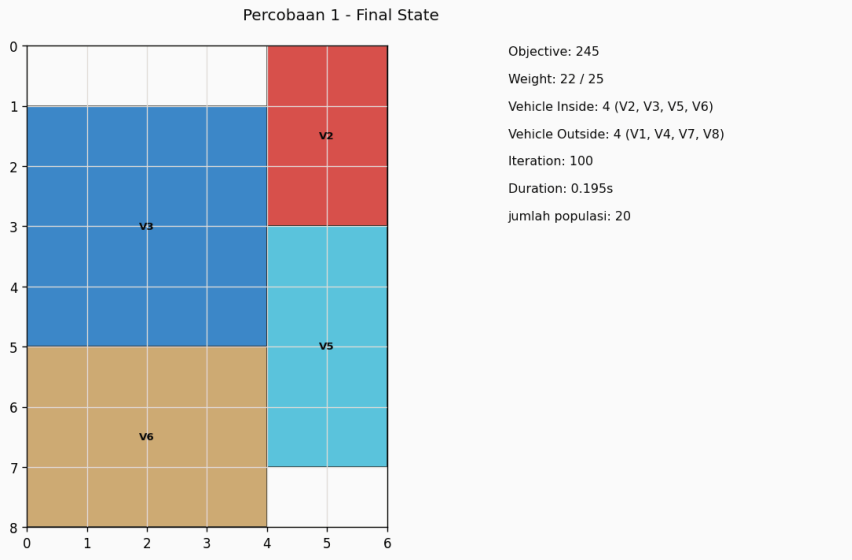
Plot Objective Function and Probability vs Iterasi	
Durasi	1.317s
Total Iterasi	200
Total Jumlah Populasi	50
Percobaan 2	
State Awal	

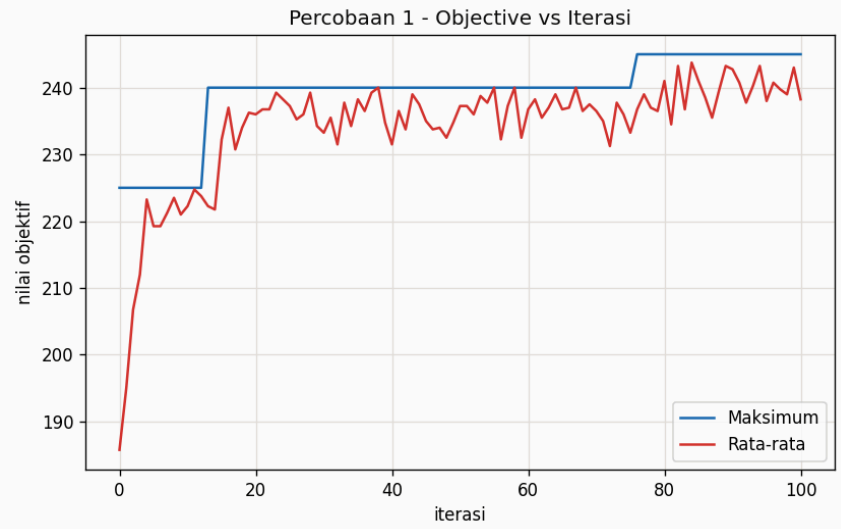
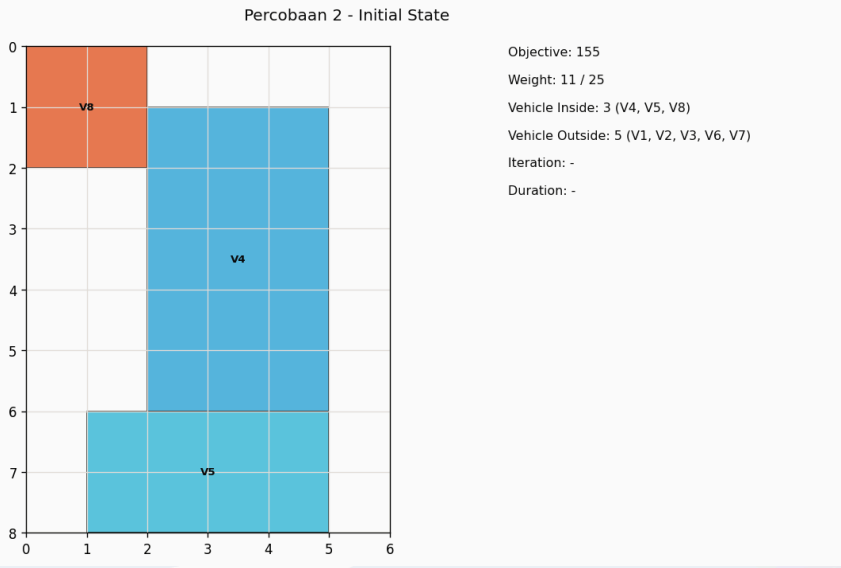
State Akhir	
Nilai Objective Function Terakhir	

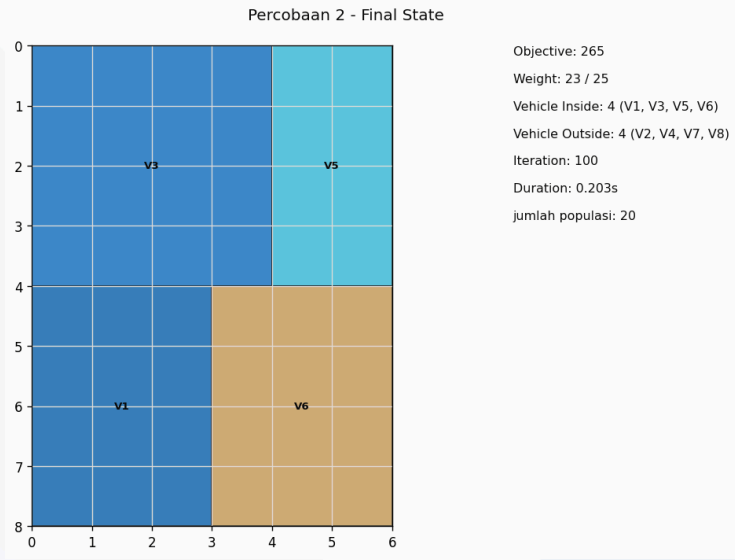
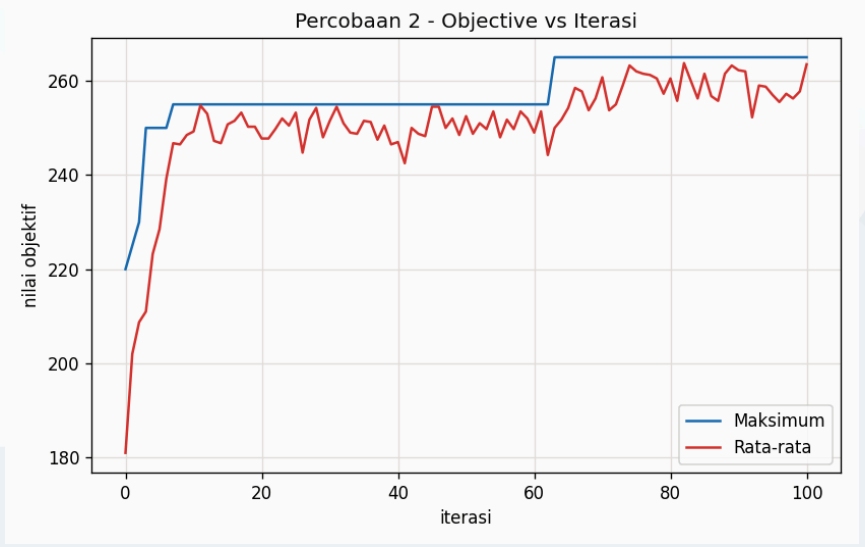
Plot Objective Function and Probability vs Iterasi	
Durasi	1.341s
Total Iterasi	200
Total Jumlah Populasi	50
Percobaan 3	
State Awal	

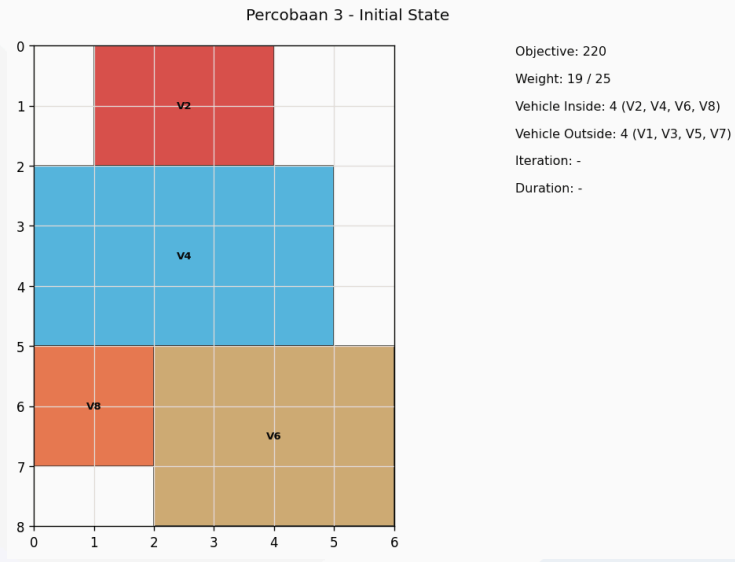
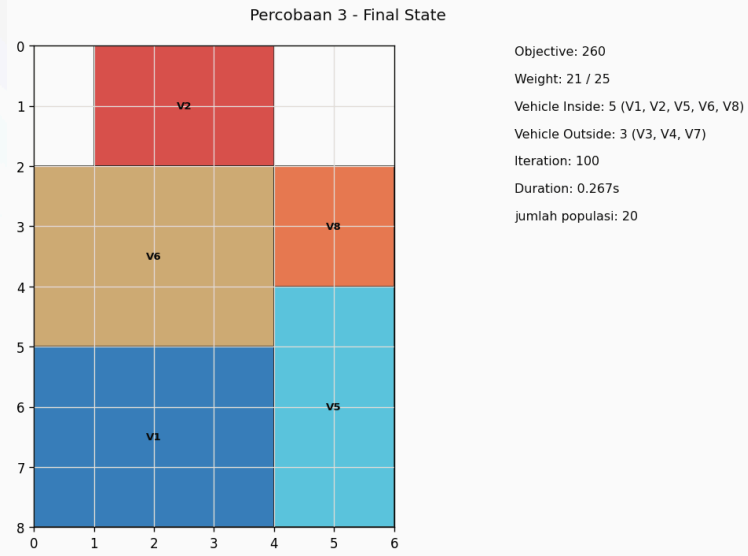
State Akhir	
Nilai Objective Function Terakhir	270
Plot Objective Function and Probability vs Iterasi	
Durasi	1.332s
Total Iterasi	200
Total Jumlah Populasi	50

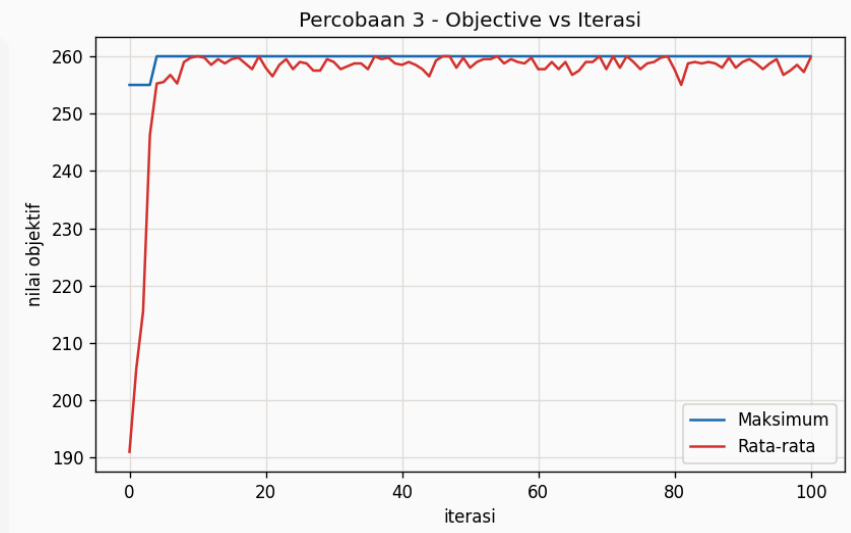
3.1.6.4 Eksperimen Iterasi Kontrol = 100 dan Populasi = 20

Percobaan 1	
State Awal	
State Akhir	
Nilai Objective Function Terakhir	245

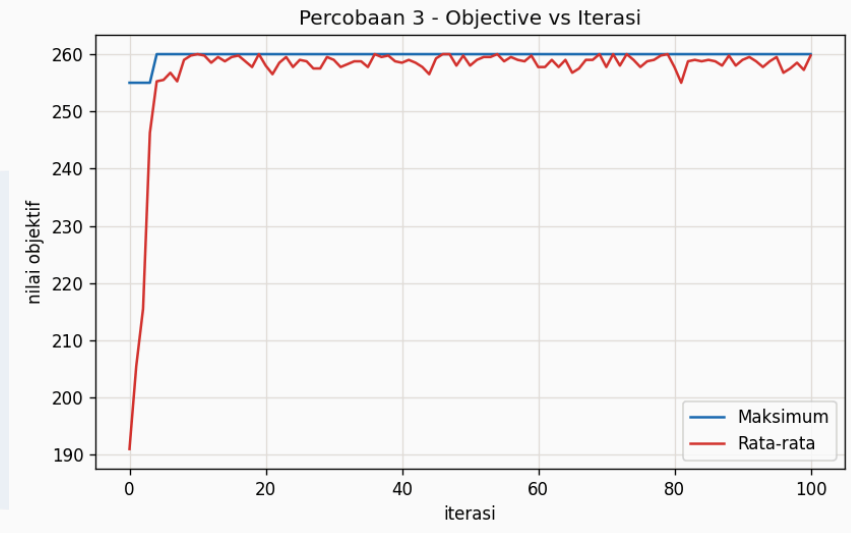
Plot Objective Function and Probability vs Iterasi	
Durasi	0.195s
Total Iterasi	100
Total Jumlah Populasi	20
Percobaan 2	
State Awal	

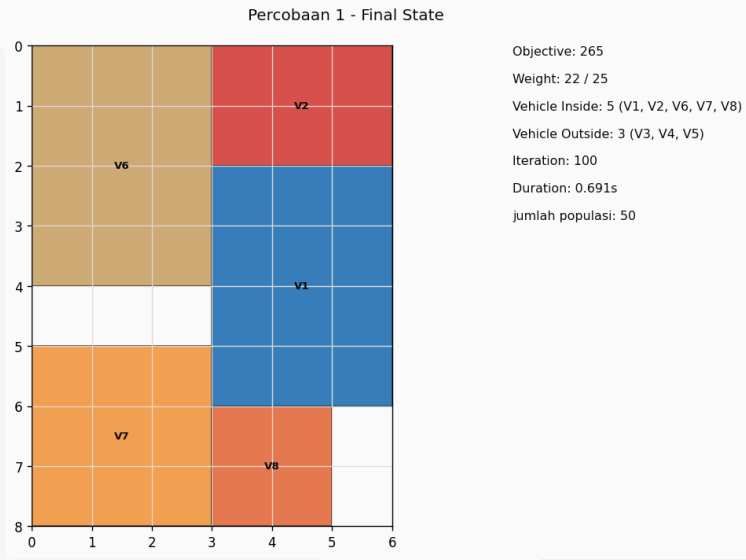
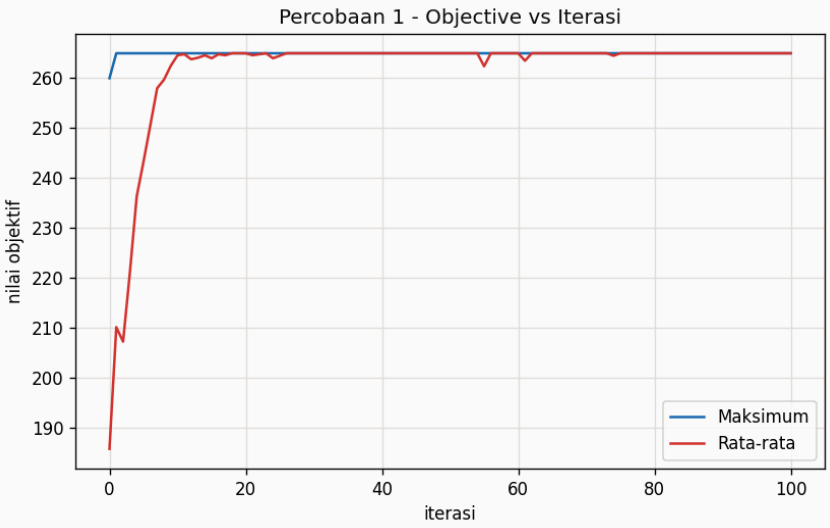
State Akhir	
Nilai Objective Function Terakhir	265
Plot Objective Function and Probability vs Iterasi	
Durasi	0.203
Total Iterasi	100
Total Jumlah Populasi	20
Percobaan 3	

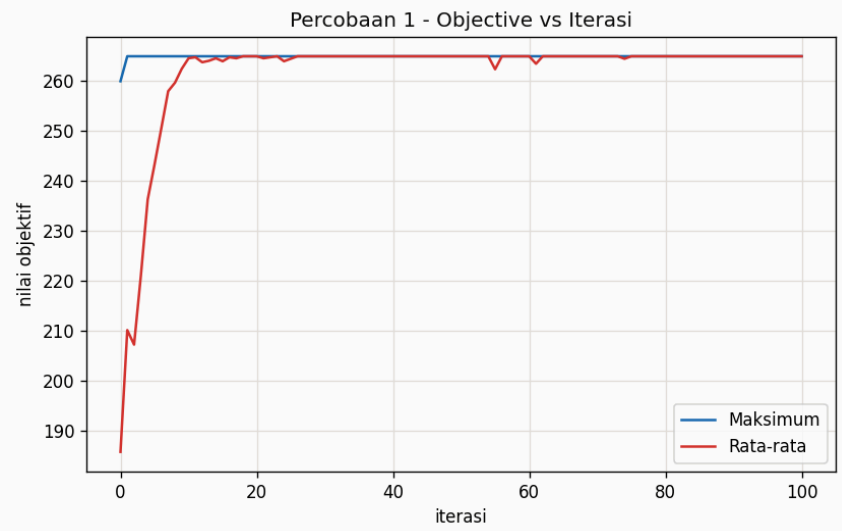
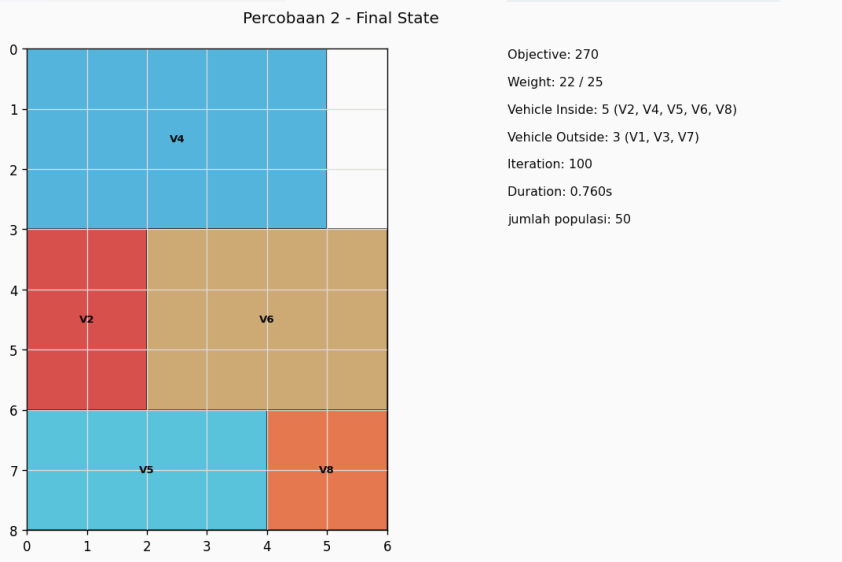
State Awal	
State Akhir	
Nilai Objective Function Terakhir	260

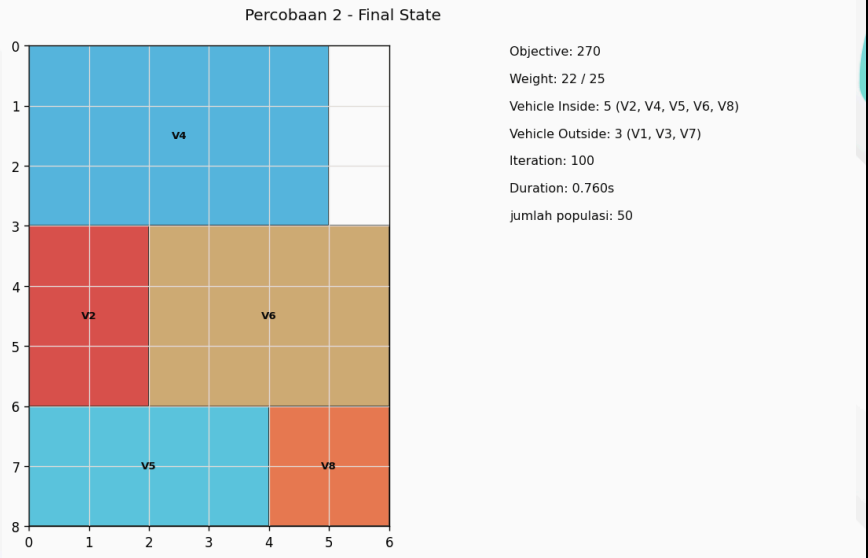
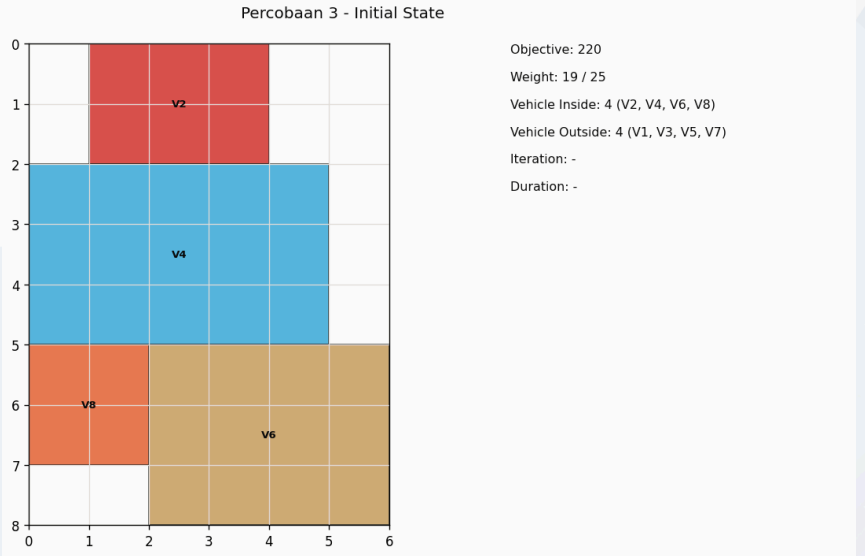
Plot Objective Function and Probability vs Iterasi	
Durasi	0.267s
Total Iterasi	100
Total Jumlah Populasi	20

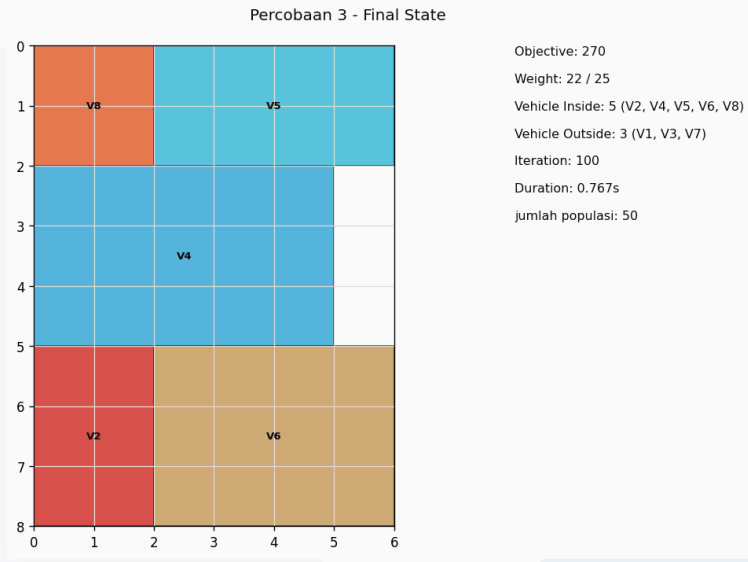
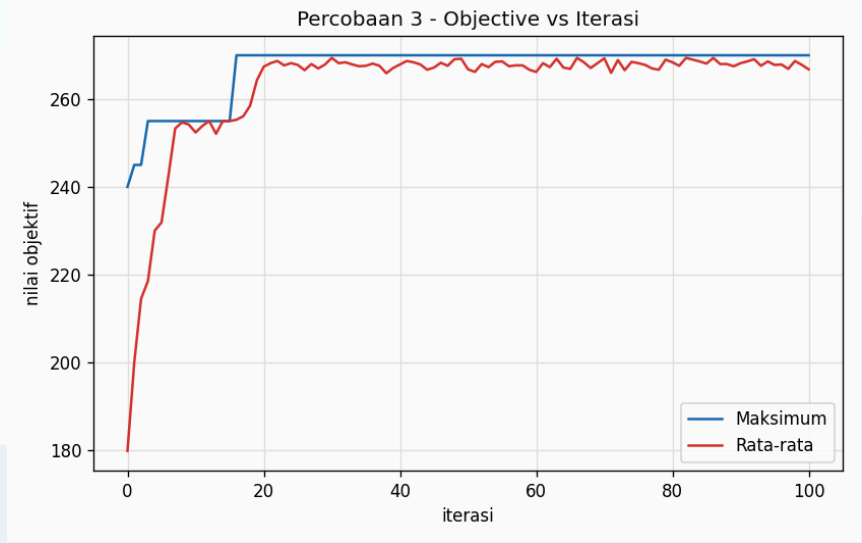
3.1.6.5 Eksperimen Iterasi Kontrol = 100 dan Populasi = 50

Percobaan 1	
State Awal	

State Akhir	
Nilai Objective Function Terakhir	265
Plot Objective Function and Probability vs Iterasi	
Durasi	0.691s
Total Iterasi	100
Total Jumlah Populasi	50
Percobaan 2	

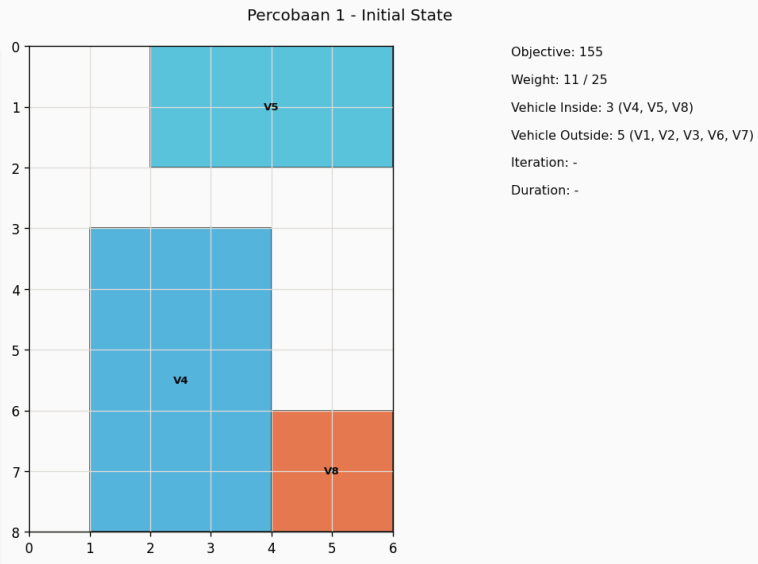
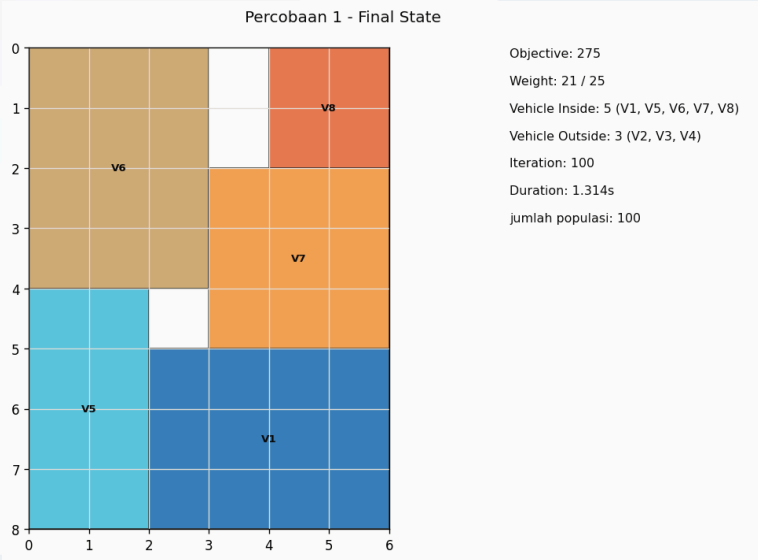
State Awal	
State Akhir	
Nilai Objective Function Terakhir	270

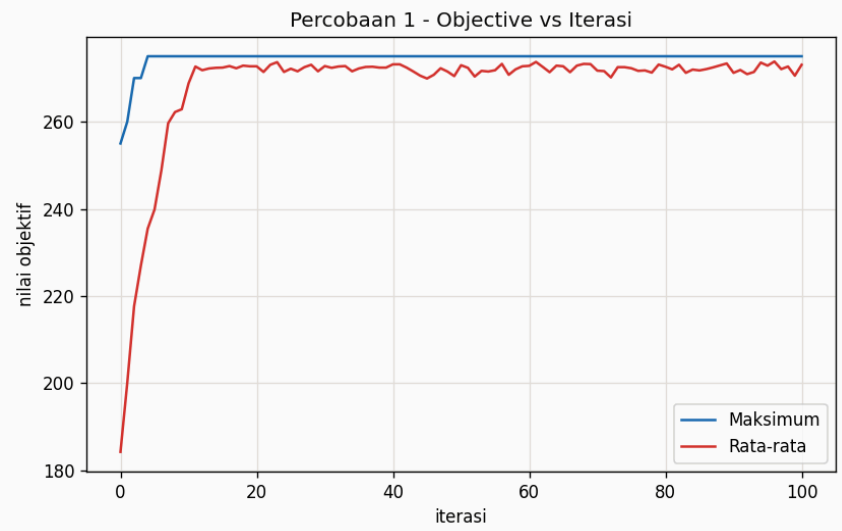
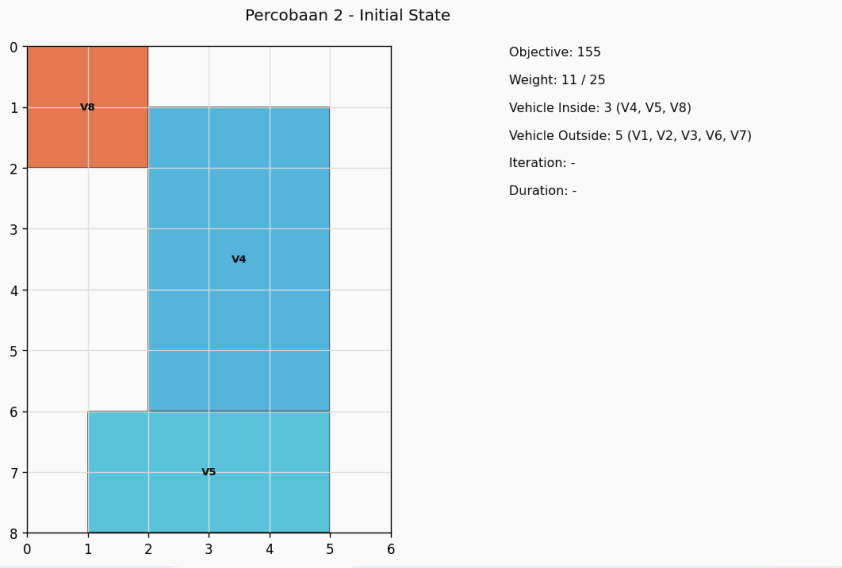
Plot Objective Function and Probability vs Iterasi	
Durasi	0.760s
Total Iterasi	100
Total Jumlah Populasi	50
Percobaan 3	
State Awal	

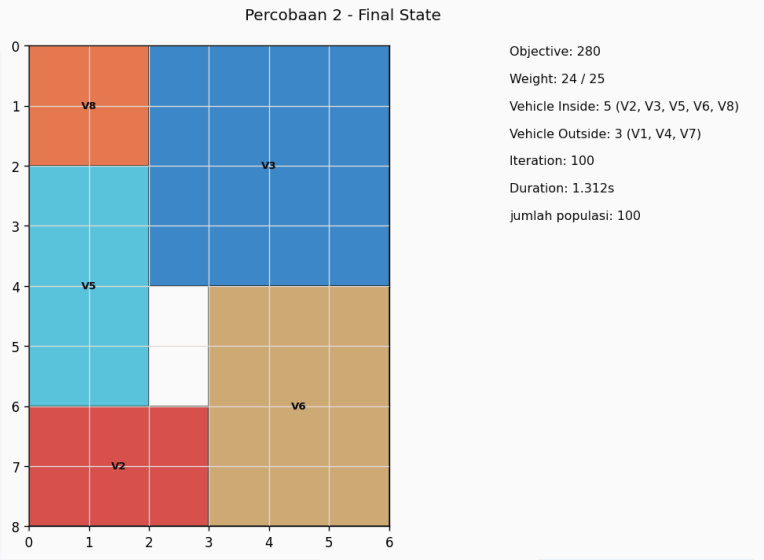
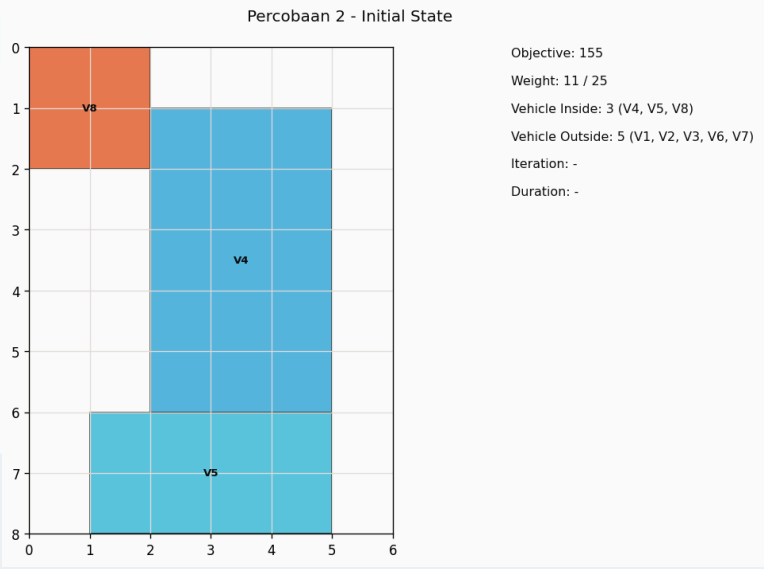
State Akhir	
Nilai Objective Function Terakhir	270
Plot Objective Function and Probability vs Iterasi	
Durasi	0.767s
Total Iterasi	100
Total Jumlah Populasi	50

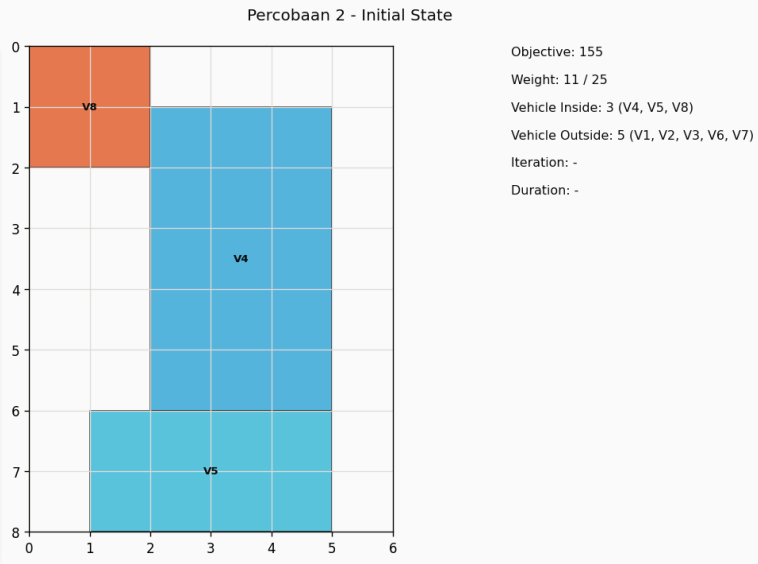
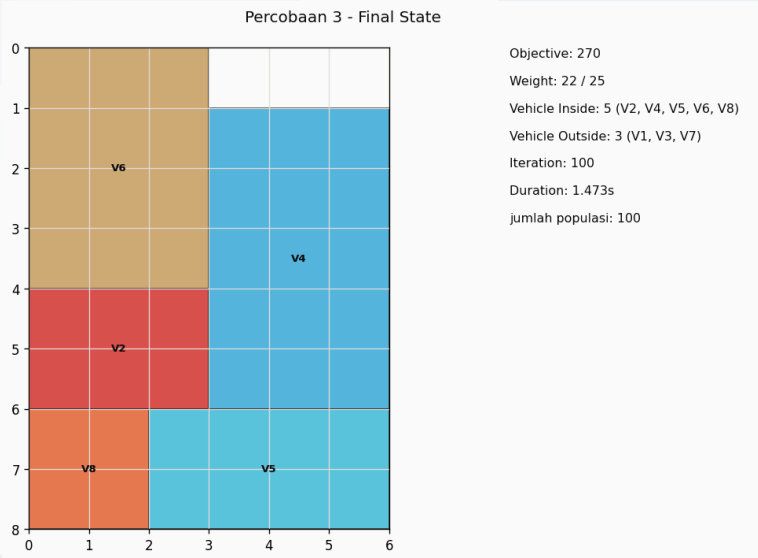
3.1.6.6 Eksperimen Iterasi Kontrol = 100 dan Populasi = 100

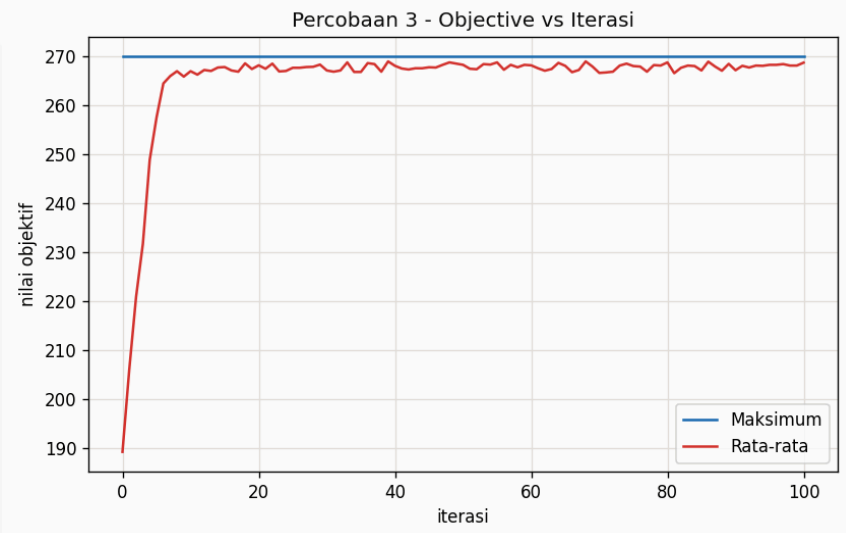
Percobaan 1

State Awal	
State Akhir	
Nilai Objective Function Terakhir	275

Plot Objective Function and Probability vs Iterasi	
Durasi	1.314s
Total Iterasi	100
Total Jumlah Populasi	100
Percobaan 2	
State Awal	

State Akhir	
Nilai Objective Function Terakhir	280
Plot Objective Function and Probability vs Iterasi	
Durasi	1.312s
Total Iterasi	100
Total Jumlah Populasi	100
Percobaan 3	

State Awal	
State Akhir	
Nilai Objective Function Terakhir	270

Plot Objective Function and Probability vs Iterasi	
Durasi	1.473s
Total Iterasi	100
Total Jumlah Populasi	100

3.2 Analisis Eksperimen

Pertanyaan 1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	Steepest ascent hill climbing meningkatkan eksplorasi dengan cara **menghasilkan semua possible neighbors pada setiap iterasi**, kemudian memilih **neighbor terbaik** diantara semua kemungkinan tersebut. Algoritma ini lebih informed bila dibandingkan dengan stochastic hill climbing karena algoritma selalu bergerak ke arah penurunan fungsi objektif yang paling curam. Hal ini membuat algoritma steepest ascent hill climbing **lebih efisien dalam menemukan local optimum** yang memiliki **kualitas lebih baik**. Namun, algoritma ini juga memiliki keterbatasan yaitu apabila **mencapai local optimum**, maka **algoritma akan terjebak dan tidak bisa keluar** karena tidak memiliki mekanisme untuk menerima solusi yang lebih buruk sementara.

Sideways Move Hill Climbing	Sideways move hill climbing memiliki kemampuan untuk menerima neighbor dengan nilai fungsi objektif yang sama. Strategi ini sangat krusial untuk **mengatasi "flat" local maximum (plateau).** Dengan memberi batasan maksimal gerakan sideways berturut-turut, algoritma bisa **melewati plateau dan berpotensi menemukan path yang menuju ke local optimum yang lebih baik**. Kemampuan ini membuat sideways move hill climbing lebih baik dari steepest ascent hill climbing dalam segi eksplorasi search space.
Stochastic Hill Climbing	Stochastic hill climbing merupakan variasi paling sederhana dari algoritma hill climbing yang bekerja dengan **menghasilkan satu neighbor secara random pada setiap iterasi**, kemudian akan **menerima neighbor tersebut apabila lebih baik dari current state**. Algoritma stochastic hill climbing memiliki kemampuan paling rendah dalam mendekati global optima karena memiliki sifat 'greedy' (hanya melihat satu langkah ke depan). Ketika **algoritma sudah menemukan local optimum (kondisi di mana tidak ada neighbor yang lebih baik), maka algoritma akan stuck**.
Random Restart Hill Climbing	Random restart hill climbing menjadi salah satu strategi yang memiliki fundamental berbeda yaitu **mengatasi kondisi hill climbing yang stuck di local optimum dengan cara melakukan random restarting**. Konsep dasarnya sederhana dan sangat berdampak, apabila dijalankan algoritma hill climbing biasa (steepest ascent hill climbing atau stochastic hill climbing) berkali-kali dari initial states yang berbeda-beda, maka masing-masing run akan memiliki **local optimum yang berbeda-beda dan bisa diambil yang terbaik diantara semua hasil yang diperoleh**. Hal ini merupakan bentuk global optimization melalui sampling dengan cukup banyak restart, maka probabilitas untuk menemukan global optimum menjadi tinggi.
Simulated Annealing	Algoritma simulated annealing **memiliki kemampuan untuk keluar dari local optimum** dengan mekanisme **acceptance probability**. Pada **temperatur yang tinggi, algoritma ini bisa menerima solusi yang lebih buruk dengan probabilitas tinggi** sehingga memungkinkan **eksplorasi lebih luas**. Sementara, pada **temperatur yang rendah, algoritma ini bisa menjadi lebih selektif** dan konvergen ke local optimum terbaik yang ditemukan.
Genetic Algorithm	Genetic algorithm memiliki mekanisme bahwa **solusi terbaik**

	dari setiap generasi di-preserve dan di-combine untuk menciptakan **offspring yang berpotensi lebih baik**. Kelebihan algoritma ini terletak pada **implicit parallelism**, algoritma ini tidak me-maintain satu solusi seperti algoritma sebelum-sebelumnya, melainkan secara paralel **mengeksplorasi multiple regions dalam search space**. Algoritma ini memanfaatkan pendekatan evolusi biologis, yaitu **proses yang meniru cara makhluk hidup beradaptasi dan berevolusi melalui seleksi alam**, dimana setiap solusi dipandang sebagai individu dalam sebuah populasi, yang kemudian "bereproduksi" melalui tiga operator utama yaitu **selection, crossover, dan mutation**.
Pertanyaan 2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	Algoritma steepest ascent hill climbing menunjukkan kemampuan eksploitasi yang tinggi karena selalu memilih tetangga dengan peningkatan nilai objektif terbaik pada setiap iterasi. Dalam permasalahan ini, pendekatan ini **dapat dengan cepat menemukan konfigurasi pengisian kendaraan yang efisien di awal proses. Namun, algoritma ini rentan terjebak pada local optimum**, terutama ketika perubahan kecil tidak lagi menghasilkan perbaikan nilai. **Keunggulan** algoritma ini terletak pada **kecepatan dan kemudahan implementasi**, tetapi **kelemahannya** ada pada **kurangnya fleksibilitas** eksplorasi ruang solusi.
Sideways Move Hill Climbing	Algoritma sideways move hill climbing **memungkinkan untuk menjelajahi daerah plateau dan keluar dari situasi di mana perubahan kecil tidak memberikan perbaikan langsung**. Dibandingkan steepest ascent, rata-rata objective naik dari 191,67 menjadi 243,33 (sekitar +27%) hanya dengan **menambahkan izin bergerak ke neighbor bernilai sama**. Meski begitu, **efektivitasnya masih tergantung pada batas jumlah sideways move yang diperbolehkan**
Stochastic Hill Climbing	Stochastic hill climbing menghasilkan nilai akhir yang sama dengan steepest ascent (180, 165, 230 dengan rata-rata 191,67) sehingga kualitasnya setara dan menjadi yang terendah bersama steepest ascent. Dibandingkan sideways move, random restart, simulated annealing, dan genetic algorithm, algoritma ini kalah **karena hanya menerima neighbor yang lebih baik dan**

	tidak punya mekanisme untuk keluar dari local optimum. Pemilihan neighbor secara acak tidak memberi keuntungan berarti pada kasus ini, pencarian berhenti setelah 1 sampai 2 iterasi dan jatuh ke local optimum yang sama dengan steepest ascent.
Random Restart Hill Climbing	Meski miliki **biaya komputasi yang meningkat karena banyaknya restart**, waktu tambahan tersebut **terbayar oleh kestabilan dan akurasinya yang tinggi**. Random restart hill climbing efektif karena mengkombinasikan kecepatan pencarian lokal dengan eksplorasi global yang lebih luas. Rata-rata objective 253,33 jauh lebih tinggi dari steepest ascent dan stochastic (+32%) serta sedikit di atas sideways move (+4%). Namun masih di bawah simulated annealing dan genetic algorithm karena **setiap restart tetap berupa hill climbing** yang **tidak dapat keluar dari local optimum**.
Simulated Annealing	Dengan adanya **mekanisme pendinginan** (*cooling schedule*), algoritma simulated annealing memungkinkan **penerimaan solusi yang lebih buruk** pada tahap awal untuk **menghindari jebakan local optimum**. Seiring **penurunan suhu**, pencarian menjadi **lebih selektif** hingga akhirnya mendekati konvergensi stabil pada solusi global. **Kekurangannya** adalah waktu **komputasi yang lebih lama** dibandingkan algoritma lain karena proses pendinginan yang bertahap, namun hal ini **sebanding dengan stabilitas dan kualitas hasil** yang diperoleh.
Genetic Algorithm	Konfigurasi paling lemah tetap mencapai 256,67, setara dengan random restart. Keunggulannya adalah **kemampuan menjaga keragaman solusi**, sehingga cocok untuk **masalah optimasi skala besar**. Namun, parameterisasi yang kurang tepat dapat menyebabkan **waktu eksekusi lebih lama** atau hasil yang kurang stabil.
Pertanyaan 3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	Durasi 0.008s, 0.006s, dan 0.006s (rata-rata 0.007s) dengan hanya 1 iterasi. Ini adalah yang **tercepat bersama stochastic**, karena algoritma **langsung berhenti** ketika **tidak ada neighbor yang lebih baik.** Walaupun tiap iterasi mengevaluasi seluruh neighbor, jumlah iterasinya sangat sedikit sehingga total waktunya kecil.

Sideways Move Hill Climbing	Durasi 0.308s, 0.361s, dan 0.318s (rata-rata 0.329s) dengan 101 sampai dengan 106 iterasi. Waktunya sekitar **47 x lebih lama** dari steepest ascent, karena **setiap iterasi tetap mengevaluasi seluruh neighbor dan algoritma harus menghabiskan hingga 100 sideways move di plateau**.
Stochastic Hill Climbing	Durasi 0.008s, 0.008s, dan 0.006s (rata-rata 0.007s) dengan 1 sampai 2 iterasi. Waktunya **setara dengan steepest ascent sehingga tercepat bersama steepest ascent**, karena hanya membangkitkan satu neighbor per iterasi dan berhenti setelah beberapa iterasi.
Random Restart Hill Climbing	Durasi 0.080s, 0.085s, dan 0.086s (rata-rata 0.084s) dengan total 12 sampai 15 iterasi untuk 10 restart. Waktunya sekitar 12 kali steepest ascent, sesuai dengan banyaknya restart (setiap restart sekitar 0 sampai 4 iterasi), dan **lebih cepat dari sideways move** karena tiap restart berhenti dengan cepat.
Simulated Annealing	Durasi 0.140s, 0.080s, dan 0.079s (rata-rata 0.100s) untuk 1000 iterasi. **Meski iterasinya paling banyak, algoritma ini lebih cepat dari sideways move karena hanya mengevaluasi satu neighbor per iterasi, bukan seluruh neighbor**. Durasinya hampir setara dengan random restart dan jauh lebih singkat dibanding genetic algorithm.
Genetic Algorithm	**Durasi paling lama** dan sangat dipengaruhi parameter. Durasi tumbuh hampir linear terhadap populasi × iterasi. Pada konfigurasi terbesar, GA sekitar **200 kali lebih lambat dari steepest ascent** dan sekitar **14 kali lebih lambat dari simulated annealing.**
Pertanyaan 4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	**Paling tidak konsisten**, nilai 180, 165, 230. Hasilnya bergantung sepenuhnya pada state awal karena pencarian berhenti di local optimum terdekat, sehingga satu state awal yang buruk langsung menghasilkan hasil buruk.
Sideways Move Hill Climbing	**Cukup konsisten**, 245, 255, 230. Jauh lebih stabil dari steepest ascent karena plateau bisa dilewati, tetapi masih ada ketergantungan pada state awal.
Stochastic Hill Climbing	**Sama tidak konsistennya dengan steepest ascent**, 180, 165,

	230. Hasil akhirnya identik pada ketiga percobaan, sehingga variasinya mengikuti variasi state awal, bukan sifat acak algoritmanya.
Random Restart Hill Climbing	**Cukup konsisten**, 265, 240, 255. **Konsistensinya jauh lebih baik dari hill climbing biasa** karena ketergantungan pada state awal berkurang.
Simulated Annealing	**Konsisten**, 255, 260, 265. Frekuensi stuck hampir sama di tiap percobaan (18, 17, 19). **Hasilnya hampir tidak terpengaruh oleh state awal** karena pencarian dimulai dengan **eksplorasi luas** pada suhu tinggi.
Genetic Algorithm	**Konsistensi bergantung parameter.** Ketika **populasi kecil** digunakan, **hasilnya tidak konsisten**, namun ketika menggunakan **populasi yang lebih besar**, hasilnya **sangat konsisten**. Hal ini disebabkan populasi yang lebih besar menjaga keragaman genetik sehingga tidak stuck di local optima.

Pertanyaan 5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Penjelasan

Berdasarkan hasil eksperimen algoritma genetika menggunakan **variasi populasi dan iterasi**, didapatkan bahwa **semakin banyak iterasi** yang dilakukan dan **semakin besar ukuran populasi**, cenderung **menghasilkan optimalisasi pada hasil algoritma ini**. Jumlah generasi yang banyak memungkinkan proses evolusi berjalan lebih lama, sehingga individu - individu dalam populasi memiliki kesempatan lebih besar untuk berekombinasi dan menghasilkan solusi yang lebih baik.

Sementara itu, **ukuran populasi yang lebih besar** menjaga **keragaman genetik dan memperluas ruang pencarian,** sehingga **risiko terjebak di local optima akan semakin kecil**. Hal ini juga **selaras dengan waktu / durasi eksekusi yang berbanding lurus**

Kesimpulan:
a. Populasi besar meningkatkan kualitas dan stabilitas
b. Iterasi di atas 50 pada populasi 50 tidak memberi perbaikan, dan
c. Biaya waktu tumbuh sekitar linear terhadap populasi × iterasi.

BAB IV
Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan keseluruhan hasil eksperimen, dapat disimpulkan bahwa untuk permasalahan kendaraan dalam kapal yang diuji, algoritma Hill Climbing with Sideways Move, Random Restart Hill Climbing, Simulated Annealing, dan Genetic Algorithm menunjukkan performa yang baik dan konsisten dalam menemukan solusi optimal.

Sebaliknya, varian Hill Climbing dasar seperti Stochastic Hill Climbing dan Steepest Ascent Hill Climbing terbukti tidak konsisten. Kedua algoritma dasar ini beberapa kali gagal mencapai global optima dan terjebak di local optima, yang menunjukkan ketergantungan besar pada kondisi state awal. Konsistensi pada algoritma Sideways Move, Random Restart, Simulated Annealing, dan Genetic Algorithm disebabkan oleh adanya mekanisme internal untuk lolos dari jebakan local optima, baik melalui sideways move, restart acak, penerimaan solusi buruk secara probabilistik, maupun eksplorasi paralel berbasis populasi.

Dari segi durasi pencarian, terdapat perbedaan signifikan. Varian Hill Climbing (khususnya Sideways Move dan Random Restart) adalah yang tercepat. Steepest Ascent secara konsisten lebih lambat daripada varian HC lainnya karena harus mengevaluasi semua tetangga di tiap iterasi. Simulated Annealing membutuhkan waktu lebih lama sebagai konsekuensi dari cooling schedule yang sengaja dibuat gradual untuk eksplorasi yang lebih baik. Genetic Algorithm adalah yang paling lambat dan durasinya paling variatif, karena harus memproses seluruh populasi di setiap generasi. Analisis parameter Genetic Algorithm mengonfirmasi bahwa penambahan jumlah generasi (iterasi) dan ukuran populasi meningkatkan konsistensi dan kualitas solusi, namun dengan konsekuensi peningkatan durasi pencarian yang signifikan.

4.2 Saran

Berdasarkan percobaan atau eksperimen yang telah dilakukan, pemilihan algoritma sebaiknya disesuaikan dengan prioritas masalah:

a. Jika **prioritas utama adalah kecepatan dan solusi** "cukup baik" sudah memadai, Random Restart Hill-Climbing atau Hill-Climbing with Sideways Move adalah pilihan optimal karena eksekusinya cepat dan hasilnya konsisten.
b. Jika **kualitas dan keandalan solusi lebih diutamakan** meski durasi pencarian yang lebih lama dapat diterima, maka Simulated Annealing atau Genetic Algorithm lebih direkomendasikan.

c. Pengembangan Genetic Algorithm dapat difokuskan pada **penyesuaian mutation rate, ukuran populasi, dan jumlah generasi** untuk mencapai keseimbangan antara kualitas solusi dan waktu pencarian.
d. Untuk **meningkatkan efisiensi** Stochastic dan Steepest Ascent Hill-Climbing, disarankan menerapkan kriteria berhenti berbasis stagnasi selain batas iterasi maksimum.

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

rLampiran

1. Pembagian tugas tiap anggota kelompok

Nama	NIM	Tugas
Nadine Arindy Octavia	18224012	1. main.py 2. genetic.py 3. stochastic.py 4. exp_genetic.py 5. exp_stochastic.py 6. visualisasi.py 7. Mengisi dokumen bagian BAB II Implementasi Algoritma Local Search 8. Mengisi kesimpulan dan saran
Anisa Aulia Alhaqi	18224080	1. model.py 2. constraints.py 3. neighbors.py 4. objectives.py 5. state.py 6. exp_simulated_annealing.py 7. simulated_annealing.py 8. sideways_move.py 9. random_restart.py
Riko Satriya Giovanni	18224108	1. steepest_ascent.py 2. random_restart.py 3. exp_steepest_ascent.py 4. exp_random_restart.py 5. visualisasi.py 6. Mengisi dokumen bagian BAB III Hasil dan Analisis Eksperimen 7. Mengisi kesimpulan dan saran

2. Deklarasi Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : 1. Nadine Arindy Octavia/18224012
2. Anisa Aulia Alhaqi/18224080
3. Riko Satriya Giovanni/18224108
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI-K)
Kode Mata Kuliah : IF3070
Nama Mata Kulia : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Nadine A. Octavia
18224012



Anisa A. Alhaqi
18224080



Riko S. Giovanni
18224108

3. Template Pewarnaan

Untuk melihat lebih jelas, dapat menggunakan link ini

